

CivilierERP Documentation

Finance Module

Type: Reference + Explanation

The Finance module manages all monetary activity in CivilierERP — outgoing payments, incoming receipts, bank reconciliation, and financial reporting. It is the authoritative source for cash flow data across projects and companies.

Overview

The Finance module consists of five pages, a dashboard, and a backend API route. All pages are accessible from the Finance sidebar and are protected by role-based permission checks.

Page	Path	Purpose
Finance Dashboard	/finance	Real-time KPI overview of all financial activity
Payment	/payments	Record and manage outgoing payments (<code>NewPayment</code>)
Received Payment	/received-payments	Record and manage incoming payments (<code>ReceivedPayment</code>)
BRS	/brs	Bank Reconciliation Statement — clear/unclear entries against passbook
Trial Balance	/transactions	Unified ledger view across payments and purchase orders
Records	/records	Document archive with file attachments for finance entries

Navigation Structure

The Finance sidebar (`FinanceSidebar.ts`) groups nav items into three sections:

Proceeding	→ /finance	(Dashboard)
Transaction		
└─ Payment	→ /payments	

```
└─ Received Payment → /received-payments
└─ BRS → /brs
Query
└─ Trial Balance → /transactions
```

Finance Dashboard

File: `src/pages/finance/FinanceDashboard.tsx` **API:** `GET /api/finance-dashboard`

What It Does

The dashboard fetches seven aggregates in a single parallel request and renders them as stat cards, summary tables, and a cheque breakdown panel. Data is cached for 60 seconds and auto-refreshes every 2 minutes.

API Response Shape

```
interface FinanceDashboardData {
  paymentsMade: {
    totalCount: number;
    todayCount: number;
    totalAmount: number; // INR, all-time
    todayAmount: number; // INR, today only
  };
  receivedPayments: {
    totalCount: number;
    todayCount: number;
    totalAmount: number;
    todayAmount: number;
    approvedCount: number;
    draftCount: number;
  };
  cheques: {
    totalCount: number;
    pendingCount: number;
    draftCount: number;
    clearedCount: number;
  };
  cards: {
    totalCount: number;
    activeCount: number;
  };
}
```

```

    inactiveCount: number;
  };
  banks: {
    totalCount: number;
    activeCount: number;
  };
  recentPaymentsMade: RecentPaymentMade[]; // Last 8 rows
  recentPaymentsReceived: RecentPaymentReceived[]; // Last 8 rows
}

```

Database Sources

Field Group	Table	Key Columns
Payments Made	<code>dbo.NewPayment</code>	<code>PDate</code> , <code>PAmount</code>
Received Payments	<code>dbo.ReceivedPayment</code>	<code>RPDocDate</code> , <code>RPAmount</code> , <code>RPStatus</code>
Cheques	<code>dbo.ChequeMaster</code>	<code>Status</code> (<code>Pending</code> , <code>Draft</code> , <code>Cleared</code> , <code>Used</code>)
Cards	<code>dbo.card_master</code>	<code>status</code> (1 = active, 0 = inactive)
Banks	<code>dbo.AccountHeadMaster</code>	<code>LHeadType</code> = 'B' , <code>LHeadStatus</code>

Caching

The route uses the shared `cache()` middleware with a 60-second TTL and the key `"finance-dashboard"`. The frontend mirrors this with `staleTime: 60_000`.

```

// Backend
router.get("/", cache("finance-dashboard", 60), async (req, res)
⇒ { ... });

// Frontend
useQuery({
  queryKey: ["financeDashboard"],
  staleTime: 60_000,
  refetchInterval: 2 * 60_000,
});

```

Shape Normalisation

The frontend `normalise()` function handles backward compatibility with the old backend shape (which used `payments`, `purchaseOrders`, `grns`). Always call `normalise(rawData)` before rendering.

```
function normalise(raw: any): FinanceDashboardData {
  const paymentsMade = raw.paymentsMade ?? {
    totalCount: raw.payments?.totalCount ?? 0,
    // ...fallback fields
  };
  return { paymentsMade, receivedPayments: raw.receivedPayments
    ?? EMPTY_DATA.receivedPayments, ... };
}
```

Permission Guard

```
router.use(checkPermissionForMethod("Finance", "Dashboard"));
```

Only users with the `Finance > Dashboard` permission can call this route. Unauthorized requests receive a `403` response before any SQL executes.

Payment (Outgoing)

File: `src/pages/finance/Payment.tsx` **Table:** `dbo.NewPayment`

What It Does

The Payment page records money going out of the organization. Each payment is linked to an expense booking, a bank account, a project, and a company. The approval workflow integrates with `ApprovalActions`.

Payment Modes

Mode	Additional Fields
Cash	None
Check	<code>PChequeNo</code> , <code>PChequeLotId</code> , <code>PChequeDate</code> , <code>PChequeAccountNumber</code> , <code>PChequeIfsc</code> , <code>PIsPostDated</code>
UPI	<code>PUpiTransactionId</code>
NEFT	<code>PNeftNumber</code>
RTGS	<code>PRtgsReference</code>
IMPS	<code>PImpsReference</code>

Mode	Additional Fields
Card	Bank linked via <code>PBankID</code>

Key Database Columns (`dbo.NewPayment`)

Column	Type	Description
<code>PPaymentID</code>	<code>int</code>	Primary key
<code>PPaymentName</code>	<code>nvarchar(200)</code>	Payment description / remarks
<code>PMode</code>	<code>nvarchar(50)</code>	Payment mode (see table above)
<code>PAmount</code>	<code>decimal</code>	Payment amount in INR
<code>PDate</code>	<code>date</code>	Payment date
<code>PBankID</code>	<code>int</code>	FK → <code>AccountHeadMaster</code>
<code>PBankName</code>	<code>nvarchar(200)</code>	Denormalized bank name
<code>PProject</code>	<code>nvarchar(200)</code>	Project reference
<code>PCompany</code>	<code>nvarchar(200)</code>	Company reference
<code>PExpenseRef</code>	<code>nvarchar(100)</code>	Linked expense booking / supplier
<code>DocNo</code>	<code>nvarchar(100)</code>	System-generated document number
<code>Status</code>	<code>nvarchar(50)</code>	<code>Draft</code> , <code>Approved</code> , <code>Rejected</code>

Validation Schema (`paymentBodySchema`)

The backend normalises field aliases before validation. Both legacy (`mode` , `amount` , `bankId`) and canonical (`PMode` , `PAmount` , `PBankID`) field names are accepted.

Required fields:

```
PMode: reqStr(50, "Payment mode is required"),
PAmount: reqNonNegativeNumber("Amount must be non-negative"),
```

All other fields are optional. The schema uses `.passthrough()` so additional metadata is preserved without rejection.

Received Payment (Incoming)

File: `src/pages/finance/ReceivedPayment.tsx` **Table:** `dbo.ReceivedPayment`

What It Does

Records money coming into the organization from customers or other parties. Supports the same payment modes as outgoing payments plus an EMI mode for installment receipts.

Payment Modes

```
type PaymentMode = "Cash" | "Check" | "UPI" | "NEFT" | "RTGS" |  
"Card" | "EMI";
```

Status Lifecycle

```
Draft → (approval) → Approved  
      → (rejection) → Rejected
```

The dashboard shows `approvedCount` and `draftCount` separately so managers can track unreviewed receipts at a glance.

Key Fields

Column	Description
<code>RPPaymentID</code>	Primary key
<code>RPReceivedFrom</code>	Payer name or reference
<code>RPMode</code>	Payment mode
<code>RPAmount</code>	Amount received in INR
<code>RPDocDate</code>	Date of receipt
<code>RPBankName</code>	Bank where funds were received
<code>RPStatus</code>	<code>Draft</code> or <code>Approved</code>

Bank Reconciliation Statement (BRS)

File: `src/pages/finance/Brs.tsx`

What It Does

BRS pulls all bank-linked transactions — both outgoing (`NewPayment`) and incoming (`ReceivedPayment`) — and lets an accountant mark each as **Clear** (verified against the bank passbook) or **Unclear** (still pending). This is the standard process for month-end bank reconciliation.

Filters

Filter	Description
Date range (from / to)	Restricts entries by transaction date
Status	<code>Clear</code> / <code>Unclear</code> / All
Company	Filters by associated company
Bank	Filters by bank account

Actions

Action	Function	Effect
Mark Clear	<code>markClear(id)</code>	Sets entry status to <code>Clear</code> in the database
Mark Unclear	<code>markUnclear(id)</code>	Reverts entry status to <code>Unclear</code>
Export	<code>ExportMenu</code>	Downloads as PDF, XLSX, or CSV

Entry Direction Indicators

Each row shows a direction arrow:

- **↑ Outgoing** — Payment made (red)
- **↓ Incoming** — Payment received (green)

Trial Balance (Transactions)

File: `src/pages/finance/Transactions.tsx`

What It Does

Displays a unified, read-only ledger across all financial transaction types. Each row shows the transaction ID, date, type, party, amount, payment mode, status, and the user who created it.

Transaction Types

Type	Source
Payment	<code>dbo.NewPayment</code>
Purchase Order	<code>dbo.PurchaseOrderMaster</code>
Receipt	<code>dbo.ReceivedPayment</code>
Journal	Journal entries

Type	Source
Contra	Inter-account transfers

Status Colours

Status	Colour
Completed / Approved	Green
Pending	Yellow
Draft	Muted grey
Rejected	Red
Posted	Primary blue

Summary Strip

The page renders a KPI strip above the table:

```
interface Summary {
  totalPayments: number; // Sum of all outgoing payments
  totalPOs: number; // Sum of all purchase orders
  netCashFlow: number; // totalPayments - totalPOs
  pendingCount: number; // Entries with Pending status
}
```

Records

File: `src/pages/finance/Records.tsx`

What It Does

Provides a document archive view of all finance entries — payments, expenses, and receipts — with support for file attachments. Users can attach PDFs, images, or spreadsheets to any record directly from this page.

Entry Types

Type	Badge Colour
Payment	Blue
Expense	Red
Receipt	Green

File Attachment

Each row has an upload cell. The component accepts any file and stores it via the `useRecords` hook. File type icons auto-detect from MIME type:

MIME Type	Icon
<code>image/*</code>	Image icon (blue)
<code>*/pdf</code>	PDF icon (red)
<code>*sheet*</code> / <code>*excel*</code> / <code>*csv</code>	Spreadsheet icon (green)
Other	Generic file icon

Validation Schemas

File: `backend/validation/financialRouteSchemas.js`

The backend uses `Zod` for all request body validation. The module exports four schemas:

Schema	Used By
<code>purchaseOrderBodySchema</code>	POST <code>/api/purchase-orders</code>
<code>purchaseOrderUpdateSchema</code>	PUT <code>/api/purchase-orders/:id</code>
<code>grnBodySchema</code>	POST <code>/api/grns</code>
<code>paymentBodySchema</code>	POST/PUT <code>/api/payments</code>

Helper Primitives

Helper	Behaviour
<code>optStr(max)</code>	Optional string, trims whitespace, max length <code>max</code>
<code>reqStr(max, msg)</code>	Required string, min 1 char, max <code>max</code> , custom error <code>msg</code>
<code>optInt</code>	Optional positive integer, coerced from string
<code>reqInt(msg)</code>	Required positive integer
<code>optNumber</code>	Optional finite number
<code>reqNonNegativeNumber(msg)</code>	Required number ≥ 0
<code>optDate</code>	Optional date, coerced

Helper	Behaviour
<code>reqDate(msg)</code>	Required date

Empty strings are treated as `undefined` by the `emptyToUndefined` preprocessor applied to all optional fields. This means the frontend can safely send empty strings for optional fields without triggering validation errors.

Currency Formatting

All monetary values in the Finance module use the `formatINR` utility and the `fmt` helper:

```
const fmt = (n: number) =>
  new Intl.NumberFormat("en-IN", {
    style: "currency",
    currency: "INR",
    maximumFractionDigits: 0,
  }).format(n);
```

This outputs values in the Indian numbering format — e.g., `₹12,50,000` — with no decimal places for whole rupee amounts.

Common Pitfalls

Dashboard shows stale data after a new payment is entered. The dashboard cache TTL is 60 seconds. Either wait for auto-refresh or click the **Refresh** button. For immediate updates after a write, call `queryClient.invalidateQueries(["financeDashboard"])` from the Payment/ReceivedPayment page after a successful mutation.

BRS entries not appearing for a bank. Entries only appear in BRS if they have a `PBankID` / `RPSBankID` set. Payments created without a bank selection will not show in the reconciliation view.

`paymentBodySchema` **rejecting a valid payload.** Check for empty string values on required fields. The schema treats `""` as missing. Also confirm `PMode` is one of the accepted mode strings — the schema enforces a max length of 50 but does not restrict to an enum, so typos pass validation but may break downstream logic.

REDIS_PASSWORD error on startup. The Finance Dashboard route depends on the `cache()` middleware which requires Redis. Ensure **REDIS_PASSWORD** is set in `backend/.env` before starting the server.

Permissions

All Finance routes are guarded by:

```
router.use(checkPermissionForMethod("Finance", "<SubModule>"));
```

The sub-module name must match an entry in `dbo.PostApprovalRights` for the user's role. Contact the system administrator to grant Finance module access.

Finance Module — Setup

The Finance Setup panel (accessible via the ⚙ Setup button in the Finance module header) provides nine master configuration pages. These masters must be configured before the transactional pages (Payment, Received Payment, BRS) can be used effectively.

Setup Item	Path	Table	Purpose
AC Group	<code>/masters/account-group</code>	<code>dbo.AccountGroupMaster</code>	Hierarchical chart of account groupings
General Ledger	<code>/masters/general-ledger</code>	<code>dbo.AccountHeadMaster</code> (<code>LHeadType='L'</code>)	Individual ledger under account groups
Suppliers	<code>/masters/suppliers</code>	<code>dbo.AccountHeadMaster</code> (<code>LHeadType='S'</code>)	Vendor/director
Contractors	<code>/masters/contractors</code>	<code>dbo.AccountHeadMaster</code> (<code>LHeadType='C'</code>)	Contractor/director categorization
Banks	<code>/masters/banks</code>	<code>dbo.AccountHeadMaster</code> (<code>LHeadType='B'</code>)	Company account
Fin Year	<code>/masters/fin-year</code>	<code>dbo.FinancialYear</code>	Financial periods

Setup Item	Path	Table	Purpose
			lock co
Cheque	/masters/cheque	dbo.ChequeMaster	Cheque lot/boo registr
Card	/masters/card	dbo.card_master	Corpora registr with ex alerts
TDS	/masters/tds	dbo.TDSMaster	TDS nature/ definiti

Important: All of Suppliers, Contractors, Banks, and General Ledger write to the same underlying table — `dbo.AccountHeadMaster` — distinguished by the `LHeadType` discriminator column (`S` , `C` , `B` , `L`).

AC Group (Account Group Master)

File: `src/pages/masters/AccountGroupMaster.tsx`

Defines the hierarchical chart of accounts. Groups can be nested — a group can have a `parentId` pointing to another group, building a tree structure. The General Ledger master then assigns individual ledger heads to these groups.

Fields:

Field	Required	Description
Name	Yes	Group name (e.g. "Current Assets", "Direct Expenses")
Code	No	Short alphanumeric code for the group
Parent Group	No	Parent group — leave empty for a root-level group

Behaviour: The page renders as an expandable tree, not a flat list. Deleting a group also cascades to all its descendant groups — the UI computes descendants before confirming deletion.

General Ledger Master

File: `src/pages/masters/GeneralLedgerMaster.tsx` **Table:**
`dbo.AccountHeadMaster` where `LHeadType = 'L'`

Individual ledger heads (accounts) that sit under an Account Group. Every payment and receipt must be linked to a ledger head.

Fields:

Field	Column	Required	Description
Ledger Name	<code>LHeadName</code>	Yes	Name of the ledger account
Code	<code>LHeadCode</code>	No	Short reference code
Account Group	<code>LBelongsTo</code>	No	FK → <code>AccountGroupMaster._id</code>
Status	<code>LHeadStatus</code>	—	Active / Inactive toggle

Suppliers

File: `src/pages/masters/SupplierMaster.tsx` **Table:**
`dbo.AccountHeadMaster` where `LHeadType = 'S'`

The supplier directory. Suppliers appear as selectable options in Purchase Orders and GRNs.

Fields:

Field	Required	Notes
Supplier Name	Yes	
Contact Person	No	
Phone Number	No	
Email Address	No	
GST Number	No	Auto-uppercased
PAN Number	No	Auto-uppercased
Supplier Category	No	<code>Goods</code> / <code>Services</code> / <code>Both</code>
GST Type	No	<code>Regular</code> , <code>Composition</code> , <code>Unregistered</code> , <code>SEZ</code> , <code>Deemed Export</code>
GST State	No	Full list of Indian states and UTs
Opening Balance	No	
Address	No	Textarea

Field	Required	Notes
Status	—	Active toggle

Contractors

File: `src/pages/masters/ContractorMaster.tsx` **Table:** `dbo.AccountHeadMaster` where `LHeadType = 'C'`

The contractor directory. Contractor Type is a dynamic dropdown sourced from the Contractor Category admin page — it is not a hardcoded enum.

Fields:

Field	Required	Notes
Contractor Name	Yes	
Contact Person	No	
Phone Number	No	
Email Address	No	
GST Number	No	Auto-uppercased
PAN Number	No	Auto-uppercased
Contractor Type	No	Dynamic — sourced from <code>dbo.ContractorCategory</code>
Payment Terms	No	Free text
Address	No	Textarea
Status	—	Active toggle

Banks

File: `src/pages/masters/BankMaster.tsx` **Table:** `dbo.AccountHeadMaster` where `LHeadType = 'B'`

Company bank accounts. Every outgoing and incoming payment can be linked to a bank registered here. The IFSC code is validated against the standard Indian format.

Fields:

Field	Required	Validation
Company	No	Dropdown from enterprise master
Bank Name	Yes	
Branch	No	
Account Number	Yes	
IFSC Code	Yes	Must match <code>^[A-Z]{4}0[A-Z0-9]{6}\$</code> (e.g. <code>SBIN0001234</code>)
Account Type	No	e.g. Current, Savings
Bank Type	No	
Account Holder Name	No	
Opening Balance	No	Must be ≥ 0
Address	No	
Status	—	Active toggle

IFSC Validation Rule: 4 uppercase letters + `0` + 6 alphanumeric characters. The field auto-uppercases input before validation.

Financial Year (Fin Year)

File: `src/pages/masters/FinancialYearMaster.tsx` **Table:** `dbo.FinancialYear`

Defines the financial year periods used throughout the system. Document numbering, reports, and the `FinYearContext` all depend on the active financial year.

Fields:

Field	Column	Required	Description
Year Label	<code>FName</code>	No	Display name e.g. <code>2024-25</code>
Start Date	<code>FStartDate</code>	No	Beginning of the financial year
End Date	<code>FEndDate</code>	No	End of the financial year
Status	<code>FStatus</code>	—	<code>Active</code> or <code>Closed</code>
Locked	<code>FisLocked</code>	—	When <code>true</code> , no new transactions can be posted to this year

Lock behaviour: A locked financial year prevents further document creation. The lock is a hard boolean — it cannot be set through the Payment form, only through this master page.

Cheque Master

File: `src/pages/masters/ChequeMaster.tsx` **Table:** `dbo.ChequeMaster`

Registers cheque books (lots) issued by a bank. Each lot defines a sequential range of cheque numbers. When a cheque payment is made, the system picks from an available cheque in the registered lot.

Fields:

Field	Required	Notes
Company	Yes	Dropdown
Bank	Yes	Filtered by company
Account Number	Yes	Auto-filled from selected bank
IFSC Code	—	Auto-filled from selected bank
Lot Number	Yes	Unique identifier for the cheque book
Cheque Start No.	Yes	Must be a positive integer
Cheque End No.	Yes	Must be $\geq$ Cheque Start No.
Total Cheques	—	Auto-calculated: <code>End - Start + 1</code>
Remarks	No	Max 500 characters
Status	—	Active toggle

Validation rule: `chqEnd  $\geq$  chqStart` — the schema enforces this at the form level and rejects with "End must be greater than or equal to start."

Card Master

File: `src/pages/masters/CardMaster.tsx` **Table:** `dbo.card_master`

Registers corporate debit/credit cards linked to a bank account. Supports expiry reminders — the system can alert users a configurable number of days before the card expires.

Fields:

Field	Required	Notes
Company	No	Dropdown
Bank	Yes	Dropdown
Account Number	—	Auto-filled from bank
IFSC Code	—	Auto-filled from bank
Network	No	e.g. Visa, Mastercard, RuPay
Card Type	No	e.g. Debit, Credit
Card Holder Name	No	Max 150 characters
Card Number	Yes	13–16 digits; non-digits stripped automatically
CVV	Yes	3–4 digits
Expiry Date	Yes	Format MM/YY – validated by regex <code>^(0[1-9] \1[0-2])\/\d{2}\$</code>
Reminder Enabled	—	Toggle to activate expiry reminder
Reminder Days	If enabled	Number of days before expiry to trigger alert; minimum 1
Status	—	Active (1) / Inactive (0) toggle

Security note: CVV is stored as entered. Card numbers are masked in the list view – only the last 4 digits are displayed. A show/hide toggle reveals the full number in the form.

TDS Master

File: `src/pages/masters/TdsMaster.tsx` **Table:** `dbo.TDSMaster`

Defines TDS (Tax Deducted at Source) nature and rate combinations. These entries are referenced during payment processing when TDS deduction is applicable.

Fields:

Field	Column	Required	Description
Nature	<code>Nature</code>	No	TDS section description (e.g. <code>194C - Contractor</code>)
Name	<code>Name</code>	No	Short identifier name

Field	Column	Required	Description
Percentage	Percentage	No	TDS rate as a decimal (e.g. 2 for 2%); defaults to 0
Status	Status	—	Active toggle

Usage: TDS entries appear as selectable deduction rates during expense bookings and payment creation. A Percentage of 0 is valid — it represents a nil-rate TDS certificate scenario.

Material Module

Type: Reference + Explanation

The Material module manages the complete procurement and inventory lifecycle in CivilierERP — from material requests through purchase orders, goods receipt, stock movement, expense booking, and stock transfers. It is the primary source of item, stock, and procurement data across all projects and companies.

Overview

The Material module consists of seven transaction pages, a dashboard, and seven setup masters.

Navigation Structure

Proceeding	→ /material	(Dashboard)
Transaction		
— Material Request	→ /material/material-request	
— Purchase Order	→ /material/purchase-order	
— GRN	→ /material/grn	
— Issues	→ /material/issues	
— Expense Booking	→ /material/expense-booking	
Stock	→ /material/stock	
Transfer	→ /material/stock-transfer	
Debit Note	→ /material/debit-note	
Amendment Menu	→ /material/amendment-menu	

Transaction Flow

Material Request → Purchase Order → GRN → Expense Booking → Payment

↓
Stock Ledger (IN)
↓
Issues (OUT)
↓
Stock Transfer

Material Dashboard

File: `src/pages/material/MaterialDashboard.tsx` **API:** `GET`
`/api/material-dashboard`

What It Does

Fetches 18 parallel SQL aggregates in a single request and renders them as KPI cards, status breakdowns, recent activity tables, and a top-items list. Data is cached for 60 seconds in Redis under the key `material_dashboard`.

API Response Shape

```
{
  items:           { count, groupCount }
  grns:            { total, thisMonth, today, totalValue,
openValue }
  purchaseOrders: { total, open, approved, pending, totalValue,
openValue }
  workOrders:      { total, open, thisMonth, totalValue }
  expenses:        { total, pending, approved, totalAmount,
pendingAmount }
  stock:           { totalEntries, totalIn, totalOut,
uniqueItems }
  uom:             { total }
  materialIssues:  { total, thisMonth, today, totalQty }
  materialRequests: { total, pending, approved, draft, ordered,
partiallyOrdered, thisMonth }
  recentGRNs:      RecentGRN[]           // last 6
  recentPOs:       RecentPO[]             // last 6
  recentWOs:       RecentWO[]             // last 6
  recentExpenses:  RecentExpense[]        // last 6
  recentIssues:    RecentIssue[]          // last 6
  recentRequests:  RecentRequest[]        // last 6
  poStatusBreakdown: { Status, Count, TotalValue }[]
}
```

```

    woStatusBreakdown: { Status, Count, TotalValue }[]
    topItems:           { ItemID, ItemName, TotalIn, TotalOut,
NetStock }[] // top 5
  }

```

Database Sources

Metric	Table	Key Columns
Items & Groups	dbo.Item_Master_Group	Parent_Id , M_IdentityCode
GRNs	dbo.GoodsReceiptNotes	GRNDate , TotalAmount , Status
Purchase Orders	dbo.PurchaseOrders	PODate , TotalAmount , Status
Work Orders	dbo.WorkOrderHeader	CreatedAt , TotalAmount , Status
Expense Bookings	dbo.ExpenseBooking	EAmount , EStatus
Stock	dbo.StockLedger	Type (IN / OUT) , Qty , ItemID
UOM	dbo.UOMMaster	—
Issues	dbo.MaterialIssues + dbo.MaterialIssueItems	CreatedAt , Quantity
Requests	dbo.MaterialRequests	Status , CreatedAt

Error Resilience

Each SQL query is wrapped in a `safeQuery()` helper that catches individual query failures and returns `null` instead of crashing the entire dashboard. Fields default to `0 / []` when a query returns null. This means a broken table never takes down the whole dashboard.

```

const safeQuery = async (sql) => {
  try { return await pool.request().query(sql); }
  catch (err) { console.error("MATERIAL DASHBOARD QUERY ERROR:",
... ); return null; }
};

```

Transactions

Material Request (MR)

File: `src/pages/material/MaterialRequest.tsx` **Backend:** `backend/routes/materialRequests.js` **Tables:** `dbo.MaterialRequests` (header), `dbo.MaterialRequestItems` (line items) **Doc Number Format:** `MR-YYYY-NNNNN`

What It Does

Records a formal request for materials from a site or department. The request goes through an approval workflow before it can be converted to a Purchase Order.

Header Fields

Field	Column	Required	Description
Document No.	<code>DocNo</code>	Auto	System-generated <code>MR-YYYY-NNNNN</code>
Company	<code>CompanyId</code>	No	FK → <code>dbo.enterprise</code>
Project	<code>ProjectId</code>	No	FK → <code>dbo.enterprise</code>
Financial Year	<code>FinYearId</code>	No	FK → <code>dbo.FinancialYear</code>
Request Date	<code>RequestDate</code>	Yes	Defaults to today
Required By	<code>RequiredByDate</code>	No	Deadline for the request
Priority	<code>Priority</code>	No	<code>Normal</code> , <code>High</code> , <code>Urgent</code>
Reason	<code>Reason</code>	Yes	Justification for the request
Remarks	<code>Remarks</code>	No	Additional notes

Line Item Fields (`MRLineItem`)

Field	Required	Description
Item	Yes	FK → <code>dbo.Item_Master_Group</code>
UOM Code	Yes	FK → <code>dbo.UOMMaster</code>
Quantity	Yes	Requested quantity
Remarks	No	Per-item notes

Status Flow

Draft → Pending → Approved | Rejected



Ordered / Partially Ordered (set by PO creation)

Purchase Order (PO)

File: `src/pages/material/PurchaseOrderMaster.tsx` **Backend:** `backend/routes/purchaseOrders.js` **Tables:** `dbo.PurchaseOrders`, `dbo.POItems` (JSON blob) **Permission:** `Material > PurchaseOrders`

What It Does

Creates a formal purchase order against a supplier. Can be raised from scratch or prefilled from an approved Material Request. Supports multi-item line entries with GST computation.

Key Fields

Field	Column	Required	Notes
PO Number	<code>PurchaseOrderNo</code>	Auto or manual	From <code>DocNumberSequence</code>
PO Date	<code>PODate</code>	No	
Expected Delivery	<code>ExpectedDeliveryDate</code>	No	
Supplier	<code>SupplierID</code>	No	FK → <code>AccountHeadMaster</code> (LHeadType='S')
Company	<code>CompanyId</code>	No	
Project	<code>ProjectId</code>	No	
Items	<code>POItems</code>	No	JSON array of line items
GST	<code>GST</code>	No	JSON blob: <code>{ hsnCode, type, rate }</code>
Discount	<code>Discount</code>	No	JSON blob
Total Amount	<code>TotalAmount</code>	—	Computed: subtotal + GST - discount
Payment Terms	<code>PaymentTerms</code>	No	Free text
Status	<code>Status</code>	—	<code>Draft</code> , <code>Pending</code> , <code>Approved</code> , <code>Closed</code> ,

Field	Column	Required	Notes
			Rejected
P0 Type	P0Type	No	
Source MR	SourceMRId , SourceMRDocNo	No	If raised from a Material Request
Source WO	SourceWOId , SourceWODocNo	No	If raised from a Work Order

GST Computation

The backend extracts scalar GST fields from the JSON blob for indexed queries:

```
const extractGstScalars = (gstVal) => ({
  hsnCode: obj.hsnCode || null,
  gstType: obj.type || null,
  gstRate: obj.rate !== null ? parseFloat(obj.rate) : null,
});
```

The `TotalAmount` stored is **inclusive of GST**: $\text{subtotal} + (\text{subtotal} \times \text{gstRate} / 100) - \text{discount}$.

GRN (Goods Receipt Note)

File: `src/pages/material/GRN.tsx` **Backend:** `backend/routes/grns.js`
Tables: `dbo.GoodsReceiptNotes`, `dbo.StockLedger` **Permission:** `Material > GRN`

What It Does

Records the physical receipt of goods from a supplier against a Purchase Order. On creation, the backend automatically writes `IN` entries to `dbo.StockLedger` for each received line item, updating available stock.

Key Fields

Field	Column	Required	Notes
GRN No.	GRNNo	Auto	From <code>DocNumberSequence</code>
GRN Date	GRNDate	Yes	
Supplier	SupplierID	Yes	FK → <code>AccountHeadMaster</code>

Field	Column	Required	Notes
P0 Reference	POID	No	FK → <code>dbo.PurchaseOrders</code>
GRN Items	GRNItems	No	JSON array of line items
Total Amount	TotalAmount	—	Computed incl. GST
Status	Status	—	<code>Draft</code> , <code>Pending</code> , <code>Approved</code> , <code>Closed</code> , <code>Rejected</code>
Remarks	Remarks	No	
Doc Type	DocTypeId	No	From <code>DocNumberSequence</code>
Parent Doc No	ParentDocNo	No	Links to source Expense Booking
Root ExB Doc No	RootExBDocNo	No	Root Expense Booking chain reference

GRN Line Item Computation

```
// Per line: base = rate × qty (or stored totalAmount if > 0)
// GST = base × (gstPct / 100)
// Line total = base + GST
function computeGRNTotal(grnItems) {
  return items.reduce((sum, item) => {
    const base = Number(item.totalAmount) > 0
      ? Number(item.totalAmount)
      : Number(item.rate || 0) * Number(item.quantity || 0);
    const gstAmt = base * (Number(item.gstPct || 0) / 100);
    return sum + base + gstAmt;
  }, 0);
}
```

Stock Ledger Side Effect

On every successful GRN save, one `IN` row is written to `dbo.StockLedger` per line item:

```
StockLedger.Type = 'IN'
StockLedger.Qty = receivedQty
StockLedger.ItemID = item.itemId
```

GRN Item Normalisation

The backend always returns `GRNItems` as a parsed array, never as a raw JSON string. The `normaliseGRNRow()` function handles this before sending to the client, preventing double-parse bugs.

Issues (Material Issue)

File: `src/pages/material/Issues.tsx` **Backend:** `backend/routes/materialIssues.js` **Tables:** `dbo.MaterialIssues` (header), `dbo.MaterialIssueItems` (line items), `dbo.StockLedger`

What It Does

Records the movement of stock out of a godown to a project or site. Each issue writes `OUT` entries to `dbo.StockLedger`, reducing available stock. Stock balance is checked before issue to prevent negative inventory.

Document Numbering

Context	Format
Normal issue	<code>ISS-YYYY-NNNNN</code>
Issue under Expense Booking	<code>ExB-ISS-YYYY-NNNNN</code>

Header Fields

Field	Column	Required	Description
Issue No.	<code>IssueNo</code>	Auto	System-generated
Doc No.	<code>DocNo</code>	Auto	From <code>DocNumberSequence</code>
Date	<code>Date</code>	Yes	Issue date
Company	<code>CompanyId</code>	No	
Project	<code>ProjectId</code>	No	
Financial Year	<code>FinYearId</code>	No	
Reason	<code>Reason</code>	No	
Status	<code>Status</code>	—	Approval workflow status
Root ExB Doc No	<code>RootExBDocNo</code>	No	Links to parent Expense Booking

Line Item Fields

Each line item represents one item being issued:

Field	Description
Item	FK → <code>dbo.Item_Master_Group</code>
UOM	Unit of measurement
Quantity	Quantity to issue
Remarks	Per-line note

Stock OUT Side Effect

On each issue, the system writes to `dbo.StockLedger` :

```
StockLedger.Type = 'OUT'
StockLedger.ItemID = item.itemId
StockLedger.Qty = quantity
```

The main godown (`dbo.Godowns WHERE IsMain = 1`) is resolved automatically as the source godown if not specified.

Expense Booking

File: `src/pages/material/MaterialExpenseBooking.tsx` **Table:** `dbo.ExpenseBooking`

What It Does

Books a financial expense against a GRN, linking material receipt to accounts payable. Supports EMI/installment payment schedules. The expense document chains to its source GRN via `ParentDocNo` and `RootExBDocNo`, creating a traceable document chain: `PO → GRN → Expense Booking → Payment`.

Key Fields

Field	Column	Description
Doc No .	<code>EDocNo</code>	Auto-generated
Doc Date	<code>EDocDate</code>	Booking date
Amount	<code>EAmount</code>	Total expense amount
Document Type	<code>EDocumentType</code>	Type from <code>DocNumberSequence</code>
Project Name	<code>EProjectName</code>	Denormalized project reference

Field	Column	Description
Status	EStatus	Draft , Pending , Approved , Rejected
GRN Reference	via ParentDocNo	Source GRN document
EMI Schedule	EmiSection	Optional installment breakdown

EMI Support

When payment is split into installments, the EmiSection component manages the schedule. Each installment has a due date and amount. The total of all installments must equal EAmount .

Document Chain

```

PO (PurchaseOrderNo)
├ GRN (ParentDocNo → PO, RootExBDocNo)
│   └ Expense Booking (ParentDocNo → GRN)
│       └ Payment (PExpenseRef → ExB)
│           └ Issues (RootExBDocNo → ExB)

```

Stock

File: src/pages/material/Stock.tsx **Backend:** backend/routes/stockLedger.js **Table:** dbo.StockLedger

What It Does

Read-only view of all stock ledger entries. Shows the full movement history for each item — every IN (from GRNs) and every OUT (from Issues and Transfers). Net stock is computed as TotalIn - TotalOut .

Key Columns Displayed

Column	Description
Item	Item name from Item_Master_Group
Type	IN or OUT
Qty	Quantity moved
Date	Movement date
Reference	Source document (GRN No., Issue No., Transfer No.)

Column	Description
Godown	Source/destination godown

Stock Transfer

File: `src/pages/material/StockTransfer.tsx` **Backend:** `backend/routes/stockTransfers.js` **Table:** `dbo.StockTransfers`

What It Does

Moves stock between godowns atomically. The backend writes paired **OUT** + **IN** rows to `dbo.StockLedger` in a single transaction, with a stock balance check before the transfer to prevent negative inventory. Redis cache for stock data is busted after every successful transfer.

Key Fields

Field	Column	Required	Description
Doc No.	<code>DocNo</code>	Auto	System-generated
Transfer Date	<code>TransferDate</code>	Yes	
From Godown	<code>FromGodownID</code>	Yes	FK → <code>dbo.Godowns</code>
To Godown	<code>ToGodownID</code>	Yes	FK → <code>dbo.Godowns</code> — must differ from source
Items	<code>TransferItems</code>	Yes	JSON array of <code>{ itemId, quantity, uom }</code>
Remarks	<code>Remarks</code>	No	
Status	<code>Status</code>	—	Transfer status

Filters

The list view supports filtering by `fromGodown` and `toGodown` via query parameters. Pagination is supported with `limit` and `page` params (default: 100 per page).

Race Condition Safety

The backend checks available stock per item before writing the transfer. If stock is insufficient for any line item, the entire transfer is rejected — no partial transfers occur.

Material Module — Setup

The Material Setup panel provides seven master configuration pages. Items, UOM, and HSN must be configured before any transactions can be created.

Setup Item	Path	Table	Purpose
Items	/masters/items	dbo.Item_Master_Group (M_IdentityCode=1)	Individual purchas items
Items Group	/masters/item-groups	dbo.Item_Master_Group (M_IdentityCode=0)	Item can grouping
Unit of Measurement	/masters/uom	dbo.UOMMaster	Units us items
HSN	/masters/hsn	dbo.HsnMaster	HSN cod rates
Billing	/masters/billing-terms	dbo.BillingTermsMaster	Billing to template
T&C	/masters/tc	dbo.TCMaster	Terms & text ten
Inventory	/masters/inventory	dbo.Godowns	Warehou location

Important: Both Items and Items Groups write to the same table — `dbo.Item_Master_Group` — distinguished by `M_IdentityCode`: `0` = Group, `1` = Item. Items also carry a `Parent_Id` pointing to their group.

Items

File: `src/pages/masters/ItemMaster.tsx` **Table:** `dbo.Item_Master_Group` where `M_IdentityCode = 1`

Defines the individual items that appear in Material Requests, POs, GRNs, and Issues. Each item can carry its own GST rates sourced from an HSN code, or overridden manually.

Fields:

Field	Column	Required	Notes
Item Name	<code>M_Name</code>	Yes	
Description	<code>M_Description</code>	No	

Field	Column	Required	Notes
Short Code	M_code	No	Quick reference code
Item Type	M_Type	No	Goods or Service
Item Group	Parent_Id	No	FK → Item_Master_Group (group row)
HSN Code	M_HSN	No	Auto-fills CGST/SGST/IGST when selected
CGST %	M_CGST	No	Overrides HSN rate if set directly
SGST %	M_SGST	No	
IGST %	M_IGST	No	
Default UOM	M_UOM	No	FK → dbo.UOMMaster
Default Supplier	default_supplier_id	No	FK → AccountHeadMaster (LHeadType='S')

HSN Auto-fill: Selecting an HSN code automatically populates `cgst`, `sgst`, and `igst` fields from the HSN master rates. These can be overridden per item.

Items Group

File: `src/pages/masters/ItemGroupMaster.tsx` **Table:** `dbo.Item_Master_Group` where `M_IdentityCode = 0`

Provides a classification structure for items. Groups appear in item dropdowns to filter the item list.

Fields:

Field	Column	Required	Description
Name	M_Name	Yes	Group name (e.g. "Cement & Concrete")
Description	M_Description	No	
Code	M_code	No	Short reference code

Groups do not carry HSN or GST data — those live on the item level.

`M_BelongsTo` , `M_Group` , `M_HSN` , and all GST fields are set to `null` for group rows.

Unit of Measurement (UOM)

File: `src/pages/material/UnitOfMeasurementMaster.tsx` **Table:** `dbo.UOMMaster`

Defines the units used on every line item across all transactions. Every item, MR line, PO line, GRN line, and issue line requires a valid UOM code.

Fields:

Field	Column	Required	Notes
Unit Code	<code>UOMCode</code>	Yes	Auto-uppercased (e.g. <code>KG</code> , <code>MTR</code> , <code>NOS</code>)
Unit Name	<code>UOMName</code>	Yes	Full name (e.g. "Kilogram")
Symbol	<code>Symbol</code>	Yes	Display symbol (e.g. <code>kg</code> , <code>m</code> , <code>pcs</code>)
Remarks	<code>Remarks</code>	No	Textarea
Active	<code>IsActive</code>	—	Toggle

HSN

File: `src/pages/masters/HsnMaster.tsx` **Table:** `dbo.HsnMaster`

Defines HSN (Harmonised System of Nomenclature) codes with associated GST rates. When an HSN code is selected on an item, the CGST/SGST/IGST rates auto-fill from this master.

Fields:

Field	Column	Required	Description
HSN Code	<code>HCode</code>	No	The HSN code (e.g. <code>25232100</code>)
Description	<code>HDescription</code>	No	Full description of the goods

Field	Column	Required	Description
Short Description	HShortDescription	No	Abbreviated description
CGST %	HCGST	No	Defaults to 0
SGST %	HSGST	No	Defaults to 0
IGST %	HIGST	No	Defaults to 0
Status	HStatus	—	Active toggle

Billing Terms

File: `src/pages/masters/BillingTermsMaster.tsx` **Table:** `dbo.BillingTermsMaster`

Defines reusable billing term templates applied to Purchase Orders. Each term specifies whether it adds to or deducts from the base PO value, and whether it applies before or after GST.

Fields:

Field	Column	Required	Description
Name	Name	Yes	Term name (e.g. "Retention 5%", "Advance 10%")
Description	Description	No	
Calculation Type	CalculationType	No	Before GST or After GST
Deduction Type	DeductionType	No	Addition or Deduction
Active	IsActive	—	Toggle

Deduction Toggle: The UI renders a segmented toggle (Addition / Deduction) instead of a standard dropdown. This controls whether the term adds to or subtracts from the PO total.

T&C (Terms & Conditions)

File: `src/pages/material/T&CMaster.tsx` **Table:** `dbo.TCMaster`

Stores reusable Terms & Conditions text blocks that can be attached to Purchase Orders.

Fields:

Field	Column	Required	Description
Name	Name	No	Short identifier (e.g. "Standard PO T&C")
Terms & Conditions	TermsAndCondition	No	Full T&C text body
Remarks	Remarks	No	Internal notes
Active	isActive	—	Toggle

Inventory (Godowns)

File: `src/pages/material/InventoryMaster.tsx` **Table:** `dbo.Godowns`

Defines physical warehouse and godown locations used in stock tracking and stock transfers. At least one godown must be marked as **Main** — this is the default source godown for material issues.

Fields:

Field	Column	Required	Description
Godown Code	GodownCode	No	Short reference code
Godown Name	GodownName	Yes	
Short Description	ShortDesc	No	
Description	Description	No	
Remarks	Remarks	No	
Company	EnterpriseID	No	FK → <code>dbo.enterprise</code>
Project	ProjectID	No	FK → <code>dbo.enterprise</code>
Active	IsActive	—	Toggle

Main Godown: The `IsMain` flag is set separately (not via the creation form). The system resolves the main godown automatically when creating stock issues if no godown is specified:

```
SELECT TOP 1 GodownID FROM dbo.Godowns WHERE IsMain = 1 AND  
IsDeleted = 0 ORDER BY GodownID
```

Soft Delete: Godowns are never hard-deleted. The `IsDeleted` flag is set to `1` on deletion, keeping historical stock records intact.

Document Numbering

All Material module transactions use the `DocNumberSequence` table with atomic locking to generate sequential, non-duplicating document numbers.

Transaction	Format
Material Request	<code>MR-YYYY-NNNNN</code>
Purchase Order	Configured via <code>DocNumberSequence</code>
GRN	Configured via <code>DocNumberSequence</code> + <code>resolveGRNPrefix</code>
Material Issue	<code>ISS-YYYY-NNNNN</code>
ExB-linked Issue	<code>ExB-ISS-YYYY-NNNNN</code>
Stock Transfer	Configured via <code>DocNumberSequence</code>

The `lockNextDocNumber()` utility acquires a row-level lock on `DocNumberSequence` before incrementing, preventing two concurrent users from receiving the same document number.

Approval Workflow Integration

All major Material transactions pass through the approval service (`approvalService.js`):

Transaction	Module Map Key	Approval Stages
GRN	<code>GRN</code>	Draft → Pending → Approved / Rejected
Purchase Order	<code>PurchaseOrders</code>	Draft → Pending → Approved / Rejected
Material Request	<i>(transition service)</i>	Draft → Pending → Approved / Rejected
Material Issue	<i>(transition service)</i>	Draft → Pending → Approved

`guardEdit()` is called on GRN and PO update routes — it blocks edits to records that are already `Approved` or `Rejected`, preventing post-approval modification.

Common Pitfalls

Stock goes negative after an issue. The issue route performs a balance check before writing `OUT` entries. If the requested quantity exceeds available stock for any line item, the issue is rejected entirely. Check `dbo.StockLedger` net balance for the item before attempting the issue.

GRN items arrive as a raw JSON string instead of an array. The backend's `normaliseGRNRow()` always parses `GRNItems` before sending to the client. If you are querying `dbo.GoodsReceiptNotes` directly (e.g. in a report query), call `JSON_QUERY` or parse the column manually — it is stored as `NVARCHAR(MAX)`.

Item HSN rates don't auto-fill in the Item form. HSN auto-fill depends on the `getHsn()` API returning `cgstRate`, `sgstRate`, and `igstRate` as numbers. If the HSN master has null rates (entered as `0` defaults), the item will show `0%` — not blank. This is correct behaviour.

Stock Transfer fails silently. Transfers require both `FromGodownID` and `ToGodownID` to resolve to valid, non-deleted godowns. If a godown was soft-deleted after a transfer was initiated, the lookup will fail. Verify godown `IsDeleted = 0` and `IsActive = 1` before troubleshooting further.

Dashboard shows stale data after a GRN or transfer. The dashboard caches for 60 seconds. Cache is busted on writes via `bumpCacheVersion()`, but the dashboard key `material_dashboard` is invalidated only on the next request after TTL expiry. Use the Refresh button for immediate updates.

Follow-Up Module

Type: Reference + Explanation

Module Overview

The Follow-Up module manages the full sales and post-sales lifecycle for real estate projects. It covers customer acquisition (Applications → Bookings → Unit Selection), agreement and closure paperwork (Agreements → NOC →

Sales Deed → Handover), construction progress updates, and cross-cutting operational tools (Tasks, Reminders).

Navigation structure (from `FollowupSidebar.ts`):

Group	Pages
Dashboard	<code>/followup</code>
Tasks	Tasks · Follow-Up Log
Reminders	P0 · W0 · CHQ · TDS · GRN Reminders
Sales	Applications · Bookings · Unit Selection · Welcome Calls
Agreement	Agreements
Closure	NOC · Sales Deed · Handover
Construction	Updates
Setup	Block Master · Unit Master · Reminders · Pending Tasks · Customers · Payment Plan

Follow-Up Setup

The **Setup** panel (accessible via the Follow-Up button in the top bar) contains five master configuration screens. These must be set up before the transactional pages are usable. The correct setup order is: Customers → Block Master → Unit Master → Payment Plan → Reminders / Pending Tasks.

1. Block Master

Type: Reference

Description: Defines physical blocks within a project (e.g. Tower A, Wing B). Blocks are the top-level spatial grouping under a project. Every unit must belong to a block.

API Endpoints

Method	Path	Description
<code>GET</code>	<code>/api/block-master</code>	List all blocks (cached 300s)
<code>GET</code>	<code>/api/block-master/projects</code>	Projects dropdown — <code>enterprise</code> where <code>business_type = 'P'</code> and <code>discontinue = 0</code> (cached 600s)

Method	Path	Description
POST	/api/block-master	Create block
PUT	/api/block-master/:id	Update block
DELETE	/api/block-master/:id	Hard delete block

Database Table: `dbo.BlockMaster`

Column	Type	Notes
Id	INT PK	Auto-increment
ProjectId	INT FK	References <code>dbo.enterprise.id</code> where <code>business_type = 'P'</code>
BlockName	NVARCHAR(100)	Required, max 100 chars
IsActive	BIT	Defaults to 1
CreatedBy	INT	Set from <code>req.user.userId</code>
CreatedAt	DATETIME2(3)	Set server-side on insert
UpdatedBy	INT	Set on update
UpdatedAt	DATETIME2(3)	Set on update

The list query joins `dbo.enterprise` on `e.id = b.ProjectId AND e.business_type = 'P'` to retrieve `ProjectName` for display. Results are ordered by project name then block name.

Form Fields

Field	Required	Notes
Project	Yes	Async select — populated from <code>/api/block-master/projects</code>
Block Name	Yes	Free text, <code>NVarChar(100)</code>
Status	No	Toggle, defaults active

Cache Behaviour

All writes call `bumpCacheVersion("block-master")`, which increments a Redis version key. The `cache("block-master", 300)` middleware on `GET /` checks this version on every request; a bumped version causes a cache miss, so the list is always fresh after any mutation.

Common Pitfalls

Deleting a block that has units under it will cause a foreign key error from `dbo.UnitMaster`. Delete or reassign all units in that block first. The DELETE route does a hard delete — there is no soft-delete or `IsDeleted` flag on this table.

2. Unit Master

Type: Reference

Description: Defines individual sellable units within a block (e.g. Flat 101, Shop A-02). Units sit one level below blocks in the hierarchy: Project → Block → Unit.

API Endpoints

Method	Path	Description
GET	<code>/api/unit-master</code>	List all units (cached 300s)
GET	<code>/api/unit-master/projects</code>	Projects dropdown (cached 600s)
GET	<code>/api/unit-master/blocks?projectId=</code>	Active blocks; optionally filtered by <code>projectId</code>
POST	<code>/api/unit-master</code>	Create unit
PUT	<code>/api/unit-master/:id</code>	Update unit
DELETE	<code>/api/unit-master/:id</code>	Hard delete unit

Database Table: `dbo.UnitMaster`

Column	Type	Notes
<code>Id</code>	<code>INT</code> PK	Auto-increment
<code>ProjectId</code>	<code>INT</code> FK	References <code>dbo.enterprise.id</code>
<code>BlockId</code>	<code>INT</code> FK	References <code>dbo.BlockMaster.Id</code>
<code>UnitName</code>	<code>NVARCHAR(100)</code>	Required
<code>IsActive</code>	<code>BIT</code>	Defaults to <code>1</code>
<code>CreatedBy</code>	<code>INT</code>	
<code>CreatedAt</code>	<code>DATETIME2(3)</code>	
<code>UpdatedBy</code>	<code>INT</code>	
<code>UpdatedAt</code>	<code>DATETIME2(3)</code>	

The list query joins both `dbo.enterprise` and `dbo.BlockMaster` to return `ProjectName` and `BlockName`. Results are ordered by project → block → unit name.

Form Fields

Field	Required	Notes
Project	Yes	Async select — populated from <code>/api/unit-master/projects</code>
Block	Yes	Cascading select — filters to blocks in the selected project
Unit Name	Yes	Free text, <code>NVarChar(100)</code>
Status	No	Toggle, defaults active

Cascading Block Filter

The Block dropdown is populated client-side from a pre-fetched list of all active blocks injected into the form via `externalFormPatch` as `__blocks`. The `optionsProvider` function filters this list by the currently selected `projectId`. When the Project field changes, `onFieldChange` clears `blockId` to `""` so no stale selection persists.

```
// From UnitMaster.tsx — optionsProvider for the Block field
optionsProvider: (_data, _currentId, form) => {
  const blocks: { Id: number; Name: string; ProjectId: number }[]
  =
    (form?.__blocks as any) ?? [];
  const selectedProject = form?.projectId as string | undefined;
  return blocks
    .filter((b) =>
      selectedProject ? String(b.ProjectId) === selectedProject :
      true
    )
    .map((b) => ({ value: String(b.Id), label: b.Name }));
}
```

Common Pitfalls

Block Master must be populated before Unit Master. If no blocks exist for the selected project, the Block dropdown renders empty and the form cannot

be submitted. The GET `/blocks` endpoint only returns blocks where `IsActive = 1` — inactive blocks do not appear as options.

3. Reminders

Type: Reference + Explanation

Description: A cross-module reminder management page. Reminders are stored in `dbo.TenantReminders` and can be associated with any Follow-Up sub-module. Each reminder has a due date and a send action that marks it dispatched.

API Endpoints

Method	Path	Description
GET	<code>/api/tenant-reminders?module=&refId=</code>	List reminders, optionally filtered by module and/or refId
POST	<code>/api/tenant-reminders</code>	Create a reminder
POST	<code>/api/tenant-reminders/send/:id</code>	Mark a reminder as sent (<code>IsSent = 1</code> , sets <code>SentAt</code>)

Database Table: `dbo.TenantReminders`

Column	Type	Notes
<code>ReminderId</code>	<code>INT</code> PK	
<code>Title</code>	<code>NVARCHAR</code>	Displayed as <code>tenantName</code> in the UI
<code>Message</code>	<code>NVARCHAR</code>	Optional body text
<code>Module</code>	<code>NVARCHAR(100)</code>	Module tag: <code>followup</code> , <code>sales</code> , <code>agreement</code> , <code>construction</code> , <code>closure</code>
<code>RefId</code>	<code>INT</code>	Optional reference to a related record; also <code>TRY_CAST</code> as <code>DECIMAL</code> for <code>amountDue</code>
<code>DueDate</code>	<code>DATE</code>	
<code>IsSent</code>	<code>BIT</code>	<code>0</code> = pending, <code>1</code> = sent
<code>SentAt</code>	<code>DATETIME</code>	Set when <code>IsSent</code> becomes <code>1</code>
<code>CreatedBy</code>	<code>NVARCHAR</code>	User name string (not an FK)
<code>CreatedAt</code>	<code>DATETIME</code>	

Status Derivation

Status is computed by the backend SQL query, not stored:

```
CASE
  WHEN IsSent = 1 THEN 'sent'
  WHEN DueDate < CAST(GETDATE() AS DATE) THEN 'overdue'
  ELSE 'scheduled'
END AS status
```

The `ReminderStatus` union type on the frontend is `"sent" | "overdue" | "scheduled"`.

Amount Due

`amountDue` is derived from `RefId` via `ISNULL(TRY_CAST(RefId AS DECIMAL(18,2)), 0)`. This is a dual-purpose column: when `RefId` holds a numeric monetary value instead of a record reference, it surfaces as the amount due. When it holds a non-numeric reference ID, `TRY_CAST` returns `NULL` and the field shows `0`.

Module Badge Colours

Module value	Colour
<code>followup</code>	Violet
<code>sales</code>	Blue
<code>agreement</code>	Emerald
<code>construction</code>	Amber
<code>closure</code>	Rose
Any other	Muted grey

Create Reminder — Payload

Field	Required	Type	Notes
<code>title</code>	Yes	string	Stored in <code>Title</code> ; shown as the tenant/title column
<code>message</code>	No	string	Optional body
<code>module</code>	No	string	Defaults to <code>"followup"</code> in the UI form
<code>dueDate</code>	No	ISO date string	<code>YYYY-MM-DD</code>
<code>refId</code>	No	number	Numeric reference to a related record

Field	Required	Type	Notes
<code>createdBy</code>	No	string	<code>currentUser.name</code> injected by the form

Sort Order

The GET query orders by urgency first: overdue unsent records (`IsSent = 0 AND DueDate < today`) sort before all others, then by `DueDate ASC` .

4. Pending Tasks

Type: Reference + Explanation

Description: A task management view scoped to the Follow-Up module. Tasks are stored in the shared `Tasks` table (same table used across the whole application) and are filtered by `module = 'followup'` . Each task has a title, priority, assignee, due date, and a four-stage status lifecycle.

API Endpoints

All task endpoints use the shared tasks router:

Method	Path	Description
<code>GET</code>	<code>/api/tasks?module=followup</code>	All tasks scoped to <code>followup</code>
<code>POST</code>	<code>/api/tasks</code>	Create a task
<code>PUT</code>	<code>/api/tasks/:id</code>	Update task (typically <code>status</code> only from this page)

Task Status Lifecycle

`open → in_progress → closed → reviewed`

Admins and `dba` role users can move a task to any status from the inline dropdown that appears on row hover. Regular users cannot create tasks (the New Task button is hidden unless `role ≡ 'admin' | 'super_admin' | 'dba'`).

Task Interface

```

interface PendingTask {
  id: string;
  title: string;
  description: string;
  module?: string | null;          // "followup" for all records on
this page
  priority: "low" | "medium" | "high" | null;
  status: "open" | "in_progress" | "closed" | "reviewed";
  assignedTo: string;              // user id
  assignedToName: string;
  createdBy: string;
  createdByName: string;
  dueDate: string;                 // YYYY-MM-DD
}

```

Priority Values

Value	Display colour
high	Red
medium	Amber
low	Emerald
null (stored)	Treated as medium in the UI

Status Colours

Status	Display
open	Muted grey
in_progress	Blue
closed	Emerald
reviewed	Primary (violet)

Overdue Calculation

A task is overdue when `dueDate` is in the past and `status` is neither `closed` nor `reviewed`. Overdue rows render with a `bg-red-500/5` background and the title in red. The Overdue stat card counts these tasks but does not act as a filter tab — it is display-only.

Create Task — Required Fields

Field	Required	Notes
title	Yes	
assignedTo	Yes	Must be a valid user ID from allUsers
dueDate	Yes	
priority	No	Defaults to "medium" in the form
description	No	
module	Hardcoded	Always sent as "followup" – not exposed in the form

Cache

The query key is ["pending-tasks-setup"] with a staleTime of 2 minutes. Both createMutation and updateStatusMutation invalidate this key on success.

5. Customer Master

Type: Reference

Description: Stores customer records for the Follow-Up / Sales pipeline. Customers are written to the shared dbo.AccountHeadMaster table with LHeadType = 'A', which distinguishes them from Suppliers (S), Contractors (C), Banks (B), and General Ledger entries (L).

API

Uses the shared accountHeadApi.ts helpers (getList, addRecord, updateRecord, deleteRecord), all of which pass LHeadType = 'A' as a discriminator.

```
const CUSTOMER_TYPE = "A"; // from CustomerMaster.tsx

getList("A")           // GET /api/account-head?type=A
addRecord(payload, "A")
updateRecord(id, payload, "A")
```

Database Table: dbo.AccountHeadMaster (rows where LHeadType = 'A')

Column	Frontend Field	Required	Notes
LHeadId	_id	—	PK, auto-increment
LHeadName	LHeadName	Yes	Customer name
LHeadContactPerson	LHeadContactPerson	No	
LHeadPhone	LHeadPhone	No	
LHeadEmail	LHeadEmail	No	
LGST	LGST	No	Auto-uppercased
LHeadPan	LHeadPan	No	Auto-uppercased
LGSTType	LGSTType	No	See GST Type options below
LGSTState	LGSTState	No	All 36 Indian states/UTs
LHeadPaymentTerms	LHeadPaymentTerms	No	See Payment Terms options
LHeadAddress	LHeadAddress	No	Full-width textarea
LHeadStatus	LHeadStatus	No	Toggle, defaults active
LHeadType	—	—	Always 'A' — set by API helper, not the form

GST Type Options

Regular · Composition · Unregistered · SEZ · Deemed Export

Payment Terms Options

Advance · 15 Days · 30 Days · 45 Days · 60 Days

Shared Table Architecture Note

Customer Master shares `dbo.AccountHeadMaster` with four other masters across the application. The table is partitioned by the `LHeadType` column:

LHeadType	Module
A	Customers (Follow-Up)
S	Suppliers (Finance)

LHeadType	Module
C	Contractors (Finance)
B	Banks (Finance)
L	General Ledger (Finance)

All API helpers enforce this discriminator. You cannot accidentally read a supplier record through the Customer Master API because every query filters on `LHeadType = 'A'`.

6. Payment Plan Master

Type: Reference

Description: Defines installment terms used in booking payment schedules. Each term has a name, a value type (`percent` or `fixed`), a numeric value, and an active flag. Terms are listed in the Payment Plan inline editor, not through the generic `MasterPage` component.

API Endpoints

Method	Path	Description
GET	<code>/api/payment-plan-master</code>	List all payment terms
POST	<code>/api/payment-plan-master</code>	Create term
PUT	<code>/api/payment-plan-master/:id</code>	Update term
DELETE	<code>/api/payment-plan-master/:id</code>	Delete term
PATCH	<code>/api/payment-plan-master/:id/toggle</code>	Toggle <code>IsActive</code>

PaymentTerm Interface

```
interface PaymentTerm {
  TermID: number;
  TermName: string;
  ValueType: "percent" | "fixed"; // percent = % of total; fixed
= ₹ absolute amount
  TermValue: number;
  IsActive: boolean;
  CreatedAt: string;
```

```
    UpdatedAt: string | null;  
  }
```

Value Type Behaviour

ValueType	Icon	Meaning
percent	%	Term value is a percentage of the total booking amount
fixed	₹	Term value is a fixed Indian Rupee amount

The UI renders a % or ₹ icon badge per row. The toggle between types is done inline during row creation via a segmented button (not a select dropdown).

UI Pattern

Payment Plan Master uses a custom inline-editor table (not the generic `MasterPage`). A draft row (`DraftRow`) is appended at the bottom for new term entry. Existing rows are edited in-place. The `IsActive` toggle is available per-row directly in the list.

Follow-Up Module — Transactional Pages

The pages below are the core working screens of the Follow-Up module. They track a real estate customer from first inquiry through to physical possession of their unit.

Dashboard

Type: Reference

Description: The Follow-Up Dashboard is the entry point to the module. It aggregates live KPI counts across all sub-modules and surfaces the most recently active records for tasks, reminders, applications, and bookings.

Path: `/followup`

What it shows:

- KPI cards for Applications, Bookings, Unit Selections, Welcome Calls, Agreements, NOC, Sales Deeds, Handovers, and Construction Updates — each with a count and a trend indicator
- A clickable task list scoped to `module = 'followup'`, showing status badges and due dates
- A reminders panel showing overdue and scheduled reminders
- A recent bookings list with booking number, applicant, and status

All data is fetched client-side via React Query. Clicking a KPI card navigates directly to the corresponding sub-page.

Tasks Group

Tasks (`/followup/follow-ups/tasks`)

The same shared `PendingTasks` component described in the Setup section, scoped to `module = 'followup'`. See **Pending Tasks** in the Setup section for full field and API reference.

Follow-Up Log (`/followup/follow-ups/log`)

Type: Reference

Description: A chronological activity log for all follow-up interactions. Each entry records what happened (a call, email, SMS, note, or payment), who it was with, when it occurred, and any amount involved.

API Endpoints:

Method	Path	Description
GET	<code>/api/followup-log?search=&type=&module=&refId=</code>	List entries; all filters optional
POST	<code>/api/followup-log</code>	Create a log entry
PUT	<code>/api/followup-log/:id</code>	Update a log entry
DELETE	<code>/api/followup-log/:id</code>	Soft-delete (<code>IsDeleted = 1</code>)

Database Table: `dbo.FollowupLog`

Column	Type	Notes
<code>Id</code>	<code>INT</code> PK	

Column	Type	Notes
LogDate	DATE	Date of the interaction
LogType	NVARCHAR(20)	One of <code>email</code> , <code>call</code> , <code>sms</code> , <code>note</code> , <code>payment</code>
Module	NVARCHAR(100)	Sub-module tag (e.g. <code>welcome_call</code> , <code>followup</code>)
Customer	NVARCHAR	Customer name string (free text, not an FK)
Amount	DECIMAL	Optional monetary amount
RefId	INT	Optional reference to a related record
Notes	NVARCHAR	Free text
IsDeleted	BIT	Soft-delete flag; <code>GET</code> always filters <code>IsDeleted = 0</code>
CreatedBy	NVARCHAR	User name string from <code>req.user.name</code>
CreatedAt	DATETIME	

Log Types: `email` · `call` · `sms` · `note` · `payment`

The backend validates `LogType` against this enum. An invalid type returns HTTP 400.

Filter Parameters:

Parameter	Behaviour
<code>search</code>	<code>LIKE</code> match on <code>Customer</code> or <code>Notes</code>
<code>type</code>	Exact match on <code>LogType</code> (case-insensitive)
<code>module</code>	Exact match on <code>LOWER(Module)</code>
<code>refId</code>	Exact match on <code>RefId</code> (must be numeric)

Results are ordered by `CreatedAt DESC, Id DESC` .

Reminders Group

The five reminder pages — **PO Reminders** , **WO Reminders** , **CHQ Reminders** , **TDS Reminders** , **GRN Reminders** — are read-only views that filter the shared `dbo.TenantReminders` table by module tag. They surface payment deadlines from the Material and Finance modules inside the Follow-Up context for easy monitoring. Each page passes a specific `module` query parameter to `/api/tenant-reminders` .

Page	module filter
P0 Reminders	po
W0 Reminders	wo
CHQ Reminders	chq
TDS Reminders	tds
GRN Reminders	grn

For the full `TenantReminders` schema and status derivation logic, see **Reminders** in the Setup section.

Sales Group

The Sales group tracks the customer acquisition funnel: a prospect submits an Application → selects a preferred Unit → the sale is confirmed as a Booking → a Welcome Call is logged.

Applications (`/followup/sales/applications`)

Type: Reference

Description: Captures prospective buyer enquiries. An application is the first formal record of a customer's interest in a project. It links to a customer (`AccountHeadMaster` , `LHeadType = 'A'`), a project, optionally a preferred unit, and an assigned sales user.

API Endpoints:

Method	Path	Description
GET	<code>/api/followup-applications/options</code>	Dropdowns: customers, projects, companies, users, units
GET	<code>/api/followup-applications?search=&status=&projectId=</code>	Paginated list
GET	<code>/api/followup-applications/:id</code>	Single record
POST	<code>/api/followup-applications</code>	Create application (returns generated <code>ApplicantNo</code>)
PUT	<code>/api/followup-applications/:id</code>	Update
DELETE	<code>/api/followup-applications/:id</code>	Soft-delete — blocked if linked <code>UnitSelections</code> or <code>Agreements</code> exist

Database Table: `dbo.FollowupApplications`

Column	Notes
<code>Id</code>	PK
<code>ApplicantNo</code>	Auto-generated on insert; unique applicant reference number
<code>CustomerId</code>	FK → <code>dbo.AccountHeadMaster.LHeadId</code> where <code>LHeadType = 'A'</code>
<code>ApplicantName</code>	Denormalised name copy
<code>PrimaryMobile</code>	
<code>Email</code>	
<code>PanNumber</code>	
<code>ApplicantAddress</code>	
<code>CoApplicantName</code>	Optional co-applicant
<code>CoApplicantPhone</code>	
<code>CorrespondenceAddress</code>	
<code>ApplicationDate</code>	
<code>City</code>	
<code>Source</code>	Lead source (walk-in, referral, online, etc.)
<code>ProjectId</code>	FK → <code>dbo.enterprise</code> where <code>business_type = 'P'</code>
<code>UnitId</code>	Optional preferred unit FK → <code>dbo.UnitMaster</code>
<code>CompanyId</code>	FK → <code>dbo.enterprise</code> where <code>business_type = 'C'</code>
<code>PreferredUnitType</code>	Free text (e.g. 2BHK, commercial)
<code>BudgetAmount</code>	Prospective budget
<code>Status</code>	See status options below
<code>AssignedTo</code>	FK → <code>dbo.users.id</code>
<code>Notes</code>	
<code>IsDeleted</code>	Soft-delete flag

Status Options: `New` · `Qualified` · `Shortlisted` · `Document Pending` · `Rejected`

Delete Guard: The DELETE endpoint counts linked records in `dbo.FollowupUnitSelections` and `dbo.FollowupAgreements` before proceeding. If either count is greater than 0, the delete is rejected.

Bookings (/followup/sales/bookings)

Type: Reference

Description: Confirms a sale. A booking is created after an application is qualified and records the financial terms: total value, booking amount, rate per sq ft, area, loan amount, and an attached payment plan schedule. Payment plan terms are resolved from `dbo.PaymentTermMaster` and stored in `dbo.BookingPaymentTerms`.

API Endpoints:

Method	Path	Description
GET	/api/followup-bookings	List all bookings
GET	/api/followup-bookings/:id	Single booking with payment terms
POST	/api/followup-bookings	Create booking + generate <code>BookingNo</code> + compute and store payment terms
PUT	/api/followup-bookings/:id	Update
DELETE	/api/followup-bookings/:id	Soft-delete

Database Tables:

`dbo.FollowupBookings` — one row per booking.

Column	Notes
<code>Id</code>	PK
<code>BookingNo</code>	Auto-generated on insert
<code>ApplicantId</code>	FK → <code>dbo.FollowupApplications.Id</code>
<code>UnitSelectionId</code>	Optional FK → <code>dbo.FollowupUnitSelections.Id</code>
<code>ProjectId</code>	FK → <code>dbo.enterprise</code>
<code>CompanyId</code>	FK → <code>dbo.enterprise</code>
<code>TotalValue</code>	Agreed sale price
<code>BookingAmount</code>	Initial payment made at booking
<code>RatePerSqFt</code>	
<code>AreaSqFt</code>	
<code>LoanAmount</code>	Optional bank loan
<code>Status</code>	<code>Confirmed</code> · <code>Pending</code> · <code>Cancelled</code>

Column	Notes
PaymentPlanSummary	JSON snapshot of resolved payment terms
AssignedTo	FK → <code>dbo.users.id</code>
IsDeleted	

`dbo.BookingPaymentTerms` — one row per payment instalment per booking.

Column	Notes
BookingID	FK → <code>dbo.FollowupBookings.Id</code>
TermID	FK → <code>dbo.PaymentTermMaster.TermID</code>
TermName	Denormalised
ValueType	<code>percent</code> or <code>fixed</code>
TermValue	Raw value from master
ResolvedAmount	Computed ₹ amount for this booking

Payment Plan Resolution:

```
// From followupBookings.js — resolveTermAmount()
function resolveTermAmount(term, totalValue) {
  const tv = Number(totalValue) || 0;
  const tv2 = Number(term.TermValue) || 0;
  if (term.ValueType === "percent") return Math.round((tv2 / 100)
* tv);
  if (term.ValueType === "fixed") return Math.round(tv2);
  return 0;
}
```

On every create or update, `BookingPaymentTerms` for that booking is deleted and re-inserted from scratch. `BookingAmount` is recalculated as the sum of all `ResolvedAmount` values and written back to `FollowupBookings`.

Payment Modes (for booking deposit): `Cheque` · `NEFT` · `RTGS` · `DD` · `Cash` · `Online`

Unit Selection (`/followup/sales/unit-selection`)

Type: Reference

Description: Records the specific unit a customer has chosen. Where Applications capture a preferred unit type, Unit Selection locks in the actual flat/shop/plot.

API Endpoints:

Method	Path	Description
GET	/api/followup-unit-selections	List
GET	/api/followup-unit-selections/:id	Single record
POST	/api/followup-unit-selections	Create; generates SelectionNo
PUT	/api/followup-unit-selections/:id	Update; also updates linked FollowupAgreements.UnitSelectionId
DELETE	/api/followup-unit-selections/:id	Soft-delete

Database Table: dbo.FollowupUnitSelections

Column	Notes
Id	PK
SelectionNo	Auto-generated
ApplicantId	FK → dbo.AccountHeadMaster.LHeadId where LHeadType = 'A'
ProjectId	FK → dbo.enterprise
CompanyId	FK → dbo.enterprise
UnitNo	Unit identifier (text, not an FK to UnitMaster)
BlockName	Denormalised block name
FloorName	
UnitType	e.g. 2BHK , Shop
AreaSqFt	
RatePerSqFt	
TotalValue	
BookingAmount	
SelectionDate	
Status	Reserved · Negotiation · Confirmed · Released

Column	Notes
Notes	
IsDeleted	

Permission guard: All routes use `checkPermissionForMethod("Followup", "UnitSelections")`.

Welcome Calls (`/followup/sales/welcome-calls`)

Type: Reference

Description: Logs post-booking welcome calls made to customers. Welcome Calls are a filtered view of `dbo.FollowupLog` – they query `module = 'welcome_call'` and create entries with that module tag.

API: Uses the same `/api/followup-log` endpoints as Follow-Up Log, with `module = 'welcome_call'` hardcoded in all requests.

Call Outcome Types: `connected` · `no_answer` · `callback` · `voicemail` · `note`

These map to `LogType` in `dbo.FollowupLog`. The Welcome Calls page uses its own outcome labels and icons rather than the generic log type names.

Agreement Group

Agreements (`/followup/agreement/agreements`)

Type: Reference

Description: Records the sale agreement document between the developer and the buyer. An agreement links back to the full chain: Applicant → Unit Selection → Booking.

API Endpoints:

Method	Path	Description
GET	<code>/api/followup-agreements</code>	List
GET	<code>/api/followup-agreements/:id</code>	Single record

Method	Path	Description
POST	/api/followup-agreements	Create; generates AgreementNo
PUT	/api/followup-agreements/:id	Update
DELETE	/api/followup-agreements/:id	Soft-delete

Database Table: `dbo.FollowupAgreements`

Column	Notes
Id	PK
AgreementNo	Auto-generated
ApplicantId	FK → <code>dbo.AccountHeadMaster</code> where <code>LHeadType = 'A'</code>
UnitSelectionId	FK → <code>dbo.FollowupUnitSelections.Id</code>
BookingId	FK → <code>dbo.FollowupBookings.Id</code>
ProjectId	FK → <code>dbo.enterprise</code>
CompanyId	FK → <code>dbo.enterprise</code>
AgreementDate	
AgreementValue	Final agreed sale value
AdvanceAmount	Amount paid to date
BalanceAmount	Outstanding amount
RegistrationDate	Optional — date of sub-registrar registration
Status	<code>Draft</code> · <code>Issued</code> · <code>Signed</code> · <code>Cancelled</code>
Notes	
IsDeleted	

Permission guard: `checkPermissionForMethod("Followup", "Agreements")`

Closure Group

The three closure pages — NOC, Sales Deed, Handover — form the final sequence of a completed sale. They all share the same upstream chain reference: Applicant → Unit Selection → Agreement.

NOC (`/followup/closure/noc`)

Type: Reference

Description: Records the No Objection Certificate issued to the buyer. Typically required before a Sales Deed can be executed.

Database Table: `dbo.FollowupNOC`

Column	Notes
<code>Id</code>	PK
<code>NOCNo</code>	Auto-generated
<code>ApplicantId</code>	FK → <code>dbo.AccountHeadMaster</code> where <code>LHeadType = 'A'</code>
<code>UnitSelectionId</code>	FK → <code>dbo.FollowupUnitSelections</code>
<code>AgreementId</code>	FK → <code>dbo.FollowupAgreements</code>
<code>ProjectId</code>	FK → <code>dbo.enterprise</code>
<code>CompanyId</code>	FK → <code>dbo.enterprise</code>
<code>NOCDate</code>	Date the NOC was created
<code>ApprovalDate</code>	Date approved
<code>IssuedDate</code>	Date issued to applicant
<code>ApprovedBy</code>	Name of approving authority
<code>Reason</code>	Reason for NOC
<code>Status</code>	<code>Pending</code> · <code>Approved</code> · <code>Issued</code> · <code>Rejected</code>
<code>Notes</code>	
<code>IsDeleted</code>	

Permission guard: `checkPermissionForMethod("Followup", "NOC")`

Sales Deed (`/followup/closure/sales-deed`)

Type: Reference

Description: Records the legal sale deed execution and sub-registrar registration. Captures stamp duty, registration fee, registration number, and possession date.

Database Table: `dbo.FollowupSalesDeeds`

Column	Notes
<code>Id</code>	PK
<code>DeedNo</code>	Auto-generated

Column	Notes
ApplicantId	FK → <code>dbo.AccountHeadMaster</code> where <code>LHeadType = 'A'</code>
UnitSelectionId	FK → <code>dbo.FollowupUnitSelections</code>
AgreementId	FK → <code>dbo.FollowupAgreements</code>
ProjectId	FK → <code>dbo.enterprise</code>
CompanyId	FK → <code>dbo.enterprise</code>
DeedValue	Value stated in deed
StampDuty	Stamp duty paid
RegistrationFee	Sub-registrar fee
SubRegistrarOffice	Office name
RegistrationNo	Registration document number
BookNo	Registry book number
PartNo	Registry part number
DeedDate	Date of deed execution
RegistrationDate	Date of sub-registrar registration
PossessionDate	Scheduled possession date
ExecutedBy	Signatory on developer side
WitnessNames	Witness names
Status	<code>Draft</code> · <code>Executed</code> · <code>Registered</code> · <code>Cancelled</code>
Notes	
IsDeleted	

Permission guard: `checkPermissionForMethod("Followup", "SalesDeed")`

Handover (`/followup/closure/handover`)

Type: Reference

Description: Records the physical handover of the unit to the buyer. Tracks key handover, utility meter transfers, snag list resolution, and whether the welcome kit was given. This is the final record in the Follow-Up lifecycle.

Database Table: `dbo.FollowupHandovers`

Column	Notes
Id	PK
HandoverNo	Auto-generated

Column	Notes
ApplicantId	FK → <code>dbo.AccountHeadMaster</code> where <code>LHeadType = 'A'</code>
UnitSelectionId	FK → <code>dbo.FollowupUnitSelections</code>
AgreementId	FK → <code>dbo.FollowupAgreements</code>
SalesDeedId	FK → <code>dbo.FollowupSalesDeeds</code>
ProjectId	FK → <code>dbo.enterprise</code>
CompanyId	FK → <code>dbo.enterprise</code>
HandoverDate	Scheduled handover date
ActualHandoverDate	Actual date completed
KeyHandoverDate	Date keys were handed over
UnitCondition	<code>Ready</code> · <code>Punch-list Pending</code> · <code>Snagging</code> · <code>Minor Works</code>
SnagListItems	Text description of outstanding snags
SnagsClearedDate	Date all snags resolved
HandedOverBy	Developer representative name
ReceivedBy	Buyer's name
WitnessNames	
ElectricMeterHandedOver	BIT
WaterConnectionHandedOver	BIT
ParkingAllotted	BIT
WelcomeKitGiven	BIT
Status	<code>Scheduled</code> · <code>Completed</code> · <code>Delayed</code> · <code>Cancelled</code>
Notes	
IsDeleted	

Permission guard: `checkPermissionForMethod("Followup", "Handover")`

Construction Group

Construction Updates (`/followup/construction/updates`)

Type: Reference

Description: Sends construction progress updates to applicants. Each update is tied to a specific applicant, unit selection, and project, and describes which construction stage has been reached. Updates can be dispatched

(status changes from **Draft** to **Sent**) and can be acknowledged or disputed by the applicant.

API Endpoints:

Method	Path	Description
GET	/api/followup-construction-updates/options	Dropdowns: applicants, unit selections, projects, companies
GET	/api/followup-construction-updates?search=&status=&projectId=	List
GET	/api/followup-construction-updates/:id	Single record
POST	/api/followup-construction-updates	Create; generates UpdateNo
PUT	/api/followup-construction-updates/:id	Update
DELETE	/api/followup-construction-updates/:id	Soft-delete

Database Table: **dbo.FollowupConstructionUpdates**

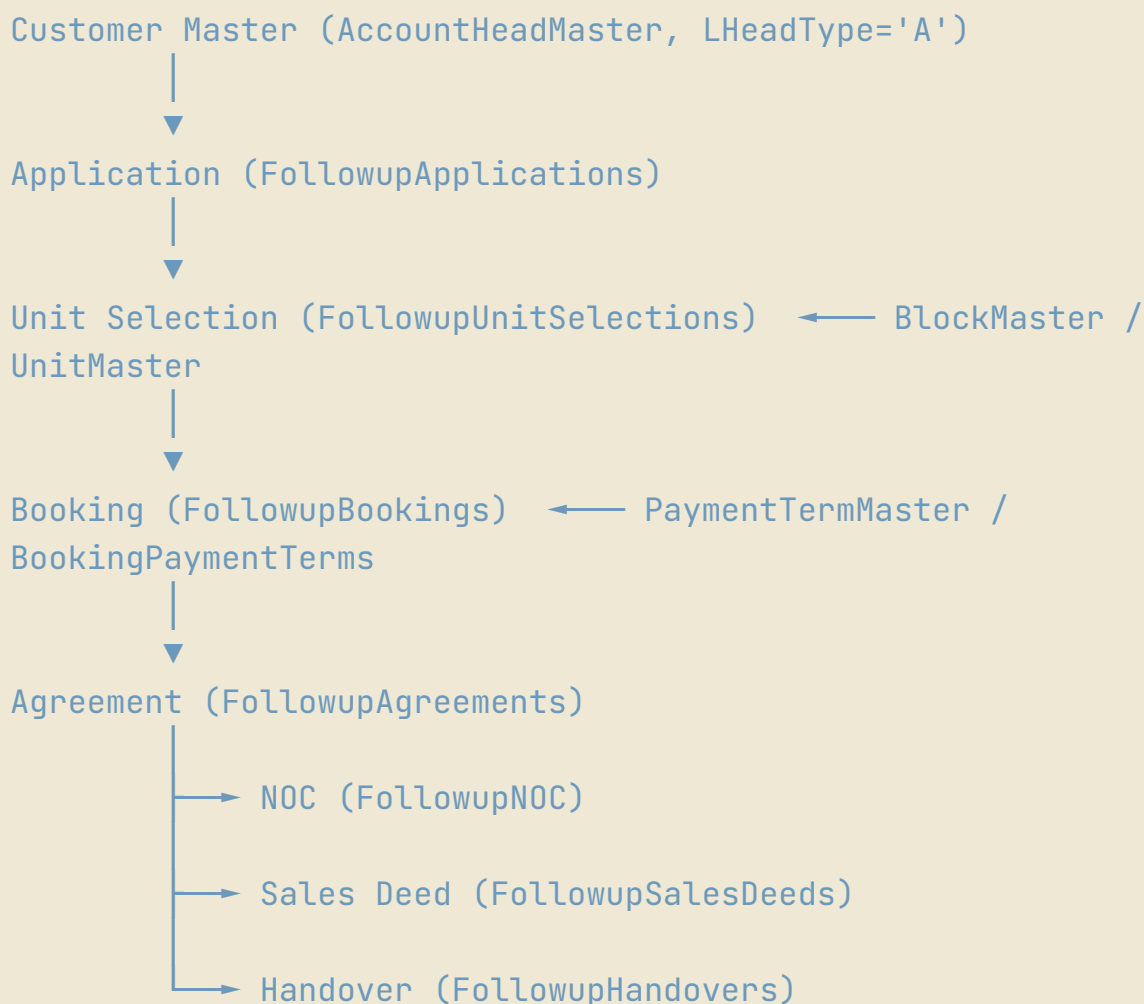
Column	Notes
Id	PK
UpdateNo	Auto-generated
ApplicantId	FK → dbo.AccountHeadMaster where LHeadType = 'A'
UnitSelectionId	FK → dbo.FollowupUnitSelections
ProjectId	FK → dbo.enterprise
CompanyId	FK → dbo.enterprise
Stage	Construction stage — see options below
PercentComplete	Numeric 0–100
UpdateDate	Date of the update
Description	Narrative description of progress
PhotoUrls	JSON array of photo URLs
Status	Draft · Sent · Acknowledged · Disputed
Notes	
IsDeleted	

Construction Stage Options: Foundation · Basement · Slab · Columns & Beams · Brickwork · Plastering · Electrical & Plumbing · Flooring · Finishing · Completion · Other

Permission guard: checkPermissionForMethod("Followup", "ConstructionUpdates")

Complete Document Chain

Every record in the Follow-Up module carries forward references to earlier records in the chain. The full chain from first enquiry to possession is:



Parallel tracks (attached at any point):

FollowupLog — activity log (calls, emails, notes, payments)

TenantReminders — due-date reminders

Tasks (module=followup) — internal action items

ConstructionUpdates — progress dispatches to applicants

Each Closure document (NOC, Sales Deed, Handover) carries `ApplicantId`, `UnitSelectionId`, and `AgreementId` so any record can be traced back to its full upstream context.

Permission Module Map

All Follow-Up routes use `checkPermission` or `checkPermissionForMethod` with `"Followup"` as the module. The submodule strings used:

Sub-module string	Route
<code>Applicants</code>	Applications
<code>Bookings</code>	Bookings
<code>UnitSelections</code>	Unit Selection
<code>Agreements</code>	Agreements
<code>NOC</code>	NOC
<code>SalesDeed</code>	Sales Deed
<code>Handover</code>	Handover
<code>ConstructionUpdates</code>	Construction Updates

Tasks and Reminders use role-level guards (`admin`, `super_admin`, `dba`) rather than permission submodule strings.

Cross-Cutting Notes

Setup Dependency Order

The transactional Follow-Up pages depend on setup data being present. The practical dependency chain is:

```
Customer Master → Applications → Bookings
Block Master    → Unit Master → Unit Selection
                  (referenced in Bookings)
Payment Plan    → Bookings (payment schedule terms)
```

Reminders and Pending Tasks have no upstream dependencies — they can be created at any time.

Cache Keys Reference

Cache Key	TTL	Busted By
<code>block-master</code>	300s	Any block create / update / delete
<code>block-master-projects</code>	600s	Not busted (project list is stable)
<code>unit-master</code>	300s	Any unit create / update / delete
<code>unit-master-projects</code>	600s	Not busted
<code>pending-tasks-setup</code> (React Query)	120s	Task create / status update
<code>followup-reminders</code> (React Query)	No <code>staleTime</code>	Reminder create / send

Common Pitfalls

Block deleted before its units. The backend will return a foreign key constraint error. Delete all units in the block first. There is no cascade delete.

Unit Master block dropdown shows no options. If a block is set to `IsActive = 0`, it will not appear in the `/api/unit-master/blocks` dropdown. Only active blocks are selectable when creating units.

Customer `LHeadType` is hardcoded. The `CUSTOMER_TYPE = "A"` constant in `CustomerMaster.tsx` is not configurable from the UI. If `AccountHeadMaster` rows appear in the wrong master, check that `LHeadType` was written correctly at creation time.

Reminder `amountDue` is `0` for non-numeric `RefId`. When a reminder's `RefId` links to a record ID rather than a monetary amount, `TRY_CAST` returns `NULL` and `amountDue` is coerced to `0`. This is expected behaviour — the Amount column will show — in the UI.

Pending Tasks creation restricted by role. The New Task button only renders for `admin`, `super_admin`, and `dba` roles. Users with other roles can view and filter tasks but cannot create them.

Engineering Module

Type: Reference + Explanation

Stack: React 18 / TypeScript (frontend) · Node.js / Express (backend) · SQL Server (database) · Redis (cache)

Module Overview

The Engineering module manages the estimation, contracting, and certification of construction work. It covers the full engineering workflow: a Bill of Quantities (BOQ) is prepared → a Work Order is issued to a contractor referencing that BOQ → Work Done entries are submitted per billing period to certify completed work → Approved Work Done entries can generate a Payment (via Expense Booking in the Material module).

Navigation structure (from `EngineeringSidebar.ts`):

Group	Pages
Dashboard	<code>/engineering</code>
Transaction	BOQ · Work Order · Work Done
Setup	Activity Master

Engineering Setup

Activity Master

Type: Reference + Explanation

Description: Defines the catalogue of work activities used in Work Orders and BOQs. The master has a two-level hierarchy: **Groups** (category containers) and **Activities** (individual line items). Both are stored in the same table, distinguished by `activity_type`.

Path: Engineering → Setup → Activity (accessible from top bar Setup panel)

API Endpoints

Method	Path	Description
<code>GET</code>	<code>/api/activity-master</code>	List all groups and activities (cached 300s)
<code>GET</code>	<code>/api/activity-master/:id</code>	Single record
<code>POST</code>	<code>/api/activity-master</code>	Create group or activity
<code>PUT</code>	<code>/api/activity-master/:id</code>	Update
<code>PATCH</code>	<code>/api/activity-master/:id/approve</code>	Set <code>approved_by</code> + <code>approved_at</code>

Method	Path	Description
DELETE	/api/activity-master/:id	Hard delete

Database Table: `dbo.ActivityMaster`

Column	Type	Notes
<code>id</code>	INT PK	Auto-increment
<code>activity_name</code>	NVARCHAR(255)	Required
<code>short_description</code>	NVARCHAR(255)	Optional
<code>activity_type</code>	TINYINT	0 = Group · 1 = Activity
<code>group_id</code>	INT	NULL for Groups; FK to parent Group's <code>id</code> for Activities
<code>is_active</code>	BIT	
<code>created_by</code>	NVARCHAR(300)	Stores user email
<code>created_datetime</code>	DATETIME2	
<code>approved_by</code>	NVARCHAR(300)	Set via PATCH <code>/approve</code>
<code>approved_at</code>	DATETIME2	
<code>updated_by</code>	NVARCHAR(300)	
<code>updated_at</code>	DATETIME2	
<code>belongsTo</code>	NVARCHAR(200)	NULL for Groups; String(<code>group_id</code>) for Activities — redundant string copy of <code>group_id</code>
<code>hsn_code</code>	NVARCHAR(50)	NULL for Groups; optional HSN code for Activities

Type Discriminator Rules

```
activity_type = 0 (Group):
  group_id  = NULL
  belongsTo = NULL
  hsn_code  = NULL    ← always cleared server-side, even if sent
```

```
activity_type = 1 (Activity):
  group_id  = INT id of parent Group
  belongsTo = String(group_id)    ← string copy, stored for query convenience
  hsn_code  = optional NVARCHAR(50)
```

The backend enforces these rules on every POST and PUT — it ignores any `hsn_code` or `group_id` sent for a Group record and sets them to `NULL`.

`toPayload()` — Frontend Payload Builder

The form stores the group selection as a display label (the string name of the group). `toPayload()` in `activityMasterApi.ts` resolves this back to the numeric group ID by searching `groupOptions`:

```
const matched = groupOptions.find((o) => o.label === r.groupId);
const groupId = matched ? Number(matched.value) : null;
```

This means if a group is renamed, existing Activities that still reference the old label will fail to resolve on next edit. Always use the group's current display name when editing an Activity.

UI — Grouped Tree View

The Activity Master page renders two panels side by side:

- **Form panel** (`MasterPage` with `hideTable` prop) — handles add/edit/delete via the standard form.
- **Tree panel** — a collapsible grouped view showing Groups as expandable headers and their Activities as indented rows. Ungrouped Activities (those with `group_id` set to a non-existent group) appear under an "Ungrouped Activities" section.

Clicking the eye icon on any row opens a slide-in detail drawer without leaving the page.

HSN Code Constraint

The HSN dropdown is **disabled** (rendered as greyed-out) when `activityType = "Group"`. Selecting a Group type clears the HSN field server-side. HSN codes flow from `dbo.HsnMaster` and are used downstream in Work Order GST calculations.

Cache

All writes call `bumpCacheVersion("activity-master")`. The Work Order meta endpoint `/api/work-orders/meta/activity-groups` has its own cache key (`wo-meta-act-groups`, 600s) and queries only `activity_type = 0` (Groups).

Common Pitfalls

Deleting a Group that has Activities under it will leave those Activities with a `group_id` pointing to a non-existent record. They will appear in the "Ungrouped Activities" section of the tree view. There is no cascade delete or referential integrity on `group_id` – clean up child Activities before deleting a Group.

Engineering Transactions

Dashboard

Type: Reference

Description: Aggregates live KPI counts across all three engineering transactions. Data is fetched from `GET /api/engineering/dashboard`.

Dashboard API response shape:

```
{
  workOrders: {
    total: number;           // all WorkOrderHeader rows
    open: number;           // status in Draft / Pending / Open / In
    Progress
    thisMonth: number;      // DocumentDate in current calendar
    month
    totalValue: number;     // SUM(TotalAmount)
  };
  boq: {
    total: number;
    approved: number;       // Status = 'Approved'
    totalValue: number;     // SUM(TotalAmount)
  };
  workDone: {
    total: number;
    pending: number;        // Status = 'Pending'
    certifiedAmount: number; // SUM(CertifiedAmount)
  };
  projects: {
    total: number;
    active: number;         // discontinue IS NULL or 0
  };
  recentWOs: WorkOrderRow[]; // TOP 6 by last update
  recentBOQs: BOQRow[];     // TOP 6 by last update
  recentWorkDone: WorkDoneRow[]; // TOP 6 by last update
}
```

```

woStatusBreakdown: { Status, Count, TotalValue }[];
workDoneStatusBreakdown: { Status, Count, TotalValue }[];
}

```

The dashboard uses `hasTable()` to detect whether `dbo.WorkDone` exists before querying it. If the migration has not been run, Work Done KPIs return zeroes without crashing the dashboard.

BOQ (Bill of Quantities)

Type: Reference

Description: A BOQ is a pre-construction cost estimate document that lists all materials and activities needed for a project, with quantities and rates. Approved BOQs can be selected when creating Work Orders, which then inherit the BOQ's line items.

Path: `/engineering/boq`

Permission guard: `checkPermissionForMethod("Engineering", "BOQ")`

API Endpoints

Method	Path	Description
GET	<code>/api/boq?page=&limit=&search=&status=</code>	Paginated list
GET	<code>/api/boq/:id</code>	Single BOQ with items and activities
POST	<code>/api/boq</code>	Create BOQ; auto-generates <code>BoqNo</code> ; auto-submits to <code>Pending</code>
PUT	<code>/api/boq/:id</code>	Update; <code>guardEdit</code> blocks post-approval edits
DELETE	<code>/api/boq/:id</code>	Hard delete (restricted by role)
PUT	<code>/api/boq/:id/submit</code>	<code>Draft → Pending</code>
PUT	<code>/api/boq/:id/approve</code>	<code>Pending → Approved</code>
PUT	<code>/api/boq/:id/reject</code>	<code>Pending → Rejected</code>
GET	<code>/api/engineering/approved-boqs</code>	Lightweight list for WO picker (cached 120s)
GET	<code>/api/engineering/boq-activities/:boqId</code>	<code>BoqActivities</code> rows for a given BOQ

Database Tables

dbo.BOQ — one row per BOQ header.

Column	Type	Notes
BoqID	INT PK	Auto-increment
BoqNo	NVARCHAR	Auto-generated document number
DocNo	NVARCHAR(100)	From <code>lockNextDocNumber()</code> if <code>DocTypeId</code> set
DocTypeId	INT	FK → <code>dbo.TypeOfDoc</code>
BoqDate	DATE	
CompanyId	INT	FK → <code>dbo.enterprise</code> where <code>business_type = 'C'</code>
ProjectId	INT	FK → <code>dbo.enterprise</code> where <code>business_type = 'P'</code>
Description	NVARCHAR	Scope description
TotalAmount	DECIMAL(18,2)	Computed as <code>SUM(Quantity × Rate)</code> across items and activities
Status	NVARCHAR(50)	See status lifecycle below
Remarks	NVARCHAR	
BoqItems	NVARCHAR(MAX)	JSON snapshot of items (also synced to <code>dbo.BoqItems</code>)
BoqActivities	NVARCHAR(MAX)	JSON snapshot of activities (also synced to <code>dbo.BoqActivities</code>)
CreatedBy	NVARCHAR(100)	User email
CreatedAt	DATETIME2	

dbo.BoqItems — normalised item rows (one per line item per BOQ).

Column	Notes
BoqID	FK → <code>dbo.BOQ.BoqID</code>
ItemId	Optional item code
ItemName	Required, max 255 chars
ItemCode	Optional
Description	Long text
Quantity	DECIMAL(18,4)
UomId	FK → <code>dbo.UOMMaster.Id</code> (resolved from <code>UomName</code>)
UomName	NVARCHAR(50)
Rate	DECIMAL(18,4)

Column	Notes
TaxPct	DECIMAL(5,2)
LineAmount	Quantity × Rate
SortOrder	INT

dbo.BoqActivities – normalised activity rows (one per activity line per BOQ).

Column	Notes
BoqID	FK → dbo.BOQ.BoqID
ActivityId	Optional activity code
ActivityName	Required
ActivityCode	Optional
Description	Long text
Quantity	DECIMAL(18,4)
UomId	FK → dbo.UOMMaster.Id
UomName	NVARCHAR(50)
Rate	DECIMAL(18,4)
LineAmount	Quantity × Rate
SortOrder	INT

Status Lifecycle

Draft → Pending → Approved
→ Rejected → Pending (re-submit)

On every create (POST), the backend auto-submits from **Draft → Pending** immediately after insert. **guardEdit()** prevents any field changes once status is **Approved** or beyond.

Total Computation

```
// From boq.js – computeTotal()
const computeTotal = (items, activities) => {
  let sum = 0;
  if (Array.isArray(items))
    sum += items.reduce((s, it) => s + (qty * rate), 0);
  if (Array.isArray(activities))
    sum += activities.reduce((s, ac) => s + (qty * rate), 0);
}
```

```
return sum;
};
```

`TotalAmount` in `dbo.BOQ` is set to this computed value on every create and update. The JSON columns `BoqItems` and `BoqActivities` are snapshot copies — the normalised rows in `dbo.BoqItems` and `dbo.BoqActivities` are the authoritative source and are deleted and re-inserted on every save.

Work Order

Type: Reference

Description: A Work Order is a formal contract document issued to a contractor. It links to an approved BOQ (optional), specifies the scope of work as a list of activities (each with labour and material breakdowns), and records GST. Approved Work Orders are the source documents for Work Done entries.

Path: `/engineering/work-order`

Permission guard: `checkPermissionForMethod("Engineering", "WorkOrders")`

API Endpoints

Method	Path	Description
GET	<code>/api/work-orders</code>	Paginated list with activity counts
GET	<code>/api/work-orders/:id</code>	Full Work Order with activities and materials
POST	<code>/api/work-orders</code>	Create header + activities atomically
PUT	<code>/api/work-orders/:id</code>	Update; <code>guardEdit</code> blocks post-approval edits
DELETE	<code>/api/work-orders/:id</code>	Hard delete (cascades activities and materials)
PUT	<code>/api/work-orders/:id/submit</code>	Draft → Pending
PUT	<code>/api/work-orders/:id/approve</code>	Pending → Approved
PUT	<code>/api/work-orders/:id/reject</code>	Pending → Rejected

Method	Path	Description
GET	/api/work-orders/meta/companies	Companies dropdown (cached 600s)
GET	/api/work-orders/meta/projects	Projects dropdown (cached 600s)
GET	/api/work-orders/meta/contractors	Contractors — <code>AccountHeadMaster</code> where <code>LHeadType = 'C'</code> (cached 600s)
GET	/api/work-orders/meta/activity-groups	Active Groups from <code>ActivityMaster</code> where <code>activity_type = 0</code> (cached 600s)
GET	/api/engineering/work-orders-with-activities	Approved WOs with activity count and totals (cached 300s) — used by Work Done picker
GET	/api/engineering/work-order-summary/:woId	Financial summary: contract value, certified, booked, paid, balance

Database Tables

`dbo.WorkOrderHeader` — one row per Work Order.

Column	Type	Notes
Id	INT PK	Auto-increment
DocumentNumber	NVARCHAR	Auto-generated on create
DocNo	NVARCHAR(100)	From <code>lockNextDocNumber()</code> if <code>DocTypeId</code> set
DocTypeId	INT	FK → <code>dbo.TypeOfDoc</code>
DocumentDate	DATE	
CompanyId	INT	FK → <code>dbo.enterprise</code> where <code>business_type = 'C'</code>
ProjectId	INT	FK → <code>dbo.enterprise</code> where <code>business_type = 'P'</code>
ContractorId	INT	FK → <code>dbo.AccountHeadMaster</code> where <code>LHeadType = 'C'</code>
SupplierId	INT	Optional FK → <code>dbo.AccountHeadMaster</code> where <code>LHeadType = 'S'</code>
TotalAmount	DECIMAL	Gross contract value

Column	Type	Notes
Remarks	NVARCHAR	
TermsAndConditions	NVARCHAR	
GST	NVARCHAR(MAX)	JSON blob containing CGST/SGST/IGST rates and amounts
BoqID	INT	Optional FK → <code>dbo.BOQ.BoqID</code>
Status	NVARCHAR	See status lifecycle below
CreatedBy	NVARCHAR(100)	User name (not email)
CreatedAt	DATETIME	
UpdatedAt	DATETIME	

dbo.WorkOrderActivities — one row per activity line per Work Order.

Column	Notes
Id	PK
WorkOrderHeaderId	FK → <code>dbo.WorkOrderHeader.Id</code>
DocNo	Denormalised W0 document number
ActivityGroupId	FK → <code>dbo.ActivityMaster.id</code> where <code>activity_type = 0</code>
ActivityId	FK → <code>dbo.ActivityMaster.id</code> where <code>activity_type = 1</code>
UOMId	FK → <code>dbo.UOMMaster.Id</code>
Rate	DECIMAL(18,4) — rate per unit
Area	DECIMAL(18,4) — quantity / area
LabourAmount	DECIMAL(18,2)
MaterialAmount	DECIMAL(18,2)
GrandTotal	DECIMAL(18,2) — <code>LabourAmount + MaterialAmount</code>
Remarks	
CreatedAt	DATETIME2

dbo.WorkOrderActivityMaterials — material sub-items per activity line.

Column	Notes
Id	PK
WorkOrderActivityId	FK → <code>dbo.WorkOrderActivities.Id</code>
MaterialId	Optional FK to material

Column	Notes
MaterialName	Text
Quantity	DECIMAL(18,4)
Unit	NVARCHAR(50)
Rate	DECIMAL(18,4)
Amount	DECIMAL(18,2)

Status Lifecycle

Draft → Pending → Approved
→ Rejected → Pending (re-submit)

The `guardEdit()` service call on PUT blocks updates to any Approved or beyond record. Approved Work Orders are listed by `GET /engineering/work-orders-with-activities` for the Work Done picker.

Work Order Summary Endpoint

`GET /engineering/work-order-summary/:woId` computes a live financial reconciliation:

Field	Source
ContractValue	WorkOrderHeader.TotalAmount
ActivityCount	COUNT of WorkOrderActivities rows
GrossAmount	SUM of WorkOrderActivities.GrandTotal
LabourAmount	SUM of WorkOrderActivities.LabourAmount
MaterialAmount	SUM of WorkOrderActivities.MaterialAmount
totalCertified	SUM of WorkDone.CertifiedAmount where Status = 'Approved'
totalBooked	SUM of ExpenseBooking.ENetAmount linked to this WO's Work Done entries
totalPaid	SUM of NewPayment.PAmount linked to those expense bookings
balance	ContractValue - totalPaid

BOQ Inheritance

When a `BoqID` is provided at Work Order creation, the frontend calls `GET /api/engineering/boq-activities/:boqId` and pre-populates the activity line items from `dbo.BoqActivities`. The link is stored in

`WorkOrderHeader.BoqID` but is not enforced as a hard FK constraint — it is an audit reference only.

Work Done

Type: Reference + Explanation

Description: A Work Done entry certifies the amount of work completed by a contractor in a specific billing period. It is the measurement document that flows into payment: `Work Order → Work Done (Approved) → Expense Booking → Payment`.

Path: `/engineering/work-done`

Permission guard: `checkPermissionForMethod("Engineering", "WorkDone")`

API Endpoints

Method	Path	Description
GET	<code>/api/engineering/work-done?page=&limit=&status=</code>	Paginated list
GET	<code>/api/engineering/work-done/:id</code>	Single entry
POST	<code>/api/engineering/work-done</code>	Create; auto-submits <code>Draft → Pending</code>
PUT	<code>/api/engineering/work-done/:id</code>	Update; <code>guardEdit</code> blocks post-approval edits
DELETE	<code>/api/engineering/work-done/:id</code>	Soft-delete (<code>Status = 'Deleted'</code>); restricted to <code>admin</code> and <code>manager</code> roles
PUT	<code>/api/engineering/work-done/:id/submit</code>	<code>Draft → Pending</code>
PUT	<code>/api/engineering/work-done/:id/approve</code>	<code>Pending → Approved</code>
PUT	<code>/api/engineering/work-done/:id/reject</code>	<code>Pending → Rejected</code>
GET	<code>/api/engineering/work-done/:id/create-po-prefill</code>	Returns Work Done shaped as a PO prefill payload — only for <code>Approved</code> entries

Database Table: `dbo.WorkDone`

Column	Type	Notes
ID	BIGINT PK	Auto-increment
DocNo	NVARCHAR(100)	Auto-generated via <code>lockNextDocNumber()</code>
DocTypeId	INT	FK → <code>dbo.TypeOfDoc</code>
DocDate	DATE	Required
CompanyId	INT	Required; FK → <code>dbo.enterprise</code>
ProjectId	INT	Required; FK → <code>dbo.enterprise</code>
FinYear	NVARCHAR(20)	Financial year string (e.g. "2024-25")
SupplierId	INT	Optional FK → <code>dbo.AccountHeadMaster</code>
WorkOrderID	INT	Optional FK → <code>dbo.WorkOrderHeader.Id</code>
PeriodFrom	DATE	Billing period start
PeriodTo	DATE	Billing period end
DescriptionOfWork	NVARCHAR(MAX)	Required
QuantityDone	DECIMAL(18,4)	
Unit	NVARCHAR(50)	Falls back to <code>UOMMaster</code> from linked Work Order if blank
RatePerUnit	DECIMAL(18,4)	
GrossAmount	DECIMAL(18,2)	$\text{QuantityDone} \times \text{RatePerUnit}$ (computed if not supplied)
Deductions	DECIMAL(18,2)	Amount to deduct from gross
CertifiedAmount	DECIMAL(18,2)	$\text{GrossAmount} - \text{Deductions}$ (computed if not supplied)
Status	NVARCHAR(50)	See status lifecycle below
Remarks	NVARCHAR(MAX)	
CreatedBy	NVARCHAR(100)	User email
CreatedAt	DATETIME (via <code>SYSDATETIME()</code>)	
UpdatedBy	NVARCHAR(100)	
UpdatedAt	DATETIME	

Status Lifecycle

Draft → Pending → Approved
→ Rejected → Pending (re-submit on next save)

Deleted ← admin/manager only, soft-delete via Status column

On every POST, the backend auto-submits **Draft → Pending** immediately after insert. On PUT, if the current status is **Draft** or **Rejected**, it auto-resubmits to **Pending**.

Period Overlap Guard

If a **WorkOrderID**, **PeriodFrom**, and **PeriodTo** are all provided, the backend checks for overlapping periods on that Work Order before allowing the insert or update:

```
SELECT COUNT(1) AS cnt
FROM dbo.WorkDone
WHERE WorkOrderID = @WOId
      AND PeriodFrom ≤ @PTo AND PeriodTo ≥ @PFrom
      AND ID <> @SelfId -- excluded on PUT
```

If an overlap is found, HTTP 409 is returned. This prevents double-billing the same period on a single Work Order.

CertifiedAmount Computation

```
const gross = body.GrossAmount ?? (quantity * rate);
const certified = body.CertifiedAmount ?? (gross - deductions);
```

If the caller supplies **GrossAmount** and **CertifiedAmount** explicitly, those values are used directly. Otherwise the backend computes them. This allows the frontend to override the auto-calculation for adjustments.

PO Prefill Endpoint

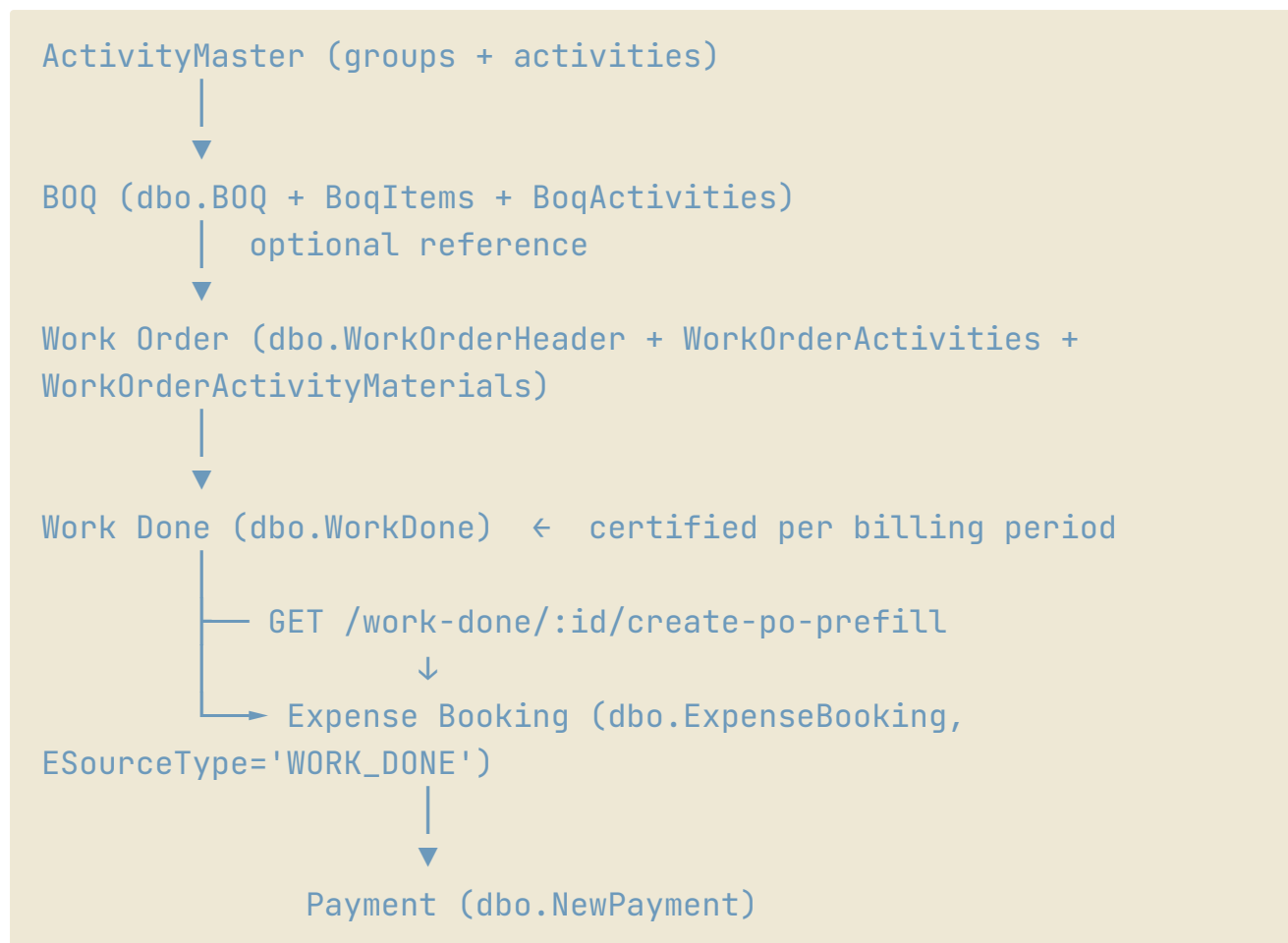
GET /engineering/work-done/:id/create-po-prefill is consumed by the Material module's Purchase Order create form. It returns the Work Done entry shaped as a PO payload with a single line item derived from **DescriptionOfWork**, **QuantityDone**, **RatePerUnit**, and **CertifiedAmount**. Only **Approved** Work Done entries are eligible — any other status returns HTTP 400.

Delete Behaviour

DELETE is a soft-delete: it sets `Status = 'Deleted'` rather than removing the row. It is restricted to `admin` and `manager` roles. Deleted entries are not filtered out by the list query's `status` filter unless the caller requests `?status=Approved` or similar — they remain visible in the full list.

Cross-Module Document Chain

Work Done is the bridge between the Engineering and Finance/Material modules:



The `work-order-summary` endpoint traverses this entire chain to compute `totalCertified`, `totalBooked`, `totalPaid`, and `balance` for any given Work Order.

Permission Module Map

All Engineering routes use `checkPermissionForMethod` with `"Engineering"` as the module:

Sub-module string	Route
BOQ	BOQ
WorkOrders	Work Order
WorkDone	Work Done

Activity Master uses role-level auth only (no submodule permission string).

Cache Keys Reference

Cache Key	TTL	Busted By
activity-master	300s	Any activity create / update / delete / approve
wo-meta-act-groups	600s	Not busted (stable; re-fetched on service restart)
wo-meta-companies	600s	Not busted
wo-meta-projects	600s	Not busted
wo-meta-contractors	600s	Not busted
work-orders	default	Any WO create / update / delete / status change
boq	default	Any BOQ create / update / delete / status change
engineering-work-done	120s	Any Work Done create / update / delete / status change
engineering-dashboard	default	Any Work Done or WO status change
eng-wo-with-activities	300s	Not explicitly busted (stale after 5 min)
eng-approved-boqs	120s	Not explicitly busted

Common Pitfalls

Deleting an Activity Group that has child Activities. There is no cascade delete or FK enforcement on `group_id`. Child Activities lose their group reference and appear as "Ungrouped" in the tree view. Delete or reassign child Activities before deleting the Group.

HSN code silently cleared on Group save. If you accidentally create a record as `activity_type = 0` (Group) with an HSN code, the backend will store `NULL` for `hsn_code` regardless. Check `activity_type` before saving.

Work Done period overlap rejected with HTTP 409. The overlap check uses `PeriodFrom ≤ PeriodTo` range intersection. Even a one-day overlap with an existing entry on the same Work Order will be rejected. Create a new Work Order if you need to split billing.

`dbo.WorkDone` migration not run. The dashboard and Work Done list use `hasTable()` to detect the table. If it is missing, the dashboard returns zeroes for Work Done KPIs and the Work Done list returns a 500 with the message: `"dbo.WorkDone is missing. Run backend/migrations/053-create-work-done.sql in SSMS."` Run the migration before using the Work Done page.

Editing an Approved BOQ or Work Order. `guardEdit()` will return HTTP 400 with an error like `"Cannot edit an Approved record"`. To make changes, reject the record first (`/reject`), edit, then re-submit.

`eng-wo-with-activities` cache is not busted on WO approval. This endpoint (used by the Work Done picker to list available Approved WOs) has a 300s TTL but no explicit `bumpCacheVersion` call on WO status changes. After approving a Work Order, the Work Done form may not show it for up to 5 minutes. A manual page refresh will force a cache miss.

Ticket Module

Type: Reference + Explanation

Overview

The Ticket module provides a full support ticket lifecycle: creation, assignment, real-time chat, file attachments, escalation, and resolution. It is self-contained under the `/ticket` route and has a separate Admin panel under `/admin`.

All endpoints sit under `/api/tickets` and require a valid JWT (enforced globally by `authMiddleware` in `server.js`). Real-time updates are delivered via Socket.IO — all mutations emit a `ticket:updated` event that causes live-polling frontends to refetch immediately.

Database Tables

dbo.tickets

The primary table. All columns are listed below.

Column	Type	Notes
id	INT IDENTITY	Primary key
subject	NVARCHAR(500)	Required
priority	NVARCHAR(50)	Urgent , High , Medium , Low — default Medium
issue_details	NVARCHAR(MAX)	Required
customer_name	NVARCHAR(255)	Required — selected from dbo.AccountHeadMaster at creation
customer_phone	NVARCHAR(50)	Optional, auto-filled when customer is selected
company_id	INT	Optional FK to company
project_id	INT	Optional FK to project
attachment_path	NVARCHAR(MAX)	Legacy field — now unused; attachments live in dbo.ticket_attachments
status	NVARCHAR(50)	Pending , InProgress , Resolved , Closed — default Pending
assigned_to	NVARCHAR(255)	Assignee display name
assigned_to_id	INT	Assignee user ID
resolved_by	NVARCHAR(255)	Resolver display name
resolved_by_id	INT	Resolver user ID
resolution_note	NVARCHAR(MAX)	Optional note written at resolution time
created_by	NVARCHAR(255)	Creator display name
created_by_id	INT	Creator user ID
escalated_at	DATETIME2	Set by the auto-escalation job; NULL means not escalated
escalation_level	INT	Default 0 ; set to 1 when SLA breached
escalation_reason	NVARCHAR(500)	Human-readable reason written by the escalation job
created_at	DATETIME2	SYSUTCDATETIME()
updated_at	DATETIME2	Updated on every state change

Column	Type	Notes
resolved_at	DATETIME2	Set when status → Resolved
closed_at	DATETIME2	Set when status → Closed

Indexes: IX_tickets_status , IX_tickets_priority , IX_tickets_assigned_to_id , IX_tickets_created_by_id , IX_tickets_escalation_open (composite on status , escalated_at , priority).

dbo.ticket_comments

Each row is one message in the ticket thread.

Column	Type	Notes
id	INT IDENTITY	Primary key
ticket_id	INT	FK → dbo.tickets(id) ON DELETE CASCADE
comment	NVARCHAR(MAX)	Message text
author_name	NVARCHAR(255)	Display name of the commenter
author_id	INT	Nullable — NULL for system-generated comments (e.g. escalation)
author_role	NVARCHAR(50)	Role string at time of comment
is_internal	BIT	1 = visible to admins only; 0 (default) = visible to all parties
created_at	DATETIME2	SYSDATETIME()

Indexes: IX_ticket_comments_ticket (on ticket_id , created_at), IX_ticket_comments_ticket_internal (on ticket_id , is_internal , created_at).

Internal comments: Non-admin users never receive rows where is_internal = 1 . The backend filters them out in GET /api/tickets/:id .

dbo.ticket_attachments

Binary file storage. Files are never written to disk.

Column	Type	Notes
id	INT IDENTITY	Primary key
ticket_id	INT	FK → <code>dbo.tickets(id)</code> ON DELETE CASCADE
comment_id	INT	<code>NULL</code> = ticket-level; non-null = attached to a specific comment
filename	NVARCHAR(255)	Original filename
mime_type	NVARCHAR(100)	e.g. <code>image/jpeg</code> , <code>application/pdf</code>
file_size	INT	Bytes
file_data	VARBINARY(MAX)	Binary content of the file
uploaded_by	INT	User ID of uploader
uploaded_at	DATETIME2	<code>SYSUTCDATETIME()</code>

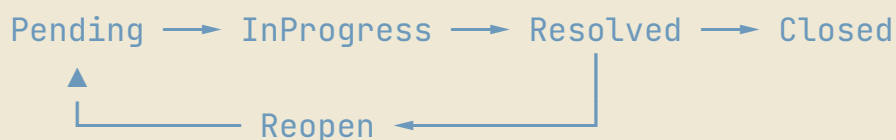
Allowed types: `jpeg`, `jpg`, `png`, `gif`, `webp`, `pdf`. Max size: **10 MB per file**, up to 20 files per upload call (enforced by Multer).

The legacy route `GET /api/tickets/file/:filename` returns `410 Gone` — all disk-based attachments were migrated. All new attachments serve from `GET /api/tickets/attachment/:id`.

dbo.users.can_accept_tickets

A `BIT` column added to `dbo.users`. When `1`, that user can call the Accept endpoint even if their role is not `admin`, `super_admin`, or `dba`. The `canAcceptTickets` middleware checks this flag against the DB on every Accept request for non-privileged roles.

Status Lifecycle



Status	When it is set
Pending	On ticket creation. Also set on Reopen (clears all resolution fields).

Status	When it is set
InProgress	When a comment is posted to a Pending ticket (automatic), or when the Accept / Assign endpoints are called.
Resolved	Explicitly via PUT /api/tickets/resolve/:id . Resolution note and resolver are recorded.
Closed	admin / super_admin / dba only via PUT /api/tickets/close/:id . Permanent unless manually reopened.

Important: The InProgress status is also computed on the fly. The list and detail endpoints use a CASE expression: if status = 'Pending' and comment_count > 0 , the returned row shows InProgress . This means the DB column may still read Pending while the API returns InProgress . This is intentional — it allows PUT /api/tickets/accept/:id to use WHERE status = 'Pending' as its atomic guard against double-acceptance.

Priority Values

Urgent · High · Medium (default) · Low

API Endpoints

List and Stats

Method	Path	Auth	Description
GET	/api/tickets	ticket-admin roles only	Paginated list of all tickets with optional status filter
GET	/api/tickets/stats	ticket-admin roles only	Aggregate counts (total, pending, in_progress, resolved, closed, escalated_open, urgent_open, high_open)
GET	/api/tickets/my	Any authenticated user	Tickets where created_by_id = actor.id OR

Method	Path	Auth	Description
			<code>assigned_to_id = actor.id</code>
GET	<code>/api/tickets/mine</code>	Any authenticated user	Alias for <code>/my</code>
GET	<code>/api/tickets/admin-users</code>	<code>admin</code> , <code>super_admin</code> , <code>dba</code>	Users eligible for ticket assignment — those with <code>can_accept_tickets = 1</code> or a privileged role

Pagination: `GET /api/tickets` accepts `?page=` (default 1) and `?limit=` (default 25, max 100). Response shape:

```
{
  data: Ticket[];
  pagination: {
    page: number;
    limit: number;
    total: number;
    totalPages: number;
  };
}
```

Status filter: `?status=Pending,InProgress` — comma-separated, validated against `VALID_STATUSES`. Each value is bound as a named parameter (no string interpolation).

`unwrapTicketList()` — because some endpoints return a plain array and others return `{ data, pagination }`, the frontend normalises both via this utility:

```
// src/lib/ticketListResponse.ts
export function unwrapTicketList<T>(payload: unknown):
TicketListResponse<T>
```

Single Ticket

Method	Path	Auth	Description
GET	/api/tickets/:id	Creator, assignee, or ticket-admin	Returns { ticket, comments, attachments }. Non-admin users receive comments filtered to is_internal = 0 only.

`getAccessibleTicket()` is the shared access guard used by all single-ticket endpoints. It returns { status: 404 }, { status: 403 }, or { status: 200, ticket }.

Create

Method	Path	Auth	Description
POST	/api/tickets	Any authenticated user	Create a ticket
POST	/api/tickets/create	Any authenticated user	Alias

Required body fields: `subject`, `issue_details`, `customer_name`. **Optional:** `priority` (defaults to `Medium`), `customer_phone`, `company_id`, `project_id`, `attachmentIds` (array of pre-uploaded attachment IDs to link).

Creation is a two-step process on the frontend: first `POST /api/tickets/create` to get a `ticketId`, then `POST /api/tickets/upload` for each file. The upload call receives the `ticketId` in the form body and links the attachment row via `UPDATE ... SET ticket_id = @ticket_id`.

State Mutations

Method	Path	Guard
PUT /api/tickets/accept/:id	<code>canAcceptTickets</code> middleware	Sets <code>assigned_to</code> , <code>assigned_to_id</code> ,

Method	Path	Guard
		<code>status = 'InProgress' .</code> Uses <code>WHERE</code> <code>status = 'Pending' AND assigned_to_id IS NULL</code> — returns <code>409</code> if already accepted. Also writes a <code>[Accepted]</code> system comment.
<code>PUT</code> <code>/api/tickets/assign/:id</code>	<code>admin</code> , <code>super_admin</code> , <code>dba</code>	Force-assigns to any user. Transitions <code>Pending → InProgress</code> if needed. Does not guard against double-assignment.
<code>PUT</code> <code>/api/tickets/resolve/:id</code>	Creator, assignee, or ticket-admin	Sets <code>status = 'Resolved'</code> , stamps <code>resolved_at</code> , stores <code>resolution_note</code> . Optional <code>rating</code> and <code>review_remarks</code> are appended to the comment.
<code>PUT</code> <code>/api/tickets/reopen/:id</code>	Creator, assignee, or ticket-admin	Sets <code>status = 'Pending'</code> . Clears <code>resolved_by</code> , <code>resolved_by_id</code> , <code>resolution_note</code> , <code>resolved_at</code> , <code>closed_at</code> .
<code>PUT</code> <code>/api/tickets/close/:id</code>	<code>admin</code> , <code>super_admin</code> , <code>dba</code> only	Sets <code>status = 'Closed'</code> , stamps <code>closed_at</code> .
<code>PUT</code> <code>/api/tickets/status/:id</code>	Creator, assignee, or ticket-admin	Generic status override — sets

Method	Path	Guard
		any valid status value directly.

Comments

Method	Path	Auth	Description
POST	/api/tickets/comment/:id	Creator, assignee, or ticket-admin	Posts a comment. Accepts { comment, attachmentIds } . At least one must be present. Automatically transitions Pending → InProgress if a comment is the first one on a pending ticket. Links pre-uploaded attachments to the new comment ID.

Attachments

Method	Path	Auth	Description
POST	/api/tickets/upload	Creator, assignee, or ticket-admin	Uploads 1–20 files. Requires ticketId in body or query. Returns { success, url, urls, attachments } .
GET	/api/tickets/attachment/:id	Creator, assignee, or	Streams binary from DB.

Method	Path	Auth	Description
		ticket-admin	Validates ticket access before serving. Headers: Content-Type , Content-Disposition: inline , Cache-Control: private, max-age=86400 .
GET	/api/tickets/file/:filename	Any authenticated user	Legacy — returns 410 Gone.

Role Model

Ticket-admin roles (checked by `isTicketAdmin()`): `admin`, `super_admin`, `dba`, `engineer`. These roles can access all tickets, see internal comments, and use admin-only endpoints (Assign, Close, Stats, List-all, Admin-users).

`can_accept_tickets` flag (`dbo.users.can_accept_tickets`): allows any non-admin user to accept tickets via `PUT /api/tickets/accept/:id`. The `canAcceptTickets` middleware checks this by querying the DB if the JWT payload does not carry the flag.

Standard users can only see their own created or assigned tickets. They never receive internal comments.

Real-Time Sync (Socket.IO)

Every mutation in `ticketRoutes.js` calls `emitTicketUpdate(action, ticketId)` which emits a `ticket:updated` event to all connected clients.

The frontend subscribes via `useTicketSync()`:

```
// src/hooks/useTicketSync.ts
export function useTicketSync(refetch: RefetchFn, ticketId?:
number | null)
```

Behaviour:

- If `ticketId` is provided, the hook joins the `ticket:${ticketId}` Socket.IO room and only refetches when the event matches that ID.
- If `ticketId` is not provided (list views), any `ticket:updated` or `ticket:escalated` event triggers a full refetch.

The escalation service emits two events simultaneously: `ticket:escalated` (with `count` and `ticketIds`) and `ticket:updated` (with `action: 'escalated'`).

`invalidateTicketQueries()` is the TanStack Query counterpart – it invalidates all query keys in the ticket module at once:

```
const TICKET_QUERY_ROOTS = new Set([
  "ticket-dashboard", "tickets", "admin-resolution-tickets",
  "admin-tickets", "admin-ticket-stats", "admin-ticket-detail",
  "ticket-detail", "home-dashboard",
]);
```

Auto-Escalation Service

File: `backend/services/ticketEscalationService.js`

The escalation job runs periodically (scheduled externally – typically via a `setInterval` in `server.js`). It finds all open (`Pending` or `InProgress`) tickets that have not been escalated yet and whose age exceeds the SLA threshold for their priority.

Default SLA thresholds:

Priority	Default (minutes)	Env variable override
Urgent	30	<code>TICKET_ESCALATION_URGENT_MINUTES</code>
High	120	<code>TICKET_ESCALATION_HIGH_MINUTES</code>
Medium	480	<code>TICKET_ESCALATION_MEDIUM_MINUTES</code>
Low	1440	<code>TICKET_ESCALATION_LOW_MINUTES</code>

On breach, the job performs a single `UPDATE ... OUTPUT` statement that atomically:

1. Sets `escalation_level = 1`, `escalated_at = SYSUTCDATETIME()`.
2. Writes `escalation_reason` (e.g. `"Open urgent priority ticket exceeded 30 minute SLA"`).
3. Returns the updated rows.

For each escalated ticket, the job then inserts a system comment (`author_name = 'System'`, `author_id = NULL`, `is_internal = 0`) with the escalation reason prefixed `[Escalated]`.

Tickets already escalated (`escalated_at IS NOT NULL`) are never touched again. The service only escalates once per ticket.

TicketChat Component

File: `src/pages/ticket/TicketChat.tsx`

A reusable component embedded in both `MyTickets.tsx`, `PendingTickets.tsx`, and `AdminTicketPanel.tsx`. It manages the message thread for a single ticket.

Props:

Prop	Type	Description
<code>ticketId</code>	<code>number</code>	The ticket to display
<code>currentUser</code>	<code>{ id, name, role }</code>	The logged-in user
<code>initialMessages</code>	<code>TicketChatMessage[]</code>	Pre-loaded comments from the parent query
<code>ticketStatus</code>	<code>string</code>	Optional — used to disable the input on closed tickets
<code>onSent</code>	<code>() ⇒ void</code>	Callback fired after a comment is posted successfully

Internal comment toggle: Admin-role users see a `Switch` control to mark a comment as internal before sending. The `is_internal` flag is sent to `POST /api/tickets/comment/:id`.

Typing indicators: The component emits `ticket:typing` events over Socket.IO and displays a "typing" overlay when another user is actively

typing in the same thread.

Attachment rendering in comments

Comments can embed legacy attachment tokens in the text body:

```
[attachment:/api/tickets/file/1779436898263-721161.pdf]
```

`parseComment()` splits the comment string into text and attachment segments. The leading slash is optional (old DB data omits it). Image segments render as thumbnails with a lightbox; non-image segments (PDFs) render as a download badge.

`useBlobUrl()` fetches auth-protected attachment URLs through `fetchWithAuth` and returns a `blob:` URL. Results are cached in a module-level `Map<string, string>` (revoked on `beforeunload`). This avoids re-fetching the same binary on re-renders.

Frontend Pages

Ticket Dashboard (`/ticket`)

Fetches `/api/tickets?limit=100` (admins/engineers) or `/api/tickets/mine` (standard users). Computes all stats client-side via `useMemo`. Polls every 30 seconds and immediately refetches on `ticket:updated` Socket.IO events.

Stat cards: Total, Pending+InProgress, Resolved, Urgent+High open. Each card is clickable and navigates to the relevant view.

Charts: Priority breakdown bar (open tickets only) and Status overview bar — both rendered as CSS progress bars, not a charting library.

Create Ticket (`/ticket/create`)

Required fields: Subject, Customer Name, Issue Details. **Optional:** Company, Project (from `/api/business/dropdown`), Phone, Priority (default `Medium`), Attachments.

Customer selection uses a shadcn `Command` combobox. Options come from `GET /api/account-head?type=A` — the same `dbo.AccountHeadMaster` table

used elsewhere. Selecting a customer auto-fills the phone field from `LHeadPhone`.

Attachment flow:

1. Ticket is created first (`POST /api/tickets/create`) to obtain `ticketId`.
2. Files (from file picker or webcam capture) are uploaded one by one via `POST /api/tickets/upload` with `ticketId` in the form body.

Webcam capture uses `react-webcam`. The component prefers `facingMode: 'environment'` (rear camera). Camera permission errors are caught and displayed inline.

Pending Tickets (`/ticket/pending`) — Admin only

Lists all `Pending` and `InProgress` tickets. Displays resolution panel inline using `TicketChat`. Supports Accept, Resolve, Reopen, and comment actions. Also allows capturing a new photo and uploading it to the ticket.

Sentiment rating: On resolve, an optional 3-point rating (Unhappy / Happy / Excellent → values 1 / 4 / 5) and free-text remarks are included in the resolve request body. These are appended to the ticket comment as `[Review N★] remarks`.

My Tickets (`/ticket/my-tickets`)

Fetches `/api/tickets/mine`. Shows all tickets the current user created or is assigned to. Groups by open vs. resolved. Allows the ticket owner to Reopen a resolved ticket.

Resolved Tickets (`/ticket/resolved`)

Fetches from `/api/tickets?status=Resolved,Closed` (admins) or `/api/tickets/mine` filtered client-side (non-admins). Read-only list with resolution notes.

Admin Ticket Panel (`/admin` → Tickets tab)

Full admin view. Uses `/api/tickets` with pagination and `/api/tickets/stats` for the header counters. Supports Assign (via dropdown of eligible users from `/api/tickets/admin-users`), Close, and inline chat. Displays escalation badges (`escalated_at IS NOT NULL`).

Navigation (Sidebar)

File: `src/components/layout/sidebars/TicketSidebar.ts`

```
export const buildTicketNavItems = (isAdminUser: boolean): NavItem[]
```

Label	Path	Visibility
Dashboard	<code>/ticket</code>	All
Create Ticket	<code>/ticket/create</code>	All
My Tickets	<code>/ticket/my-tickets</code>	All
Pending Tickets	<code>/ticket/pending</code>	Admin only
Resolved Tickets	<code>/ticket/resolved</code>	All

`isAdminUser` is `true` for roles `super_admin`, `admin`, `dba`.

Common Pitfalls

Accept race condition: Two users clicking Accept simultaneously will result in one receiving a `409 Conflict`. The backend guards this with `WHERE status = 'Pending' AND assigned_to_id IS NULL`. The frontend should handle the 409 gracefully and refresh the ticket list.

Computed `InProgress` vs. DB `status`: The `status` column in `dbo.tickets` may read `Pending` while the API returns `InProgress`. Never read ticket status directly from the DB for display purposes — always use the API response which applies the `CASE` expression.

Internal comments leak: The `GET /api/tickets/:id` endpoint applies the `is_internal` filter. However, `GET /api/tickets/my` and `GET /api/tickets` do not expose individual comments — they only return `comment_count`. Internal comments are only visible in the detail endpoint, where filtering is applied.

Legacy attachment tokens: Comments written before migration 078 may contain `[attachment:api/tickets/file/...]` tokens (no leading slash) pointing to files that no longer exist on disk. `parseComment()` handles the missing slash, but `GET /api/tickets/file/:filename` will return `410`. The UI renders a "Failed" badge in this case.

Escalation is one-shot: The job uses `WHERE escalated_at IS NULL` — a ticket is only escalated once regardless of how long it stays open. `escalation_level` is always `1`; there is no multi-level escalation implemented.

`can_accept_tickets` **JWT lag:** The JWT payload may not carry the latest `can_accept_tickets` value if the flag was toggled after login. `canAcceptTickets` middleware falls back to a live DB query for non-privileged roles, so the actual check is always current. The JWT value is only used as an optimisation to skip the DB query.

Admin Module

Type: Reference + Explanation **Framework:** Diátaxis

Overview

The Admin module controls authorization, user management, system configuration, and cross-cutting operational concerns. It is exposed under the `/admin` route (and `/superadmin` for the top-level tenant panel). Access is role-gated at two levels: coarse-grained via `allowRoles()` middleware and fine-grained via the `checkPermission()` system backed by `dbo.RoleRights` and `dbo.UserPageRightsJson`.

Authorization Architecture

There are two independent authorization layers in the system. Both must pass for a non-privileged user to access a resource.

Layer 1 — `allowRoles()` (Coarse, Route-Level)

File: `backend/middleware/role.js`

Reads the `role` field from the JWT and normalises it via `normalizeRole()` before comparing to the allowed list. Normalisation maps human-readable DB strings to canonical slugs.

```
allowRoles("admin", "super_admin", "dba")
// 401 if no token, 403 if role not in list
```

Canonical role slugs:

Slug	Aliases accepted from DB
super_admin	"Super Admin", "superadmin", "super administrator", "sa"
admin	"Admin", "administrator", "system admin", "system administrator"
dba	"DBA", "db admin", "database admin", "db_admin"
engineer	"engineer", "site engineer", "field engineer"
customer	"customer", "customer demo", "client", "end user", "portal user"
branch_manager	"branch manager", "branch admin"
finance_manager	"finance manager", "finance", "accountant"
store_manager	"store manager", "material manager"
user	"user", "employee", "staff", "standard user"

Unknown role strings fall through unchanged (lowercased) — they will be denied unless the route explicitly allows that slug.

Layer 2 — `checkPermission()` (Fine-Grained, Module/Action)

File: `backend/middleware/permissions.js`

Used on routes that require per-module action gates beyond just role membership.

```
checkPermission("Finance", "Payments", "CanAdd")
// or auto-mapped from HTTP method:
checkPermissionForMethod("Material", "GRN") // GET→CanView,
POST→CanAdd, etc.
```

Superuser bypass: Roles `super_admin`, `dba`, `admin` (and `sa`) always pass — they never need a `RoleRights` row. Without this, privileged users with no explicit row would get silent 403s.

Permission resolution order:

1. If role is in `SUPERUSER_ROLES` → allow.
2. Query `dbo.RoleRights` for `(RoleId, Module, SubModule)` → check the relevant `CanView/CanAdd/CanEdit/CanDelete` bit.
3. If no role-level row, fall back to `dbo.UserPageRightsJson` for user-level overrides.
4. Deny with 403.

In-process cache: Both `permissionCache` (role-level) and `userPermissionCache` (user-level) use an in-process `Map` with a 5-minute TTL. After any write to `dbo.RoleRights`, call `permissionCache.invalidateRole(roleId)` to evict immediately. This is done automatically by `POST /api/roles/:roleId/rights`.

`checkPermissionForMethod()` (`backend/middleware/routePermission.js`) maps HTTP verbs to permission actions automatically:

HTTP Method	Permission Action
<code>GET</code> , <code>HEAD</code> , <code>OPTIONS</code>	<code>CanView</code>
<code>POST</code>	<code>CanAdd</code>
<code>PUT</code> , <code>PATCH</code>	<code>CanEdit</code>
<code>DELETE</code>	<code>CanDelete</code>

Database Tables

`dbo.Role`

Stores all role definitions.

Column	Type	Notes
<code>RIId</code>	<code>INT IDENTITY</code>	Primary key
<code>RName</code>	<code>NVARCHAR(100)</code>	Must be unique
<code>RCode</code>	<code>NVARCHAR(20)</code>	Auto-generated: first 3 chars of single-word names, initials for multi-word
<code>RDesc</code>	<code>NVARCHAR(255)</code>	Optional description
<code>RCreatedBy</code>	<code>NVARCHAR(100)</code>	User email at time of creation
<code>RCreatedAt</code>	<code>DATETIME2</code>	Creation timestamp
<code>RUpdatedBy</code>	<code>NVARCHAR(100)</code>	

Column	Type	Notes
RUpdatedAt	DATETIME2	
RApprovedBy	NVARCHAR(100)	
RApprovedAt	DATETIME2	

dbo.RoleRights

Stores the CRUD permission matrix per role per module/submodule combination.

Column	Type	Notes
RoleId	INT	FK → <code>dbo.Role.RId</code>
Module	NVARCHAR(100)	e.g. "Finance", "Material", "Rights", "User Control"
SubModule	NVARCHAR(100)	e.g. "Payments", "GRN", "Menu", "Manage Users"
CanView	BIT	
CanAdd	BIT	
CanEdit	BIT	
CanDelete	BIT	

Rows are replaced wholesale on save — `POST /api/roles/:roleId/rights` runs `DELETE FROM dbo.RoleRights WHERE RoleId=@RoleId` then re-inserts all rows in a transaction.

Known module:submodule pairs and their frontend page keys:

Module	SubModule	Page key
Rights	Menu	menu-rights
Rights	Role Master	roles
Rights	Widgets	widgets-rights
Rights	Financial Year	fin-year
User Control	Manage Users	users
UserActivity	List	activity-browser

dbo.UserPageRightsJson

User-level permission overrides stored as a JSON blob. Takes precedence over `RoleRights` when no role-level row exists.

Column	Type	Notes
<code>UserId</code>	<code>INT</code>	FK → <code>dbo.users.id</code>
<code>RightsJson</code>	<code>NVARCHAR(MAX)</code>	JSON array: <code>[{ page: "grn-master", actions: ["view","create","edit"]}]</code>
<code>IsActive</code>	<code>BIT</code>	Only rows where <code>IsActive = 1</code> are read

The `actions` array uses the strings `"view"`, `"create"`, `"edit"`, `"delete"`, `"print"`, `"export"`.

dbo.PageDefinitions

The DB-backed registry of all page keys available for permission assignment. Replaces the old hardcoded array in `MenuRights.tsx`.

Column	Type	Notes
<code>PageDefId</code>	<code>INT</code> <code>IDENTITY</code>	Primary key
<code>PageKey</code>	<code>NVARCHAR</code>	Stable identifier used in <code>UserPageRightsJson</code> (e.g. <code>"grn-master"</code>)
<code>Label</code>	<code>NVARCHAR</code>	Human-readable display name
<code>Module</code>	<code>NVARCHAR</code>	Module grouping (e.g. <code>"Material"</code> , <code>"Finance"</code>)
<code>GroupName</code>	<code>NVARCHAR</code>	Sub-group within the module
<code>Actions</code>	<code>NVARCHAR</code>	Comma-separated allowed actions (e.g. <code>"view,create,edit,delete"</code>)
<code>SortOrder</code>	<code>INT</code>	Display order
<code>IsActive</code>	<code>BIT</code>	Soft-delete flag — <code>0</code> hides from <code>MenuRights</code>

`GET /api/page-definitions` returns only `IsActive = 1` rows. `GET /api/page-definitions/all` returns all (super_admin only). Soft-delete via `DELETE /api/page-definitions/:id` sets `IsActive = 0` — existing `UserPageRightsJson` entries referencing that key are unaffected.

dbo.MenuMaster

Named menu entry definitions. Used to populate document-type name dropdowns in transaction modules (e.g. the name that appears on a Payment or Purchase Order header).

Column	Type	Notes
Id	INT IDENTITY	Primary key
Name	NVARCHAR(200)	Required, the display label
Description	NVARCHAR(500)	Optional
CreatedBy	INT	User ID
UpdatedBy	INT	User ID
CreatedAt	DATETIME2	
UpdatedAt	DATETIME2	

Cached at "menu-master" key for 300 seconds. Cache is bumped on every create/update/delete.

dbo.MenuType

Configures the display labels for key document types per entry configuration. Each row stores named overrides for up to 5 document categories.

Column	Type	Notes
Id	INT IDENTITY	Primary key
MenuReceipt	NVARCHAR(200)	Label for the Receipt document type
MenuPayment	NVARCHAR(200)	Label for the Payment document type
MenuBOQ	NVARCHAR(200)	Label for BOQ
MenuPurchaseOrder	NVARCHAR(200)	Label for Purchase Order
MenuWorkOrder	NVARCHAR(200)	Label for Work Order
CreatedBy	NVARCHAR(100)	
UpdatedBy	NVARCHAR(100)	
ApprovedBy	NVARCHAR(100)	
CreatedAt	DATETIME	
UpdatedAt	DATETIME	
ApprovedAt	DATETIME	

dbo.Entry_Type

Controls document numbering and prefix generation per project and entry category.

Column	Type	Notes
E_Id	UNIQUEIDENTIFIER	Primary key
project_id	INT	FK → <code>dbo.enterprise.id</code> (where <code>business_type = 'P'</code>)
Epname	NVARCHAR(255)	Project name — kept in sync with <code>dbo.enterprise.name</code> on every write
EntryType	NVARCHAR	Entry category label (e.g. <code>"Purchase"</code> , <code>"Payment"</code>)
Eprefix	NVARCHAR	Document number prefix — auto or custom, computed from project initials + entry type
EDoc_N	INT	Starting document sequence number (default 1)
E_CreatedBy	NVARCHAR(100)	Creator email
E_CreatedAt	DATETIME2	

Cached at `"entry-type"` for 300 seconds. Cache is bumped on every mutation.

The frontend `Eprefix` field supports two modes: `"auto"` (generates from first 2 letters of project name + first 3 letters of entry type) and `"custom"` (free text).

Setup Panel (visible in screenshot)

The Setup panel (accessible to `admin` / `super_admin` / `dba`) contains four items:

Item	Path	Backend	Table
Entry Type	<code>/admin/masters/entry-type</code> (via <code>NamedEntryTypeMaster</code>)	<code>GET/POST/PUT/DELETE /api/entry-types</code>	<code>dbo.Entry_T</code>

Item	Path	Backend	Table
Type of Doc	/admin/masters/type-of-doc (via MenuMaster)	GET/POST/PUT/DELETE /api/menu-master	dbo.MenuMas
Role Master	/admin/masters/role-master	GET/POST/PUT/DELETE /api/roles	dbo.Role
Menu Types	/admin/masters/menu-types	GET/POST/PUT/DELETE /api/menu-types	dbo.MenuTyp

Role Sections by Authorization Level

Customer

Canonical slug: `customer` **Aliases:** `"customer demo"`, `"client"`, `"end user"`, `"portal user"`

The most restricted role. Customers access only the Ticket module. They cannot reach any Finance, Material, Engineering, or Admin pages.

Access summary:

- Ticket module: Create Ticket, My Tickets, Resolved Tickets (own tickets only)
- No access to Pending Tickets (admin-only sidebar item)
- No access to any `/admin/*` routes
- `GET /api/tickets/my` — sees only tickets where `created_by_id = self` OR `assigned_to_id = self`
- `GET /api/tickets/:id` — comments filtered: `is_internal = 1` rows are hidden
- Cannot call Accept, Assign, Close, or Stats endpoints
- No `RoleRights` rows needed — access is controlled entirely by `allowRoles()` at the route level

Cannot do:

- View or manage any financial, material, or engineering documents
- Access Admin module, Setup panel, Rights, or User Control
- See internal ticket comments
- Access `GET /api/tickets` (all-tickets list — admin only)

Engineer

Canonical slug: `engineer` **Aliases:** `"site engineer"`, `"field engineer"`

Engineers are ticket-admin capable and have read+create+edit access across Engineering, Material, and Ticket modules. They do not have financial access or admin privileges.

Access summary:

- Ticket module: all views including Pending Tickets (`isTicketAdmin = true` for `engineer`)
- Can Accept tickets (`canAcceptTickets` middleware: `engineer` is not in the privileged set — must have `can_accept_tickets = 1` on their user row, or be granted it via Admin)
- Engineering module: BOQ, Work Orders, Work Done
- Material module: Purchase Orders, GRN, Material Requests, Stock Ledger, Issues
- `GET /api/tickets` — full list access (`isTicketAdmin = true`)
- Internal ticket comments: visible (`isTicketAdmin = true`)
- `GET /api/admin-dashboard` — blocked (not in `allowRoles("admin", "super_admin", "dba")`)

Preset page grants (from `MenuRights.tsx` `ROLE_PRESETS`):

```
dashboard, engineering-dashboard, boq, engineering-work-order,
work-done,
purchase-orders, grn-master, work-order, material-request, stock-
ledger,
tickets, followup-applicants, followup-bookings, reports, tasks
```

Default actions: `view`, `create`, `edit`.

Cannot do:

- Access Finance module (payments, BRS, received payments)
- Access Admin module routes
- Assign or Force-close tickets
- Manage users, roles, or rights
- Access Setup panel (Entry Type, Role Master, Menu Types, Type of Doc)

Admin

Canonical slug: `admin` **Aliases:** `"administrator"`, `"system admin"`, `"system administrator"`

Admin is the operational management role. Admins bypass `checkPermission()` entirely (member of `SUPERUSER_ROLES`). They can access all user and rights management features but not super-admin-specific system configuration.

Access summary:

- Full `checkPermission()` bypass — no `RoleRights` row needed
- `GET /api/admin-dashboard` — allowed
- `GET /api/tickets` — full list, all statuses
- `PUT /api/tickets/assign/:id` — can force-assign any ticket
- `PUT /api/tickets/close/:id` — can close tickets
- `GET /api/tickets/stats` — allowed
- `GET /api/tickets/admin-users` — allowed
- User Control: Manage Users (`/users`), Activity Browser (`/admin/activity-browser`)
- Rights: Menu Rights (`/admin/rights/menu`), Widgets Rights (`/admin/rights/widgets`), Financial Year Rights (`/admin/rights/fin-year`)
- Enterprise: Enterprise, Company, Project masters
- Approval: Inbox, Approval Setup, Post Approval Rights
- Security: Password Reset
- Communicator: SMS, Email, WhatsApp setup + Integration Channels
- Masters: Contractor Categories, Godowns, Page Definitions
- Support Tickets: Resolution panel
- API Integration, Live Metrics, Signature pages

Cannot do (vs. `super_admin`):

- Access `GET /superadmin` (tenant management panel)
- Manage Role Master (requires `super_admin` sidebar path `/admin/masters/role-master`)
- Manage Menu Types (`/admin/masters/menu-types`)
- Access DBA Console (`/dba`)
- Create or modify `dbo.PageDefinitions` (`super_admin` only via `requireSuperAdmin` in `pageDefinitions.js`)

Sidebar: `buildAdminNavItems()` in `AdminSidebar.ts`

Super Admin

Canonical slug: `super_admin` **Aliases:** `"Super Admin"`, `"superadmin"`, `"super administrator"`, `"sa"`

The top-level organizational role. Has everything Admin has, plus tenant-level control, full system configuration, and DBA Console access.

Access summary:

- Full `checkPermission()` bypass
- All Admin capabilities (see above)
- `GET /superadmin` – tenant panel (`SuperAdminDashboard.tsx`)
- System Config:
 - API Integration (`/admin/api-integration`)
 - **Role Master** (`/admin/masters/role-master`) – `GET/POST/PUT/DELETE /api/roles` with `checkPermission("Rights", "Menu", *)`
 - **Menu Types** (`/admin/masters/menu-types`) – `GET/POST/PUT/DELETE /api/menu-types`
 - Signature (`/admin/signature`)
- **Page Definitions** (`/admin/page-definitions`) – full CRUD via `requireSuperAdmin` guard in `pageDefinitions.js`. Admin role cannot create/update/delete page definitions.
- DBA Console (`/dba`)

Sidebar: `superAdminNavItems` in `SuperAdminSidebar.ts`

Role Master operations: `GET /api/roles` – full role list with audit fields
`POST /api/roles` – create role (duplicate `RName` rejected) `PUT /api/roles/:id` – update; auto-regenerates `RCode` from new `RName` `GET /api/roles/:roleId/rights` – fetch role's current permission rows `POST /api/roles/:roleId/rights` – replace all permissions for the role (transactional delete+insert), then calls `permissionCache.invalidateRole(roleId)`

DBA

Canonical slug: `dba` **Aliases:** `"db admin"`, `"db administrator"`, `"database admin"`, `"database administrator"`, `"db_admin"`

Technical role with the same `checkPermission()` bypass as `super_admin` and `admin`. Primarily used for access to the DBA Console and low-level system tooling. Navigation mirrors `super_admin` for most paths.

Access summary:

- Full `checkPermission()` bypass
- All Admin capabilities
- DBA Console (`/dba`) — full access
- `GET /api/admin-dashboard` — allowed
- `PUT /api/tickets/assign/:id`, `PUT /api/tickets/close/:id` — allowed
- `GET /api/tickets/admin-users` — allowed
- `GET /api/user-rights/users` — allowed (the `adminOnly` guard includes `dba`)
- `PUT /api/user-rights/:userId` — can save any user's `UserPageRightsJson`

Effectively identical to `super_admin` at the backend level. The distinction is navigation — DBA users are not expected to manage enterprise structure or perform business-level configuration. The `requireSuperAdmin` check in `pageDefinitions.js` does block DBA from Page Definitions CRUD.

Setup Panel — Detailed API Reference

Entry Type

Route: `/api/entry-types` **Table:** `dbo.Entry_Type` **Cache key:** `"entry-type"` (300s TTL)

Endpoint	Description
<code>GET /api/entry-types/projects</code>	Returns projects from <code>dbo.enterprise</code> where <code>business_type = 'P'</code> — populates the project dropdown
<code>GET /api/entry-types</code>	All entry types with resolved <code>Epname</code> via LEFT JOIN on <code>dbo.enterprise</code>
<code>POST /api/entry-types</code>	Create entry type. Resolves <code>Epname</code> from <code>dbo.enterprise</code> at write time.
<code>PUT /api/entry-types/:id</code>	Update. Re-resolves <code>Epname</code> from enterprise on every update to stay in sync.

Endpoint	Description
<code>DELETE /api/entry-types/:id</code>	Hard delete by <code>UNIQUEIDENTIFIER</code> .

Key fields:

Field	Required	Notes
<code>project_id</code>	Yes	FK to <code>dbo.enterprise.id</code>
<code>EntryType</code>	No	Category label
<code>Eprefix</code>	No	Document prefix string
<code>EDoc_N</code>	No	Starting doc number, defaults to 1

Type of Doc (Menu Master)

Route: `/api/menu-master` **Table:** `dbo.MenuMaster` **Cache key:** `"menu-master"` (300s TTL)

Endpoint	Description
<code>GET /api/menu-master</code>	All entries ordered by name
<code>GET /api/menu-master/:id</code>	Single entry
<code>POST /api/menu-master</code>	Create. <code>Name</code> is required.
<code>PUT /api/menu-master/:id</code>	Update. <code>Name</code> is required.
<code>DELETE /api/menu-master/:id</code>	Hard delete.

Role Master

Route: `/api/roles` **Table:** `dbo.Role` + `dbo.RoleRights` **Cache key:** `"roles"` (120s), `"roles:list"` (120s) **Guard:** `checkPermission("Rights", "Menu", *)` on all write endpoints

Endpoint	Description
<code>GET /api/roles/list</code>	Minimal list (<code>RIId</code> , <code>RName</code>) — any authenticated user, used for dropdowns
<code>GET /api/roles</code>	Full list with audit fields — <code>checkPermission("Rights", "Menu",</code>

Endpoint	Description
	"CanView")
POST /api/roles	Create role. RName must be unique. RCode auto-generated.
PUT /api/roles/:id	Update role. Uniqueness re-validated excluding current row.
DELETE /api/roles/:id	Hard delete.
GET /api/roles/:roleId/rights	Returns current RoleRights rows as { page, actions } array
POST /api/roles/:roleId/rights	Replace all permissions in a transaction. Immediately invalidates permission cache.

Menu Types

Route: /api/menu-types **Table:** dbo.MenuType **Validation:** Zod schemas (menuTypeCreateSchema , menuTypeUpdateSchema)

Endpoint	Description
GET /api/menu-types	All rows ordered by Id ASC
GET /api/menu-types/:id	Single row
POST /api/menu-types	Create. All 5 label fields are optional NVARCHAR(200) .
PUT /api/menu-types/:id	Update. Also accepts ApprovedBy and ApprovedAt .
DELETE /api/menu-types/:id	Hard delete.

Rights Management

Menu Rights (/admin/rights/menu)

File: src/pages/admin/MenuRights.tsx **Backend:** GET/PUT /api/user-rights/:userId , GET /api/user-rights/users , GET /api/page-definitions

Assigns per-user page permissions. On load, fetches all active `PageDefinitions` from the DB, then lists all non-admin users from `GET /api/user-rights/users`. On selecting a user, loads their current `RightsJson` from `dbo.UserPageRightsJson`.

Actions available per page: `view`, `create`, `edit`, `delete`, `print`, `export` (as defined in `PageDefinitions.Actions` for that page).

Preset templates available for quick assignment:

Preset	Pages	Default Actions
Engineer	engineering-dashboard, boq, purchase-orders, grn-master, material-request, stock-ledger, tickets, reports...	view, create, edit
Finance	finance-dashboard, expense-booking, new-payment, received-payment, bank-master, brs, debit-note, reports...	view, create, edit, print, export
Store Manager	material-dashboard, purchase-orders, grn-master, material-issues, stock-ledger, stock-transfers, inventory-master...	view, create, edit, print, export
View Only	all active pages	view only

Saving calls `PUT /api/user-rights/:userId` with the full `rightsJson` array, then invalidates `userPermissionCache` for that user.

Widgets Rights (`/admin/rights/widgets`)

File: `src/pages/admin/WidgetsRights.tsx` **Backend:** `GET /api/user-widget-rights/users`, `GET/POST /api/user-widget-rights/:userId`

Table: `dbo.UserWidgetRights`

Controls which dashboard widgets each user can see. Completely separate from Menu Rights — uses its own table and API. Widget catalog is fetched from `/api/widget-catalog`. Admins toggle widgets on/off per user and save.

Financial Year Rights (`/admin/rights/fin-year`)

File: `src/pages/admin/FinYearRights.tsx` **Context:** `FinYearContext`

Manages Financial Year records and their lock state. A locked financial year blocks further transactions for that period. The `IsLocked` boolean is the only field editable here — no amount or date edits are permitted after creation.

Activity Browser (`/admin/activity-browser`)

Files: `src/pages/admin/Activitybrowser/` (7 files) **Guard:**
`checkPermission("UserActivity", "List", "CanView")`

Reads the user activity log with filters (user, date range, module, action). Provides analytics, pagination, chart view, and summary tabs.
`super_admin`, `admin`, and `dba` bypass the permission check.

Common Pitfalls

`checkPermission()` not finding a `RoleRights` row: If a non-superuser role has no row in `RoleRights` for the given module/submodule, the middleware first tries `UserPageRightsJson`. If that also has no matching entry, it returns `403 "Access denied (no permission)"`. The fix is either to add a `RoleRights` row via the Role Master or a user-level override via Menu Rights.

Permission cache staleness: After modifying `RoleRights` outside the normal Role Master UI (direct DB edit), the in-process cache will not reflect the change until its 5-minute TTL expires or the server restarts. The API automatically invalidates on `POST /api/roles/:roleId/rights` only.

`normalizeRole()` unknown slugs: If a DB role string doesn't match any alias in `ROLE_ALIASES`, it falls through as a lowercased slug. A role named `"Project Lead"` becomes `"project lead"` — it won't match any `allowRoles()` call unless you add it to the alias table. Check the DB role name against `ROLE_ALIASES` when users report unexpected 403s.

`requireSuperAdmin` in `pageDefinitions.js`: This guard checks `req.user?.role` against the string values `"superadmin"`, `"super_admin"`, `"SuperAdmin"`, or `req.user?.isSuperAdmin === true`. It does not use `normalizeRole()`. A DBA or admin cannot create/update/delete page definitions even though they bypass `checkPermission()`.

Menu Rights vs. `allowRoles()` : Menu Rights (`UserPageRightsJson`) and Widgets Rights control what the frontend shows and what `checkPermission()` permits on specific routes. They do not override `allowRoles()` guards. A user with a `view` right on `"admin-dashboard"` but role `customer` will still get 403 from the `allowRoles("admin", "super_admin", "dba")` guard on `GET /api/admin-dashboard`.

Reports & Widgets

Type: Reference **Framework:** Diátaxis

Both are accessible from the top navigation bar. Reports is a read-only data export surface. Widgets is a per-user interactive dashboard panel – the visible set is controlled by admin-assigned rights.

Reports

File: `src/pages/Reports.tsx` **Path:** `/reports`

What it does

A module-grouped report browser. Reports are organised into three collapsible sections (Finance, Material, Admin). Clicking a section expands it and shows a filter bar and a grid of report tiles. Clicking a tile fetches data from the relevant existing API endpoint and renders it in a paginated table. Any report can be exported to CSV.

There is no dedicated backend route for Reports. Each report calls the same API endpoint used by its source module, with additional query parameters applied from the filter bar.

Filter Bar

Filters are per-section and shared across all tiles within that open section.

Filter	Input Type	Backend param
Company	Select (from <code>/api/reports/companies</code>)	Varies per report – see FilterConfig below
Financial Year	Select (from <code>/api/fin-year</code>)	Varies per report
Date (single day)	Date picker	Varies per report

Filter	Input Type	Backend param
Date (range)	Two date pickers (From → To)	Varies per report

Filter priority: Financial Year takes precedence over calendar dates. When a Financial Year is selected, the date pickers are disabled. Clearing the Financial Year re-enables them.

FilterConfig — each report defines exactly which backend param names it accepts for each filter slot, and can set any to `null` to disable it entirely. This is necessary because different backend routes use different parameter names (`dateFrom` / `dateTo` vs `fromDate` / `toDate` vs `date` vs `finYear`).

Report Sections

Finance

7 reports. Accent colour: indigo (`#6366f1`).

Report	API Endpoint	Company	Fin Year	Date	Descri
Payment Register	<code>GET /api/new-payment</code>	—	<code>?finYear=</code>	<code>?date=</code> (single only)	All out payme mode, bank,
Received Payment	<code>GET /api/received-payment</code>	—	—	—	All inv receipt filter this e
EMI Register	<code>GET /api/received-payment?isEmi=1</code>	—	—	—	Receiv payme have a sched EMI to month
Pending Payments	<code>GET /api/new-payment?status=Pending</code>	—	<code>?finYear=</code>	<code>?date=</code> (single only)	Payme Pendi only
Bank Report	<code>GET /api/brs</code>	—	—	<code>?fromDate=</code> / <code>?toDate=</code>	BRS er cleare status

Report	API Endpoint	Company	Fin Year	Date	Descri
BRS Report	GET /api/brs	—	—	? fromDate= / ? toDate=	Full B source payme column
Ledger Report	GET /api/general- ledger	—	—	—	Genera accou no filt

Columns rendered:

Report	Columns
Payment Register	Doc No, Date, Party, Mode, Amount (₹), Bank, Status
Received Payment	Doc No, Date, Received From, Project, Amount (₹), Mode, Status
EMI Register	Doc No, Date, Received From, EMI Total (₹), EMI Months, Mode, Status
Pending Payments	Doc No, Date, Party, Amount (₹), Mode, Status
Bank Report	Date, Bank, Party, Amount (₹), Mode, Doc No, Cleared (Yes/No)
BRS Report	Doc No, Date, Bank, Party, Amount (₹), Type, Status, Cleared (Yes/No)
Ledger Report	Account, Code, Type, Group

Material

7 reports. Accent colour: emerald (#10b981).

Report	API Endpoint	Company	Fin Year	Date	Des
PO Register	GET /api/purchase-orders	—	—	—	All purc orde with ven and
GRN Register	GET /api/grns	? companyId=	—	—	Goo rece note

Report	API Endpoint	Company	Fin Year	Date	Description
					only reports this section that supplies companies filtered
Issue Register	GET /api/material-issues	—	—	—	Material issue site
Stock Summary	GET /api/inventory-master	—	—	? date= (single)	Current stock level item god
Work Order Register	GET /api/work-orders	—	—	—	All work orders with completion and completion value
BOQ Register	GET /api/boq	—	—	—	Bill of Materials with project and completion
Work Done	GET /api/engineering/work-done	—	—	—	Certification work entry with reference

Columns rendered:

Report	Columns
PO Register	PO No, Date, Vendor, Project, Amount (₹), Status
GRN Register	GRN No, Date, Supplier, Doc No, Status, Remarks
Issue Register	Issue No, Doc No, Project, Items, Total Qty, Status
Stock Summary	Item, Group, Location (+[MAIN] flag), Opening, In, Out, Closing, UOM

Report	Columns
Work Order Register	WO No, Date, Contractor, Project, Value (₹), Status
B0Q Register	B0Q No, Project, Company, Total Value (₹), Status
Work Done	Doc No, Date, WO No, Project, Certified (₹), Status

Stock Summary — Godown Switcher: This report adds a godown selector in the table header (populated from `GET /api/godowns`). It defaults to the godown where `IsMain = 1`. Changing the selection re-fetches with `?godownId=<id>`.

Admin

4 reports. Accent colour: amber (`#f59e0b`).

Report	API Endpoint	Company	Fin Year	Date	Des
Invoice Register	<code>GET /api/expense-booking</code>	—	<code>?finYear=</code>	—	Exp boc wit am
Supplier Report	<code>GET /api/enterprises?business_type=S</code>	—	—	—	Sup dire wit anc cor det
Pending Requests	<code>GET /api/approval-inbox</code>	—	—	—	Iter the app inb acr mo
User Activity	<code>GET /api/user-activity?limit=200</code>	—	—	<code>?dateFrom= / ?dateTo=</code>	Auc — l acc anc act eve

Columns rendered:

Report	Columns
Invoice Register	Invoice No, Date, Project, Amount (₹), Net Amount (₹), Status
Supplier Report	Supplier Name, GST No, PAN, Contact, Phone, City
Pending Requests	Reference, Module, Date, Status, Amount (₹)
User Activity	User, Email, Role, Event, Action, Resource, IP, Timestamp

Data Loading

Each report fetches up to 500 rows (`limit=500` appended automatically). Responses are normalised by `ReportTable` — the component handles both plain arrays and `{ data: [...] }` wrappers. The `filterConfig.dataKey` field overrides which response key is read (used by Stock Summary which uses `{ data: [...] }`).

Client-side pagination is applied at **20 rows per page**. All rows are fetched at once; pagination is purely a display concern.

Export

Every report has an **Export CSV** button (disabled while loading or when no rows exist). This calls `exportToCsv(rows, columns, reportId)` from `src/lib/export.ts` and downloads the file immediately. All INR values are exported with the ₹ prefix as rendered.

Widgets

File: `src/pages/Widgets.tsx` **Path:** `/widgets`

What it does

A personal interactive dashboard panel. Each widget is a self-contained mini-application. Which widgets a user can see is controlled by the admin via **Widgets Rights** (`/admin/rights/widgets`) — stored in

`dbo.UserWidgetRights`. Users with no configured rights see an empty state with a prompt to contact their administrator.

Data Sources

All data-backed widgets share a single backend fetch via `GET /api/widgets` — no widget makes its own API call. The response (`WidgetsDashboardData`) is cached in TanStack Query for 60 seconds and re-fetched every 5 minutes.

`WidgetsDashboardData` shape:

```
{
  generatedAt: string;
  summary: {
    totalUsers, activeUsers, totalActivity, activity24h,
    totalPayments, paymentAmountTotal, paymentAmountToday,
    pendingCheques, supplierCount, customerCount, ledgerCount,
    totalPOs, openPOs, openPOValue, grnsThisMonth, grnsToday,
    totalWorkOrders, totalTasks, openTasks, inProgressTasks,
    closedTasks, reviewedTasks, overdueTasks, dueSoonTasks
  };
  trends: {
    dailyFlow: { date, label, activityCount, tasksCreated }[] //
    last 7 days
  };
  recent: {
    users: WidgetUserRecord[];
    activity: WidgetActivityRecord[]; // last N rows from
    user activity log
    tasks: WidgetTaskRecord[]; // last N tasks
    payments: WidgetPaymentRecord[];
    purchaseOrders: WidgetPurchaseOrderRecord[];
    grns: WidgetGrnRecord[];
  };
  alerts: WidgetAlert[]; // system-generated: critical /
  warning / info
  insights: WidgetInsight[];
  permissions: { canViewUsers, canViewActivity, canViewFinance,
  ... };
  quickActions: WidgetQuickAction[];
  modules: WidgetModule[];
```

```
meta: { cacheTTL, role, scope, audienceLabel };
}
```

Widget Catalog

The catalog of available widgets is fetched from `GET /api/widgets/catalog` (backed by `dbo.WidgetCatalog`). Each row defines:

```
interface WidgetCatalogItem {
  key: string;           // stable identifier used in
UserWidgetRights
  label: string;         // display name
  iconKey: string;       // maps to an icon in widgetIcons
registry
  category: string;
  description: string;
  sortOrder: number;
  isActive: boolean;
}
```

Only catalog items where `isActive = true` **and** the user has an entry in `dbo.UserWidgetRights` are shown. The page fetches the user's allowed set from `GET /api/user-widget-rights/my` and intersects it with the catalog.

The 12 Widgets

Widget	Key	Icon	Data source
Bar Chart	Bar Chart	bar-chart-2	summary.trends.dailyFlow
Line Chart	Line Chart	trending-up	summary.trends.dailyFlow
Pie Chart	Pie Chart	pie-chart	summary

Widget	Key	Icon	Data source
Stat Card	Stat Card	hash	summary
Data Table	Data Table	table-2	recent.tasks
Calendar	Calendar	calendar	Local (no API)
Notifications	Notifications	bell	alerts
Activity Feed	Activity Feed	message-square	recent.activity
Map View	Map View	map	None
File Uploader	File Uploader	paperclip	Local (no API)
Progress Ring	Progress Ring	refresh-cw	summary

Widget	Key	Icon	Data source
Calculator	Calculator	calculator	Local (no API)

Access Control

Who can see which widgets is managed by admins at `/admin/rights/widgets` (the `WidgetsRights` page). This page:

- Lists all users (non-admin)
- Loads their current allowed widget keys from `GET /api/user-widget-rights/:userId`
- Shows all catalog widgets as toggles
- Saves via `POST /api/user-widget-rights/:userId` to `dbo.UserWidgetRights`

On the user-facing Widgets page, the allowed set is fetched from `GET /api/user-widget-rights/my`. If this endpoint errors (e.g. no row configured), `allowedSet` defaults to an empty `Set` and the user sees the empty state. There is no fallback to showing all widgets.

The widget catalog (`dbo.WidgetCatalog`) is managed by admins at `/admin/widget-catalog` (`WidgetCatalogAdmin.tsx`). An admin can set `isActive = false` on any catalog item to remove it from the grant UI and the user-facing page simultaneously, regardless of existing user rights for that key.

Common Pitfalls

Reports show no data for a filter that looks valid. Each report's `FilterConfig` defines which filter params it actually accepts. A report with `companyParam: null` will not pass `companyId` to the backend even if the

company filter is set. Refer to the report table above to check which filters each report supports.

BRS Report and Bank Report hit the same endpoint. Both call `GET /api/brs`. They differ only in which columns they display — BRS Report adds `SourceType` and `PayStatus` columns. The data returned is identical.

Stock Summary defaults to the main godown. On first load, the godown switcher auto-selects the godown where `IsMain = 1`. If no main godown is set, `godownId` remains empty and the endpoint will return results for all godowns or its own default.

EMI Register shows the same rows as Received Payment. The EMI Register uses `?isEmi=1` as a query param to `/api/received-payment`. If the backend does not filter on this param, both reports will return identical data. Confirm the received-payment route implements `isEmi` filtering.

User Activity report fetches at most 200 rows (`limit=200` is hardcoded in `defaultParams`). For high-volume environments this may not show the complete date range. There is no pagination control beyond the client-side 20-per-page display.

Widget shows empty state instead of data. The most common cause is that the user has no row in `dbo.UserWidgetRights` for that key. Admins should go to Widgets Rights and explicitly enable each widget for the user. The page does not fall back to showing all widgets.

Map View widget never shows live data. It is a placeholder with no data binding. A map API integration (Google Maps, Mapbox, etc.) would need to be wired in before it becomes functional.

File Uploader widget does not persist files. Files dropped or selected are stored in React local state only. They are not uploaded to the server and are lost on navigation away from the widget.

Authentication & Session Management

Type: Reference + Explanation

Authentication in CivilierERP uses JSON Web Tokens (JWT) issued at login and revoked at logout via a Redis blacklist. Every API route except `/health`, `/health/live`, and `POST /api/users/login` requires a valid Bearer token. The system is **fail-closed on Redis failures** — if the blacklist cannot be consulted, the request is rejected with `503` rather than allowed through.

Overview

Concern	Mechanism
Credential verification	bcrypt (12 salt rounds) against <code>dbo.users.password</code>
Token format	JWT, signed with <code>JWT_SECRET</code> , 7-day TTL
Token storage (client)	<code>localStorage</code> (XSS-mitigated by short TTL + blacklist on logout)
Token revocation	Redis key <code>blacklist:<token></code> with TTL = remaining token lifetime
Brute-force protection	Redis lockout after 5 failed attempts (15-minute window)
Role normalisation	<code>normalizeRole()</code> maps DB role strings to canonical slugs at login time

Environment Variables

These must be set in `backend/.env` (or the production env file at `/etc/civilier/prod.env`) before the server starts:

Variable	Required	Description
<code>JWT_SECRET</code>	Yes	Secret used to sign and verify JWTs. Generate with <code>openssl rand -hex 64</code> .
<code>JWT_EXPIRES_IN</code>	No	Token lifetime. Defaults to <code>7d</code> .
<code>REDIS_HOST</code>	Yes	Redis hostname. Use <code>redis</code> inside Docker Compose.
<code>REDIS_PORT</code>	Yes	Redis port. Typically <code>6379</code> .
<code>REDIS_PASSWORD</code>	Yes	Redis auth password.
<code>REDIS_DB</code>	No	Redis database index. Defaults to <code>0</code> .

Important: The auth middleware uses `redisGetStrict` — unlike the regular `redisGet` helper that swallows errors, `redisGetStrict` throws on Redis failure. This is intentional. A Redis outage causes `503` responses on all authenticated routes until Redis recovers.

Database Table: `dbo.users`

Column	Type	Notes
id	INT IDENTITY	Primary key
name	NVARCHAR	Display name
email	NVARCHAR	Login identifier — normalised to lowercase on every read/write
password	NVARCHAR	bcrypt hash (12 rounds)
RoleId	INT	FK → <code>dbo.Role.RId</code>
discontinue	BIT	1 = account disabled; login returns 403
can_accept_tickets	BIT	1 = user can accept tickets regardless of role
created_datetime	DATETIME	Creation timestamp

API Endpoints

POST /api/users/login

Authenticates a user and returns a signed JWT.

Auth required: No

Request body:

```
{  
  "email": "user@example.com",  
  "password": "plaintext_password"  
}
```

Success response — 200 OK:

```
{  
  "success": true,  
  "token": "<jwt>",  
  "user": {  
    "id": 42,  
    "name": "Prithwijit Ghosh",  
    "email": "prith@example.com",  
    "role": "admin",  
    "roleId": 3,  
  }  
}
```

```

    "can_accept_tickets": false,
    "pagePermissions": [
      { "page": "grn-master", "actions": ["view", "create",
"edit"] }
    ]
  }
}

```

`pagePermissions` is loaded from `dbo.UserPageRightsJson` at login and included in the response so the frontend does not need a second round-trip. For privileged roles (`super_admin`, `admin`, `dba`), the field is always `null` — those roles bypass the permission check entirely.

Error responses:

Status	Condition
400	<code>email</code> or <code>password</code> missing from body
401	User not found, or password does not match
403	<code>user.discontinue = 1</code> (account disabled)
429	Account locked after 5 failed attempts — try again in 15 minutes
500	Internal server error

Brute-force logout behaviour:

On each failed login attempt (wrong password or unknown email), the backend increments `login:attempts:<email>` in Redis. After 5 failures within any 15-minute window, it sets `login:lock:<email>` and all subsequent login attempts return 429 until the lock TTL expires. On a successful login, both keys are deleted immediately.

In development (`NODE_ENV=development`), the threshold is raised to 50 attempts and local IPs (`127.0.0.1`, `::1`) skip the lockout check entirely.

```

Attempt 1-4  → 401 (credentials wrong, counter incremented)
Attempt 5    → 401 (credentials wrong, lock set)
Attempt 6+   → 429 "Too many attempts. Try again later."
Successful   → 200 (both Redis keys deleted)

```

POST /api/users/logout

Invalidates the caller's JWT by writing it to the Redis blacklist.

Auth required: Yes (Bearer token)

Request body: None

Success response — 200 OK :

```
{ "success": true }
```

Behaviour:

The endpoint decodes the token's `exp` claim and writes `blacklist:<token> = "1"` to Redis with `EX = remaining_seconds`. This uses `getRedis()` directly (not the safe wrapper) so a Redis failure surfaces as a 500 error rather than silently succeeding. A logout that fails to write the blacklist entry must never return success.

Frontend integration (from `AuthContext.tsx`):

```
// logout() in AuthContext
localStorage.removeItem("token");
localStorage.removeItem("user");
disconnectSocket();
setCurrentUser(null);
```

The backend call to `POST /api/users/logout` is handled before the local storage is cleared so the token is still present in the `Authorization` header at the moment of blacklisting. After logout, the socket connection is torn down and all local auth state is cleared.

GET /api/users

Returns all users with normalised role slugs. Available to `super_admin`, `admin`, and `dba` only.

Auth required: Yes — `allowRoles("super_admin", "admin", "dba")`

Response:

```
[
  {
    "id": 42,
```

```
    "name": "Prithwijit Ghosh",
    "email": "prith@example.com",
    "roleId": 3,
    "roleName": "Admin",
    "role": "admin",
    "can_accept_tickets": false,
    "discontinue": 0,
    "created_datetime": "2026-03-01T00:00:00.000Z"
  }
]
```

The `role` field is the normalised canonical slug (e.g. `"admin"`). The `roleName` field is the raw string from `dbo.Role.RName` (e.g. `"Admin"` or `"System Administrator"`).

POST /api/users

Creates a new user account. Requires `CanAdd` on the `Users / List` permission.

Auth required: Yes — `checkPermission("Users", "List", "CanAdd")`

Request body:

Field	Required	Notes
<code>name</code>	Yes	Display name
<code>email</code>	Yes	Must be unique — returns <code>409</code> if already exists
<code>password</code>	Yes	Stored as bcrypt hash (12 rounds)
<code>RoleId</code>	Yes	FK → <code>dbo.Role.RId</code> (also accepted as <code>roleId</code>)
<code>can_accept_tickets</code>	No	Boolean — defaults to <code>false</code>

Error responses:

Status	Condition
<code>400</code>	<code>password</code> missing
<code>409</code>	Email already in use
<code>500</code>	DB error

PUT /api/users/:id

Updates an existing user. Requires `CanEdit` on `Users / List`.

Toggle-only mode: If only `discontinue` is sent in the body (all other fields omitted), the endpoint issues a minimal `UPDATE ... SET discontinue = @val` query. This prevents accidentally NULLing name, email, or role when the caller only wants to activate or deactivate the account.

DELETE /api/users/:id

Hard-deletes a user. Requires `CanDelete` on `Users / List`.

Self-delete guard: Returns `400` if `id` matches `req.user.userId`.

Before deleting the row, the endpoint reassigns or clears all FK references to the user across 20+ tables:

- `NOT NULL` FK columns (e.g. `Tasks.AssignedTo`, `TaskComments.UserId`) are **reassigned to the deleting admin's ID**.
- Nullable audit columns (e.g. `UpdatedBy`, `ApprovedBy` across most master tables) are **set to NULL**.
- Ownership rows in `dbo.UserPageRightsJson` and `dbo.UserWidgetRights` are **deleted entirely** (their `UserId` columns are `NOT NULL` and cannot be reassigned).

This ensures the `DELETE FROM dbo.users` always succeeds without FK constraint errors.

PATCH /api/users/:id/reset-password

Admin-initiated password reset. The user does not need to provide their current password.

Auth required: Yes — `checkPermission("Users", "List", "CanEdit")`

Request body:

```
{ "new_password": "new_secret_123" }
```

Minimum length: 6 characters. The new password is hashed with bcrypt (12 rounds) before storage. Existing sessions using the old password are **not invalidated** — the user's current JWT remains valid until it expires or they log out. If you need to force re-authentication after a reset, call the logout endpoint for that user's active token separately.

PATCH /api/users/:id/ticket-access

Toggles the `can_accept_tickets` flag for a user. Requires `CanEdit` on `Users / List`.

Request body:

```
{ "can_accept_tickets": true }
```

The flag grants the user the ability to accept tickets even if their role is not `admin`, `super_admin`, or `dba`. See the Ticket module documentation for the full `canAcceptTickets` middleware behaviour.

JWT lag: The `can_accept_tickets` flag is carried in the JWT payload at login time. Toggling it after login does not update the existing JWT — the `canAcceptTickets` middleware falls back to a live DB query for non-privileged roles, so the actual permission is always current regardless of what the token says.

PATCH /api/users/:id/permissions

Saves a user's `pagePermissions` array to `dbo.UserPageRightsJson`. Uses a SQL `MERGE` (upsert) so it creates the row on first call and updates it on subsequent calls.

After saving, the endpoint calls `userPermissionCache.invalidateUser(id)` to evict the in-process permission cache for that user immediately.

Auth required: Yes — `checkPermission("Users", "List", "CanEdit")`

Request body:

```
{
  "pagePermissions": [
    { "page": "grn-master", "actions": ["view", "create", "edit"]
  }
]
```

```
},  
  { "page": "purchase-orders", "actions": ["view"] }  
]  
}
```

JWT Middleware (`backend/middleware/auth.js`)

Every protected route applies this middleware. It runs three checks in sequence:

1. Extract Bearer token from Authorization header
↓ missing → 401
2. Check Redis blacklist (via in-process cache, then Redis)
↓ Redis down → 503 (fail-closed)
↓ blacklisted → 401 "Token has been invalidated"
3. Verify JWT signature and expiry
↓ invalid/expired → 401 "Invalid token"
4. Attach decoded payload to `req.user`, raw token to `req.token`
5. Fire-and-forget: `pfaddActiveUser(userId)` for metrics
↓ `next()`

In-process blacklist cache: To avoid a Redis round-trip on every single request, the middleware keeps a `localBlacklist` Map with a 10-second TTL per entry. A token that was blacklisted at logout becomes visible to new requests within 10 seconds at most. The map is lazily evicted when it grows beyond 5,000 entries.

Timing instrumentation: If `req.timing` is present (set by the `addRequestTiming` middleware), the auth middleware records `auth.redis_blacklist` and `auth.jwt_verify` stage durations. If total auth time exceeds 1 second, a `WARN` log with `AUTH_SLOW` event is emitted.

Frontend: AuthContext

File: `src/contexts/AuthContext.tsx`

`AuthContext` wraps the entire application and exposes the following to all components via `useAuth()`:

Property / Method	Type	Description
<code>currentUser</code>	<code>AppUser</code> <code> null</code>	The currently logged-in user, hydrated from <code>localStorage</code> on mount
<code>allUsers</code>	<code>AppUser[]</code>	Full user list — only populated for privileged roles
<code>login(email, password)</code>	<code>Promise</code>	Calls <code>POST /api/users/</code> , stores token + user, connects socket
<code>logout()</code>	<code>void</code>	Clears storage, disconnects socket, calls <code>recordLog</code> callback
<code>canAccessPage(page)</code>	<code>boolean</code>	Checks <code>currentUser.pagePermissions</code> for the given page key
<code>canDoAction(page, action)</code>	<code>boolean</code>	Checks <code>currentUser.pagePermissions</code> for a specific action
<code>updateUserPagePermissions(id, perms)</code>	<code>Promise</code>	Calls <code>PATCH /api/users/:id/permissions</code>

AppUser type:

```
interface AppUser {
  id: string;           // stringified DB integer
  name: string;
  email: string;
  role: UserRole;      // canonical slug — "admin" |
                        // "super_admin" | "dba" | "engineer" | "user" | "customer"
  initials: string;    // auto-derived from name
  avatarUrl?: string | null;
  can_accept_tickets?: boolean;
  pagePermissions: PagePermission[];
  isActive: boolean;   // derived from !user.discontinue
}
```

Session hydration on page reload:

On mount, `AuthContext` reads `localStorage["user"]` and `localStorage["token"]`. If either is missing, `currentUser` is `null` and the user sees the login page. Stale sessions from before the

`pagePermissions` field existed are patched with role-based defaults via `getPermissionsByRole()`.

Permission resolution for non-privileged users:

After a successful login, `AuthContext` makes a second request to `GET /api/user-rights/my` to fetch the latest `pagePermissions` from the DB. If that response contains a non-empty array, it overwrites the permissions returned in the login response. This ensures permissions updated by an admin since the last login are applied immediately without requiring a re-login.

Privileged roles (`super_admin`, `admin`, `dba`) skip this second fetch — they always receive `FULL_ACCESS` permissions regardless of what is stored in `dbo.UserPageRightsJson`.

Avatar hydration:

After login or page reload, `AuthContext` calls `GET /api/user-profile/:id/profile` to fetch the current user's `avatar_url`. This is fire-and-forget — a failure is silently ignored because the avatar is cosmetic.

Role Normalisation

The DB stores role names as human-readable strings (e.g. `"Super Admin"`, `"System Administrator"`). The `normalizeRole()` function in `backend/middleware/role.js` maps these to canonical slugs before they are written into the JWT and returned to the frontend.

Canonical Slug	DB strings it matches
<code>super_admin</code>	<code>"Super Admin"</code> , <code>"superadmin"</code> , <code>"super administrator"</code> , <code>"sa"</code>
<code>admin</code>	<code>"Admin"</code> , <code>"administrator"</code> , <code>"system admin"</code> , <code>"system administrator"</code>
<code>dba</code>	<code>"DBA"</code> , <code>"db admin"</code> , <code>"database admin"</code> , <code>"db_admin"</code>
<code>engineer</code>	<code>"engineer"</code> , <code>"site engineer"</code> , <code>"field engineer"</code>
<code>customer</code>	<code>"customer"</code> , <code>"customer demo"</code> , <code>"client"</code> , <code>"end user"</code> , <code>"portal user"</code>
<code>user</code>	<code>"user"</code> , <code>"employee"</code> , <code>"staff"</code> , <code>"standard user"</code>

Unknown strings fall through as lowercased (e.g. `"Project Lead"` → `"project lead"`). They will not match any `allowRoles()` call unless explicitly added to the alias table.

Token Storage Security Note

JWTs are stored in `localStorage`, which is accessible to JavaScript running on the page (XSS risk). This is an accepted tradeoff for a single-page application. The risk is mitigated by:

- Short token TTL (7 days, configurable via `JWT_EXPIRES_IN`)
- Immediate Redis blacklisting on logout
- Brute-force lockout on the login endpoint
- `helmet()` middleware setting security headers including `Content-Security-Policy`

If the application moves to server-side rendering or adds `httpOnly` cookie support, the token storage strategy should be revisited.

Common Pitfalls

Login returns 503 instead of 401. Redis is down or unreachable. The auth middleware is fail-closed — it rejects all authenticated requests until Redis recovers. Check `REDIS_HOST`, `REDIS_PORT`, and `REDIS_PASSWORD` in the env file and verify the Redis container is running.

User gets 429 on first login attempt. A previous failed-login run may have left a `login:lock:<email>` key in Redis. Flush it with `redis-cli DEL login:lock:<email>` or wait 15 minutes for the TTL to expire.

Token remains valid after password reset. `PATCH /api/users/:id/reset-password` does not invalidate existing sessions. The user's current JWT is still accepted until it expires. To force re-authentication, call `POST /api/users/logout` with the user's active token, or wait for the 7-day TTL.

`normalizeRole()` returns an unexpected slug. If a user with a non-standard role string (e.g. `"Project Lead"`) logs in, `normalizeRole()` returns `"project lead"`. This slug will not match any `allowRoles()` guard, so the user will receive 403 on all protected routes. Add the string to

`ROLE_ALIASES` in `backend/middleware/role.js` or rename the role in `dbo.Role`.

`pagePermissions` is `null` after login for a privileged user. This is expected. `super_admin`, `admin`, and `dba` users bypass `checkPermission()` entirely. Their `pagePermissions` field in the login response is always `null`. The frontend substitutes `FULL_ACCESS` via `getPermissionsByRole()`.

`can_accept_tickets` change not reflected immediately. The flag is carried in the JWT payload issued at login. After toggling it via `PATCH /api/users/:id/ticket-access`, the change takes effect immediately at the backend (the `canAcceptTickets` middleware does a live DB lookup for non-privileged roles) but the user's JWT still carries the old value. The user must re-login to get a JWT with the updated flag.

Redis Infrastructure

Type: Reference + Explanation

Redis is a required runtime dependency for CivilierERP. It serves four distinct purposes — response caching, JWT token blacklisting, brute-force login protection, and distributed rate limiting. The backend is designed so that a Redis outage **never crashes the application**, but the security posture varies by purpose: cache failures are fail-open (the app falls through to SQL Server), while blacklist failures are fail-closed (authenticated requests are rejected until Redis recovers).

Overview

Purpose	Fail behaviour	Key prefix
Response caching	Fail-open — falls through to SQL Server	<code>cache:</code>
JWT blacklisting	Fail-closed — returns <code>503</code>	<code>blacklist:</code>
Brute-force lockout	Fail-open — lockout check skipped	<code>login:</code>
Distributed rate limiting	Falls back to in-process memory counters	<code>rl:</code>

Environment Variables

Set these in `backend/.env` (development) or `/etc/civilier/prod.env` (production):

Variable	Required	Default	Description
<code>REDIS_HOST</code>	Yes	<code>127.0.0.1</code>	Redis server hostname. Use <code>redis</code> inside Docker Compose.
<code>REDIS_PORT</code>	Yes	<code>6379</code>	Redis server port.
<code>REDIS_PASSWORD</code>	Yes (prod)	—	Redis auth password. Leave blank only for unauthenticated local instances.
<code>REDIS_DB</code>	No	<code>0</code>	Redis logical database index.

Client: `backend/redis.js`

The Redis client is a singleton backed by `ioredis` with `lazyConnect: true`. The connection is not opened until the first operation, which prevents `rate-limit-redis` from firing Lua scripts against an uninitialised stream at startup.

Connection options

```
{
  lazyConnect: true,
  enableOfflineQueue: false,
  connectTimeout: 3000, // ms — TCP handshake timeout
  commandTimeout: 2000, // ms — per-command timeout
  retryStrategy: (times) => times > 5 ? null : Math.min(times * 500, 3000)
}
```

After 5 failed reconnection attempts, the client stops retrying. Subsequent calls to `getRedis()` will throw, which `safeExec` swallows — the app continues serving from SQL Server.

`safeExec` — the fault-isolation wrapper

Every public helper in `redis.js` passes through `safeExec`:

```

async function safeExec(fn, fallback = null) {
  try {
    const redis = await getRedis();
    return await fn(redis);
  } catch {
    return fallback;
  }
}

```

If Redis is down or the command times out, `safeExec` returns the `fallback` value (default `null`) without throwing. This is what makes Redis outages transparent to route handlers.

The one deliberate exception is `redisGetStrict` — it calls `getRedis()` and `redis.get()` directly, without `safeExec`. Use this only where a Redis failure must be treated as a hard error (for example, the token blacklist check).

Exported helpers

Function	Failure behaviour	Description
<code>getRedis()</code>	Throws	Returns the connected client. Used internally.
<code>isRedisReady()</code>	Returns <code>false</code>	Sends <code>PING</code> and checks for <code>PONG</code> . Used by <code>GET /health/ready</code> .
<code>redisGet(key)</code>	Returns <code>null</code>	Safe string GET.
<code>redisGetStrict(key)</code>	Throws	Security-critical GET — used for blacklist checks.
<code>redisSet(key, value, ttl?)</code>	Returns <code>null</code>	Safe SET with optional <code>EX</code> TTL in seconds.
<code>redisDel(key)</code>	Returns <code>null</code>	Safe DEL.
<code>redisDelPattern(pattern)</code>	Returns <code>0</code>	DEL all keys matching pattern via a Lua script (avoids <code>KEYS</code> in production clusters).
<code>redisLock(key, ttlSeconds)</code>	Returns <code>null</code>	<code>SET NX PX</code> distributed lock. Returns <code>"OK"</code> on

Function	Failure behaviour	Description
		acquisition, <code>null</code> if already held.
<code>bumpCacheVersion(ns)</code>	Silent no-op	Increments <code>cache:version:<ns></code> and evicts the local version cache.
<code>getCacheVersion(ns)</code>	Returns <code>0</code>	Returns current version counter for a namespace.
<code>compress(value)</code>	Returns <code>null</code>	Gzip-compresses a value and base64-encodes it with a <code>gz:</code> prefix.
<code>decompress(str)</code>	Returns <code>null</code>	Reverses <code>compress</code> . Handles both <code>gz:</code> -prefixed and legacy plain JSON.
<code>pfaddActiveUser(userId)</code>	Silent no-op	Adds <code>userId</code> to today's HyperLogLog key for active user counting.
<code>incrGlobalRequests()</code>	Silent no-op	Increments the global request counter (60s TTL).
<code>getSystemMetrics()</code>	Returns last cached	Fetches RPM, active users, memory usage, cache hit rate in one pipeline. Cached for 2 seconds in-process.
<code>getDynamicLimit(score, rpm, memoryUsage)</code>	Pure function	Computes the per-user API rate limit from engagement score and system load.
<code>redisPipelineExec(commands)</code>	Returns <code>null</code>	Executes an array of <code>[command, ...args]</code> pairs in one pipeline round-trip.

Purpose 1 — Response Caching

Cache middleware: `backend/middleware/cache.js`

Apply the `cache()` middleware to any `GET` route to enable caching:

```
const { cache } = require("../middleware/cache");

// Per-user cache, 300-second TTL
router.get("/", cache("grns", 300), async (req, res) => { ...
});

// Shared cache (same entry for all users) – for master data that
is not user-scoped
router.get("/", cache("enterprises", 300, { shared: true }),
async (req, res) => { ... });
```

How a cache hit works

```
GET /api/grns
  ↓
cache() middleware
  ↓
1. Circuit breaker open? → BYPASS (skip all cache logic, go to
DB)
  ↓
2. Fetch version: localVersionCache.get("grns") → e.g. version =
4
  ↓
3. Build key: cache:grns:v4:<userId>:{ "path":"/", "params":
{}, "query": {} }
  ↓
4. Redis GET → HIT
  ↓
5. decompress (gz: prefix → gunzip → JSON.parse)
  ↓
6. res.setHeader("X-Cache", "HIT")
   res.json(data)    ← responds immediately, DB never queried
```

How a cache miss works

```
GET /api/grns (no entry in Redis)
  ↓
1-3. Same as above
  ↓
4. Redis GET → MISS
  ↓
5. Acquire distributed lock: SET cachelock:<key> NX PX 30000
```

- Lock acquired → proceed
- Lock not acquired (another process is refreshing):
 - Try stale key (cache:stale:<baseKey>)
 - If stale hit → X-Cache: STALE, return stale data
 - If no stale → X-Cache: MISS-FALLTHROUGH, go to DB

↓

- Intercept res.json → run SQL query → on response:
 - Compute dynamic TTL (60-300s based on RPM and memory)
 - Compress payload (gzip if > 1024 bytes)
 - Write to both versioned key (TTL) and stale key (TTL × 2)
 - Release lock
 - res.setHeader("X-Cache", "MISS")
 - res.setHeader("X-Cache-TTL", "<n>s")

Cache key structure

cache:<namespace>:v<version>:<scopeId>:<routeScope>

Segment	Value
namespace	The string passed to <code>cache()</code> , e.g. "grns"
version	Current value of <code>cache:version:<namespace></code> in Redis
scopeId	<code>req.user.userId</code> for per-user caches; "shared" when <code>{ shared: true }</code>
routeScope	JSON of <code>{ path, params, query }</code> – differentiates paginated or filtered requests

Two parallel keys exist for every entry:

- **Versioned key** — `cache:<ns>:v<n>:<scope>:<route>` — serves live cache hits
- **Stale key** — `cache:stale:<ns>:<scope>:<route>` — TTL = `2 × live TTL`, serves during stampede protection

Payload compression

Payloads larger than 1,024 bytes are compressed with Node's native `zlib.gzip` before storage. The compressed value is stored as `gz:<base64>`. On retrieval, `decompress()` detects the prefix and decompresses transparently. Legacy values stored without the prefix are still parsed as plain JSON.

The `REDIS.md` file mentions LZString compression — this was replaced with `zlib.gzip` (native, non-blocking) after LZString was found to take 130–570 ms on large payloads.

Dynamic TTL

The cache middleware does not always use the TTL passed to `cache()`. Every 10 seconds it recomputes a dynamic TTL based on current system load:

Predicted RPM	Memory usage	Effective TTL
> 10,000	any	120s
> 5,000	any	180s
≤ 5,000	any	300s
any	> 80%	<code>floor(computed TTL × 0.5)</code>
(minimum)	—	60s

For `5xx` responses, the TTL is capped at 30 seconds so errors are not cached for long.

Circuit breaker

The cache middleware has a three-state circuit breaker (`closed` → `open` → `half-open`) that prevents Redis timeouts from cascading into slow API responses. After 3 consecutive Redis failures the circuit opens for 10 seconds. While open, all cache logic is skipped instantly and requests go directly to SQL Server. After 10 seconds, the circuit moves to `half-open` and allows one probe request to test whether Redis has recovered.

Response headers

Header	Values	Meaning
<code>X-Cache</code>	<code>HIT</code>	Served from Redis
<code>X-Cache</code>	<code>MISS</code>	DB queried, result written to Redis
<code>X-Cache</code>	<code>STALE</code>	Served from stale key during stampede
<code>X-Cache</code>	<code>BYPASS</code>	Redis circuit open or lock failure
<code>X-Cache</code>	<code>MISS-FALLTHROUGH</code>	Lock held by another process, no stale data, fell through to DB
<code>X-Cache-TTL</code>	e.g. <code>300s</code>	The TTL written to Redis for this response

Cache invalidation: `bumpCacheVersion(namespace)`

When a route mutates data, it calls `bumpCacheVersion()` instead of deleting individual keys. This atomically increments `cache:version:<namespace>` in Redis. The next GET for that namespace fetches a new version number, making all previous keys (which embed the old version) unreachable — they expire naturally.

```
// Example from grns.js — after a GRN is created or updated:
await bumpCacheVersion("grns");
await bumpCacheVersion("stock-ledger");
await bumpCacheVersion("material-dashboard");
```

`bumpCacheVersion` also calls `invalidateLocalCacheVersion(ns)`, which evicts the in-process version copy held by `localVersionCache` in `cache.js`. Without this, the in-process cache would serve the old version number for up to 2 seconds after a bump.

Cached namespaces — full reference

The following namespaces are used across the codebase. All are invalidated on their corresponding write operations.

Namespace	Route	Default TTL	Notes
<code>account-group</code>	<code>/api/account-group</code>	300s	
<code>account-head-master</code>	<code>/api/account-head</code>	300s	
<code>activity-master</code>	<code>/api/activity-master</code>	300s	
<code>bank-master</code>	<code>/api/bank-master</code>	300s	
<code>billing-terms</code>	<code>/api/billing-terms</code>	300s	
<code>block-master</code>	<code>/api/block-master</code>	300s	
<code>block-master-projects</code>	<code>/api/block-master/projects</code>	600s	Not explicitly bumped — stable data
<code>boq</code>	<code>/api/boq</code>	300s	
<code>brs</code>	<code>/api/brs</code>	300s	Bumped on payment

Namespace	Route	Default TTL	Notes
			mutations
card-master	/api/card-master	300s	
cheque-master	/api/cheque-master	300s	
company-master	/api/company-master	300s	
document-type	/api/document-type	300s	
eng-approved-boqs	/api/engineering/approved-boqs	120s	Not explicitly bumped
eng-wo-with-activities	/api/engineering/work-orders-with-activities	300s	Not explicitly bumped – stale for up to 5 min after W0 approval
enterprises	/api/enterprises	300s	
entry-type	/api/entry-type	300s	
expense-booking	/api/expense-booking	300s	
expense-booking-options	/api/expense-booking (options sub-route)	300s	
expense-booking-source-ids	/api/expense-booking (source IDs)	300s	
fin-year	/api/fin-year	300s	
finance-dashboard	/api/finance-dashboard	60s	
general-ledger	/api/general-ledger	300s	
general-ledger-detail	/api/general-ledger/:id	300s	
godowns	/api/godowns	300s	Bumped on stock transfers
grns	/api/grns	300s	
hsn	/api/hsn	300s	

Namespace	Route	Default TTL	Notes
inventory-master	/api/inventory-master	300s	
item-groups	/api/item-groups	300s	
item-master	/api/item-master	300s	
material-issues	/api/material-issues	300s	
menu-master	/api/menu-master	300s	
new-payment	/api/new-payment	300s	
project-master	/api/project-master	300s	
purchase-orders	/api/purchase-orders	300s	
received-payment	/api/received-payment	300s	
roles	/api/roles	120s	
roles:list	/api/roles/list	120s	
stock-ledger	/api/stock-ledger	300s	
stock-transfers	/api/stock-transfers	300s	
tasks	/api/tasks	300s	
tc-master	/api/tc-master	300s	
tds-master	/api/tds-master	300s	
tenants	/api/tenants	300s	
unit-master	/api/unit-master	300s	
unit-master-projects	/api/unit-master/projects	600s	Not explicitly bumped — stable
uom-master	/api/uom-master	300s	
user-activity	/api/user-activity	300s	
wo-meta-act-groups	/api/work-orders/meta/activity-groups	600s	Not explicitly bumped
wo-meta-activities	/api/work-orders/meta/activities	600s	Not explicitly bumped
wo-meta-companies	/api/work-orders/meta/companies	600s	Not explicitly

Namespace	Route	Default TTL	Notes
			bumped
wo-meta-contractors	/api/work-orders/meta/contractors	600s	Not explicitly bumped
wo-meta-projects	/api/work-orders/meta/projects	600s	Not explicitly bumped
wo-meta-uoms	/api/work-orders/meta/uoms	600s	Not explicitly bumped
work-orders	/api/work-orders	300s	

Purpose 2 — JWT Blacklisting

When a user logs out, the backend writes `blacklist:<token> = "1"` to Redis with a TTL equal to the token's remaining lifetime. Every subsequent authenticated request checks this key.

The blacklist check is fail-closed. If Redis is unavailable, the auth middleware returns `503` rather than allowing a potentially revoked token through. This is intentional for a financial ERP — a brief service interruption is safer than a revoked session silently remaining active.

See the **Authentication & Session Management** documentation for the complete flow, including the in-process 10-second cache that avoids a Redis round-trip on every request.

Purpose 3 — Brute-Force Login Protection

Redis key	TTL	Purpose
login:attempts:<email>	900s (15 min)	Incremented on each failed login
login:lock:<email>	900s (15 min)	Set when attempts reach the threshold

The threshold is 5 attempts in production and 50 in development. Local IPs skip the check entirely in development.

This check is fail-open — if Redis is unavailable, the lock check is skipped and the login proceeds. Brute-force protection is temporarily inactive during a Redis outage, but logins continue to work.

On a successful login, both keys are deleted immediately.

Purpose 4 — Distributed Rate Limiting

Two rate limiters are mounted globally in `server.js` using `express-rate-limit` with a `RedisStore` backend:

Login limiter

Applied only to `POST /api/users/login`.

Setting	Value
Window	15 minutes
Max requests	10 per window
Redis key prefix	<code>rl:login:</code>
Key generator	Default (<code>req.ip</code>)
Skip	Local IPs in development

API limiter

Applied to all `/api/*` routes. Skips `/api/user-activity` (high-frequency activity logging).

Setting	Value
Window	60 seconds
Max requests	Dynamic — computed per user from engagement score
Redis key prefix	<code>rl:api:</code>
Key generator	<code>user:<userId></code> if authenticated; <code>req.ip</code> otherwise

The dynamic limit is computed by `getDynamicLimit(score, rpm, memoryUsage)`:

```
function getDynamicLimit(score, rpm, memoryUsage) {
  const base = 20 + Math.sqrt(score) * 10; // higher engagement
  = higher base limit
  let loadFactor = rpm > 10000 ? 0.5 : rpm > 5000 ? 0.7 : 1;
```

```
const memoryFactor = memoryUsage > 0.8 ? 0.7 : 1;
return Math.floor(Math.min(base * loadFactor * memoryFactor,
500));
}
```

The engagement score is a ZSET value in Redis (`engagement:score` , member = `userId`). It is incremented on user activity and decays over time via a Lua script run by the worker process. Users with higher engagement scores receive higher rate limits; new users start at the minimum base.

If Redis fails during limit computation, the API limiter falls back to 500 req/min (a safe permissive default).

Full Redis Key Schema

Key pattern	Purpose	TTL
<code>cache:version:<ns></code>	Cache version counter for namespace <code>ns</code>	No TTL — persists until bumped
<code>cache:<ns>:v<n>:<scopeId>:<routeScope></code>	Cached GET response (gzip compressed if > 1 KB)	Dynamic, 60–300s
<code>cache:stale:<ns>:<scopeId>:<routeScope></code>	Stale-while-revalidate copy	<code>2 × live TTL</code>
<code>cachelock:<cacheKey></code>	Stampede-protection lock	30s
<code>blacklist:<jwt></code>	Revoked JWT token	Remaining JWT lifetime
<code>login:attempts:<email></code>	Failed login counter	900s
<code>login:lock:<email></code>	Account lockout flag	900s
<code>rl:login:<ip></code>	Login rate limit counter	900s
<code>rl:api:user:<userId></code>	Per-user API rate limit counter	60s
<code>rl:api:<ip></code>	Per-IP API rate limit counter (unauthenticated)	60s
<code>engagement:score</code>	ZSET — user engagement scores (member = <code>userId</code>)	30 days
<code>engagement:last:<userId></code>	Timestamp of last activity for decay cleanup	30 days

Key pattern	Purpose	TTL
<code>active:users:<YYYYMMDD></code>	HyperLogLog — unique active users per day	86400s
<code>global:requests</code>	Rolling request counter for RPM	60s
<code>global:cache_hits</code>	Rolling cache hit counter	60s
<code>global:cache_misses</code>	Rolling cache miss counter	60s
<code>metrics:hour:<H>:total_load</code>	Hourly load accumulator (for predicted RPM)	7 days
<code>metrics:hour:<H>:count</code>	Hourly request count	7 days

Graceful Degradation Summary

Feature	Redis down behaviour
Response caching	Cache miss on every request → all queries hit SQL Server. Slower but correct.
JWT blacklist check	Returns 503 — fail-closed. All authenticated routes are blocked until Redis recovers.
Login brute-force lockout	Lock check skipped → logins proceed even for locked accounts.
Rate limiting	Falls back to in-process memory counters per process instance. Limits no longer shared across instances.
Active user tracking	<code>pfaddActiveUser</code> is fire-and-forget — silently fails. Dashboard active user count stops updating.
System metrics	<code>getSystemMetrics()</code> returns last cached value (updated at most every 2 seconds).
Dynamic rate limit	Falls back to 500 req/min per user.

The application **never crashes** due to a Redis failure. The one security exception is the JWT blacklist — that is an explicit architectural decision and is documented as such.

Adding Cache to a New Route

To cache a new `GET` route, follow this pattern:

```

const { cache } = require("../middleware/cache");
const { bumpCacheVersion } = require("../redis");

const CACHE_NS = "my-resource";

// Cache the GET
router.get("/", cache(CACHE_NS, 300), async (req, res) => {
  const data = await pool.request().query("SELECT ...");
  res.json(data.recordset);
});

// Bust the cache on any write
router.post("/", async (req, res) => {
  await pool.request().query("INSERT ...");
  await bumpCacheVersion(CACHE_NS);
  res.json({ success: true });
});

router.put("/:id", async (req, res) => {
  await pool.request().query("UPDATE ...");
  await bumpCacheVersion(CACHE_NS);
  res.json({ success: true });
});

router.delete("/:id", async (req, res) => {
  await pool.request().query("DELETE ...");
  await bumpCacheVersion(CACHE_NS);
  res.json({ success: true });
});

```

Use `{ shared: true }` for master data that is identical for all users (company names, project lists, HSN codes). Omit it for transactional data that is user-scoped.

Common Pitfalls

All authenticated routes return 503 after a Redis restart. Redis is not accepting connections. The JWT blacklist check uses `redisGetStrict` which throws on failure. Check that `REDIS_HOST`, `REDIS_PORT`, and `REDIS_PASSWORD` are correct. Verify the Redis container is running with `docker compose ps` and check its logs with `docker compose logs redis`.

Data is stale after a write even after waiting longer than the TTL. The route that performs the write is not calling `bumpCacheVersion()`. Check the POST/PUT/DELETE handler for the relevant route. The version counter must be bumped on every mutation — not just one of the three methods.

eng-wo-with-activities still shows old data after approving a Work Order. This namespace (`wo-meta-*` group) has a 300–600 second TTL and is not explicitly bumped on Work Order status changes. After approving a Work Order, the Work Done picker may not show it for up to 5 minutes. This is a known gap — a manual page refresh forces a cache miss.

The circuit breaker opens during a load spike and all caches stop working. The circuit breaker opens after 3 consecutive Redis failures. If Redis is running but slow (CPU-bound during a large `KEYS` scan, for example), commands may time out and trigger the breaker. Check Redis memory usage and avoid running `KEYS *` manually on a loaded instance. The breaker recovers automatically after 10 seconds.

Cache hit rate is 0% in the metrics dashboard even though Redis is running. The `global:cache_hits` and `global:cache_misses` counters have a 60-second TTL. If no requests have been made recently, the counters expire and hit rate shows 0. This is expected — the counters are rolling 60-second windows, not lifetime totals.

login:lock key not expiring — user permanently locked out. The `LOCKOUT_SECONDS` constant is 900 (15 minutes). If a lock was written with a shorter TTL due to a Redis misconfiguration or an earlier code version, it may not expire. Delete it manually: `redis-cli DEL login:lock:<email>`.

Health Endpoints

Type: Reference

The health endpoints let infrastructure tooling — load balancers, container orchestrators, and deployment scripts — verify that the CivilierERP API process is alive and its dependencies are ready to serve traffic. The four endpoints form a deliberate two-tier design: two public probes that reveal nothing sensitive, and two token-guarded probes that expose per-service dependency status for internal use only.

Environment Variables

Variable	Required	Description
HEALTH_TOKEN	Required for guarded endpoints	Static bearer token used by <code>requireHealthToken</code> . If unset, the guarded endpoints return <code>503</code> immediately (fail-safe).

Set this in your `.env` file:

```
HEALTH_TOKEN=your-secret-token-here
```

Endpoint Overview

Endpoint	Auth	HTTP Method	Purpose
GET <code>/health</code>	None	GET	Public ping — confirms the process is running
GET <code>/health/live</code>	None	GET	Liveness probe — reports process uptime
GET <code>/health/startup</code>	X-Health-Token header	GET	Startup timestamp — for internal orchestration
GET <code>/health/ready</code>	X-Health-Token header	GET	Readiness probe — checks DB, Redis, and Worker

All four endpoints are mounted **outside** the `/api` prefix. They bypass JWT authentication and the API rate limiter entirely. A separate rate limiter of **100 requests per 15 minutes** applies to the entire `/health` router to prevent abuse.

GET `/health`

Authentication: None

Purpose: Minimal public ping. Use this as a basic availability check in your load balancer (e.g., AWS ALB target group health check). The response contains no environment data or infrastructure details.

Example request:

```
curl https://your-api-domain.com/health
```

Example response (200 OK):

```
{
  "status": "ok",
  "name": "CivilierERP API"
}
```

GET /health/live

Authentication: None

Purpose: Liveness probe. Use this to tell your container orchestrator (Kubernetes, Render, ECS) that the Node.js process is alive and has not hung or crashed. It does not check database or Redis connectivity — only process uptime.

Example request:

```
curl https://your-api-domain.com/health/live
```

Example response (200 OK):

```
{
  "status": "alive",
  "uptimeSeconds": 3742,
  "requestId": "req-abc123"
}
```

Response fields:

Field	Type	Description
status	string	Always "alive" when the process responds
uptimeSeconds	number	Seconds since the Node.js process started, rounded to the nearest integer
requestId	string	Request correlation ID from the request logger

GET /health/startup

Authentication: `X-Health-Token` header (or `?token=` query param)

Purpose: Startup timestamp probe for internal orchestration. Use this to verify that a new deployment has fully initialised before routing traffic to it. Returns when `createApp()` completed and the server began accepting requests.

Example request:

```
curl https://your-api-domain.com/health/startup \
-H "X-Health-Token: your-secret-token-here"
```

Example response (`200 OK`):

```
{
  "status": "ok",
  "startedAt": "2025-05-14T09:32:11.204Z",
  "uptimeSeconds": 3742,
  "requestId": "req-abc123"
}
```

Response fields:

Field	Type	Description
<code>status</code>	<code>string</code>	Always <code>"ok"</code> on success
<code>startedAt</code>	<code>string</code> <code> null</code>	ISO 8601 timestamp of when <code>createApp()</code> set <code>app.locals.startupTime</code> . <code>null</code> if the value was not set.
<code>uptimeSeconds</code>	<code>number</code>	Seconds since process start
<code>requestId</code>	<code>string</code>	Request correlation ID

GET /health/ready

Authentication: `X-Health-Token` header (or `?token=` query param)

Purpose: Readiness probe. Checks whether all three critical dependencies — SQL Server, Redis, and the background worker — are operational. Use this in

Kubernetes readiness probes or deployment smoke tests to block traffic until the service is fully ready.

Returns **503** if **any** dependency reports a failure.

Example request:

```
curl https://your-api-domain.com/health/ready \
-H "X-Health-Token: your-secret-token-here"
```

Example response — all systems healthy (**200 OK**):

```
{
  "status": "ok",
  "details": {
    "db": "ok",
    "redis": "ok",
    "worker": "ok"
  },
  "requestId": "req-abc123"
}
```

Example response — Redis unavailable (**503 Service Unavailable**):

```
{
  "status": "fail",
  "details": {
    "db": "ok",
    "redis": "fail",
    "worker": "ok"
  },
  "requestId": "req-abc123"
}
```

Response fields:

Field	Type	Description
<code>status</code>	<code>"ok"</code> <code>"fail"</code>	<code>"ok"</code> only when all three dependencies are healthy
<code>details.db</code>	<code>"ok"</code> <code>"fail"</code>	Result of <code>isDbReady()</code> — SQL Server connection pool

Field	Type	Description
<code>details.redis</code>	<code>"ok"</code> <code>"fail"</code>	Result of <code>isRedisReady()</code> — Redis connection
<code>details.worker</code>	<code>"ok"</code> <code>"fail"</code>	Result of <code>worker.isRunning()</code> — background job worker
<code>requestId</code>	<code>string</code>	Request correlation ID

Note: Pool statistics (connection counts, pool sizes) are intentionally excluded from this response. They expose infrastructure capacity details. Use `GET /api/system/metrics` (requires authentication) to access that data.

Authentication: `requireHealthToken`

The `requireHealthToken` middleware protects `GET /health/startup` and `GET /health/ready`. It reads the expected token from `process.env.HEALTH_TOKEN` and compares it against the incoming request.

Token delivery: Pass the token via the `X-Health-Token` request header or the `token` query parameter.

```
# Header (preferred)
curl /health/ready -H "X-Health-Token: my-token"

# Query param (acceptable for quick tests, avoid in production)
curl "/health/ready?token=my-token"
```

Fail-safe behaviour: If `HEALTH_TOKEN` is not set in the environment, `requireHealthToken` immediately returns `503` with `{ "error": "Health check not configured" }`. It logs the event at `WARN` level. This is intentional — misconfigured infrastructure should never silently expose service internals.

Kubernetes / Render Configuration Example

```
# Kubernetes probe configuration example
livenessProbe:
  httpGet:
    path: /health/live
```

```
port: 5000
initialDelaySeconds: 10
periodSeconds: 15

readinessProbe:
  httpGet:
    path: /health/ready
    port: 5000
    httpHeaders:
      - name: X-Health-Token
        value: "${HEALTH_TOKEN}"
  initialDelaySeconds: 15
  periodSeconds: 10
  failureThreshold: 3
```

Common Pitfalls & Troubleshooting

`GET /health/ready` or `GET /health/startup` returns `503` with `"Health check not configured"`

The `HEALTH_TOKEN` environment variable is not set. Add it to your `.env` file and restart the server. Without this variable, the guarded endpoints will always return `503`, regardless of actual system health.

`GET /health/ready` returns `401 Unauthorized`

The `X-Health-Token` header value does not match `HEALTH_TOKEN`. Check for trailing whitespace or encoding issues in the secret.

`GET /health/ready` returns `"worker": "fail"`

The background worker failed to start or has stopped. Check the server logs for errors from `worker.js`. The worker status is checked by calling `worker.isRunning()` — it returns `false` if the worker process is not active.

`GET /health/ready` returns `"db": "fail"` after deployment

The SQL Server connection pool has not initialised yet, or the DB is unreachable. `isDbReady()` returns `false` if no pool is available. Check `MSSQL_*` connection environment variables and network access to the database host.

Best Practices

- Use `GET /health` for load balancer target group health checks. It is lightweight and public.
- Use `GET /health/live` for container liveness probes. It confirms the process has not crashed without making any DB or Redis calls.
- Use `GET /health/ready` for readiness probes and post-deployment smoke tests. Route traffic only when it returns `200`.
- Store `HEALTH_TOKEN` as a secret in your CI/CD environment (GitHub Actions secret, AWS Secrets Manager, etc.) – never commit it to source control.
- Do not use `GET /health/ready` as a load balancer health check. It makes async DB and Redis calls on every request and will add latency under high-frequency polling.

Deployment & Infrastructure

Type: How-to Guide

This guide covers everything needed to deploy CivilierERP to AWS from scratch and to push subsequent updates through the automated deployment pipeline. The production stack runs one EC2 Docker host behind an AWS Application Load Balancer, with SQL Server on RDS and Redis running as a Docker container on the same host.

Architecture Overview

```
Internet
  |
  | HTTPS 443
  ▼
AWS ALB (ap-south-1, ACM certificate)
  |
  | HTTP 80 (private subnet only)
  ▼
EC2 Instance (private subnet)
  |— Docker: nginx — serves built frontend + proxies /api
  | to backend
  |— Docker: backend — Node.js API on port 5000 (internal
  | only)
  |— Docker: redis — Redis 7 (internal network only)
  |
  | SQL 1433 (private subnet only)
```

▼
RDS SQL Server (private subnet)

Security group rules:

Resource	Inbound rule	Source
ALB	443 (HTTPS)	Internet (0.0.0.0/0)
EC2	80 (HTTP)	ALB security group only
RDS	1433 (SQL Server)	EC2 security group only

EC2 has no inbound SSH port open. Shell access uses **AWS Systems Manager (SSM)** instead.

Step 1 — Generate Production Secrets

Run these commands locally before touching AWS. Store the output somewhere secure (a password manager or AWS Secrets Manager).

```
openssl rand -hex 64 # → JWT_SECRET
openssl rand -hex 32 # → REDIS_PASSWORD
openssl rand -hex 32 # → HEALTH_TOKEN
```

Never commit these values to source control. Never put them in `backend/.env` on a production host.

Step 2 — First EC2 Setup

SSH into the instance via SSM, then run the setup script. Pass your repository URL as an environment variable:

```
REPO_URL=https://github.com/YOUR_ORG/CivilierERP.git bash
scripts/setup-ec2.sh
```

The script detects whether the OS uses `dnf` (Amazon Linux / RHEL) or `apt-get` (Ubuntu/Debian) and installs the following:

- `git`
- `docker`

- `docker-compose-plugin`

It also:

- Enables and starts the Docker daemon
- Adds the current user to the `docker` group
- Creates `/etc/civilier/` with mode `700`
- Clones the repository to `/opt/civilier` (default; override with `APP_DIR=`)

After the script completes, **log out and back in** so the `docker` group membership takes effect.

Step 3 — Create the Production Environment File

Create `/etc/civilier/prod.env` using `backend/.env.example` as the template. Fill in every value:

```
NODE_ENV=production
PORT=5000

# SQL Server (RDS)
DB_SERVER=your-rds-endpoint.ap-south-1.rds.amazonaws.com
DB_PORT=1433
DB_NAME=CivilierERP
DB_USER=your_db_user
DB_PASSWORD=your_db_password
DB_ENCRYPT=true
DB_TRUST_SERVER_CERTIFICATE=false

# Auth
JWT_SECRET=<generated_hex_64>
JWT_EXPIRES_IN=7d

# Redis (use service name "redis" – resolved by Docker's internal
DNS)
REDIS_HOST=redis
REDIS_PORT=6379
REDIS_PASSWORD=<generated_hex_32>
REDIS_DB=0

# CORS
```

```
ALLOWED_ORIGINS=https://civiliererp.in,https://www.civiliererp.in

# Health probe token
HEALTH_TOKEN=<generated_hex_32>
```

Lock down the file immediately after creating it:

```
sudo chown root:root /etc/civilier/prod.env
sudo chmod 600 /etc/civilier/prod.env
```

Step 4 — First Deploy

```
cd /opt/civilier
CIVILIER_ENV_FILE=/etc/civilier/prod.env bash scripts/deploy.sh
main
```

The deploy script (`scripts/deploy.sh`) does the following in order:

1. **Validates the env file.** Checks that all required keys (`DB_SERVER` , `DB_NAME` , `DB_USER` , `DB_PASSWORD` , `JWT_SECRET` , `REDIS_PASSWORD` , `HEALTH_TOKEN`) have non-empty values. Exits immediately if any are missing.
2. **Pulls the latest code** from the specified branch (`main` by default) using `git pull --ff-only`.
3. **Builds the backend** Docker image: `docker compose build backend`.
4. **Starts Redis** and waits for its health check to pass.
5. **Runs database migrations:** `docker compose run --rm backend npm run migrate`.
6. **Builds the frontend** static files using the `frontend-build` profile.
7. **Starts the full stack:** Redis, backend, and nginx.
8. **Polls the health endpoint** (`GET /health/live`) every 2 seconds for up to 60 seconds. Exits `0` on success; prints logs and exits `1` on timeout.

To deploy a different branch:

```
CIVILIER_ENV_FILE=/etc/civilier/prod.env bash scripts/deploy.sh
dev
```

Docker Compose Services

The `docker-compose.yml` defines four services:

Service	Image	Purpose	Network port
<code>frontend-build</code>	<code>node:20-alpine</code>	One-shot build step (profile: <code>build</code>)	None
<code>backend</code>	Built from <code>./backend</code>	Node.js API	Internal <code>5000</code>
<code>redis</code>	<code>redis:7-alpine</code>	Cache, JWT blacklist, rate limiting	Internal only
<code>nginx</code>	<code>nginx:1.27-alpine</code>	Reverse proxy + static file server	<code>80</code> (host)

Backend health check (used by Docker and nginx's `depends_on`):

```
node -e "require('http').get('http://localhost:5000/health/live', ...)"
```

Interval: 30s, Timeout: 10s, Retries: 3, Start period: 40s.

Redis health check:

```
redis-cli -a "$REDIS_PASSWORD" --no-auth-warning ping | grep PONG
```

Named volumes:

Volume	Contents
<code>redis_data</code>	Redis AOF persistence
<code>frontend_dist</code>	Built frontend assets (shared between <code>frontend-build</code> and <code>nginx</code>)
<code>backend_uploads</code>	Ticket attachments and other user uploads
<code>frontend_node_modules</code>	<code>node_modules</code> cache for frontend builds

GitHub Actions CI/CD

Pushes to `dev` and pull requests run checks only. Pushes to `main` run checks **and** deploy automatically via AWS SSM.

Add these four repository secrets in **Settings → Secrets and variables → Actions**:

Secret	Description
<code>AWS_ACCESS_KEY_ID</code>	IAM access key for the deployment user
<code>AWS_SECRET_ACCESS_KEY</code>	IAM secret key for the deployment user
<code>AWS_REGION</code>	AWS region, e.g. <code>ap-south-1</code>
<code>EC2_INSTANCE_ID</code>	The target EC2 instance ID, e.g. <code>i-0abc123def456</code>

The IAM user needs only the `ssm:SendCommand` permission on the target instance. It does not need EC2 full access.

Smoke Tests

Run these immediately after every deployment to confirm the stack is healthy:

```
# 1. Confirm all containers are running
docker compose ps

# 2. Public liveness probe (no token needed)
curl http://localhost/health/live

# 3. Readiness probe – checks DB, Redis, and worker
set -a && source /etc/civilier/prod.env && set +a
curl -H "X-Health-Token: $HEALTH_TOKEN"
http://localhost/health/ready
```

A healthy readiness response looks like:

```
{
  "status": "ok",
  "details": { "db": "ok", "redis": "ok", "worker": "ok" }
}
```

After confirming the API is healthy, verify the following in the browser:

- Login and logout
- Socket-based updates (ticket chat)

- Approval workflow transitions
 - Purchase order creation
 - Reports loading
-

Common Pitfalls & Troubleshooting

Deploy script exits with "Missing required value in env file"

One of the seven required keys is missing or blank in `/etc/civilier/prod.env`. The error message names the specific key. Add the value and re-run.

Health check times out after deploy

The backend container did not become healthy within 60 seconds. Run `docker compose logs --tail=100 backend` to check for startup errors. Common causes: DB connection string wrong, RDS security group not allowing port 1433 from the EC2 security group, or migrations failing.

Redis container fails to start

`REDIS_PASSWORD` is not set or is empty. The Redis command uses `${REDIS_PASSWORD:?REDIS_PASSWORD is required}` which causes an immediate exit if the variable is unset.

Frontend shows stale content after deploy

The `frontend-build` container did not run. Confirm the deploy script ran with `--profile build` (this happens automatically via `docker compose --profile build run --rm frontend-build`). Check the `frontend_dist` volume contains recent files.

Cannot SSH into EC2

This is expected — no inbound SSH is configured. Use AWS Systems Manager Session Manager: `aws ssm start-session --target i-0yourinstanceid`.

Notes

- The `REDIS_HOST` value in the production env file must be `redis` — the Docker Compose service name — not `localhost`. Docker's internal DNS resolves service names on the `civilier-network` bridge network.

- `DB_TRUST_SERVER_CERTIFICATE` must be `false` in production. Set it to `true` only in local development against a self-signed certificate.
- The deploy script uses `git pull --ff-only`. If the branch history has diverged (e.g. a force push), the pull will fail. Resolve the divergence manually before re-running the deploy.

Document Numbering System

Type: Reference + Explanation

CivilierERP uses a unified document numbering system across all transactional modules — Purchase Orders, Work Orders, GRNs, Expense Bookings, Payments, Material Issues, and more. Every document number is unique across the entire database, race-safe under concurrent inserts, and carries its own lineage (parent document, root Expense Booking) directly in the number string.

The system lives in `backend/utils/docNumberLock.js` and is backed by two database tables: `dbo.TypeOfDoc` (configuration) and `dbo.DocNumberSequence` (the lock ledger).

How the Lock Works

`lockNextDocNumber()` uses an optimistic locking pattern rather than a database transaction or an application-level mutex:

1. Query `DocNumberSequence` and the target table to find the current maximum serial number.
2. Build a candidate document number at `maxSerial + 1`.
3. **INSERT** the candidate into `DocNumberSequence`. This table has a `UNIQUE` constraint on `DocNo`.
4. If the INSERT succeeds, return the document number.
5. If the INSERT fails with a uniqueness violation (a concurrent request grabbed the same number), re-query the maximum, increment, and retry up to 5 times.

Critical: Always pass the connection `pool` directly to `lockNextDocNumber` — never pass a transaction object. The INSERT into `DocNumberSequence` must commit independently so the `UNIQUE` constraint can detect collisions from other connections.

Three-Tier Number Format

The system supports three document number formats, called tiers. The tier for any given document type is determined purely by which columns are populated on its `dbo.TypeOfDoc` row. Existing pre-migration rows continue to work without any data changes.

Tier 1 — Project-Scoped (migration 046+)

Condition: `TypeOfDoc.ModuleCode IS NOT NULL AND TypeOfDoc.ProjectCode IS NOT NULL AND TypeOfDoc.FinYearReset = 1`

Format: `{ProjectCode}-{ModuleCode}/{Serial}/{FinYear}`

Example: `GC-WO/0001/26-27`

Counter scope: Resets each Indian financial year (April–March), scoped per `(DocNoPrefix, FinYear)`.

This tier is for project-specific document types. `ProjectCode` is a snapshot of `enterprise.short_name` captured when the `TypeOfDoc` row is created, so number generation never needs a runtime join to the enterprise table.

Tier 2 — Dash Format (migration 035)

Condition: `TypeOfDoc.DocNoPrefix IS NOT NULL` (and not Tier 1)

Format: `{DocNoPrefix}-{CalendarYear}-{Serial}`

Example: `ExB-PO-GRN-2026-00008`

Counter scope: Resets each calendar year, scoped per `(DocNoPrefix, DocYear)`.

This is the standard format for most module-level document types introduced in migration 035. `DocNoPrefix` values are seeded by the migration (see the [Seeded TypeOfDoc Rows](#) section below).

Tier 3 — Legacy Slash Format (pre-migration 035)

Condition: `TypeOfDoc.DocNoPrefix IS NULL`

Format: `{FullPrefix}{Serial}/{FinYear}`

Example: `CI/WO/000042/2024-25`

Counter scope: Global — never resets. The serial increments indefinitely.

This tier exists for backward compatibility. Legacy rows that pre-date migration 035 have `DocNoPrefix = NULL` and continue to use this format unchanged.

Tier Detection Logic

The `detectTier()` function evaluates the `TypeOfDoc` row in order:

```
if (ModuleCode && ProjectCode && FinYearReset == true) → "new-  
project" (Tier 1)  
else if (DocNoPrefix) → "new-  
dash" (Tier 2)  
else →  
"legacy" (Tier 3)
```

Database Schema

`dbo.TypeOfDoc`

Configuration table. One row per document type. Migrations 035 and 046 added columns to this table; all additions are guarded with `IF NOT EXISTS`.

Column	Type	Added by	Purpose
<code>TypeOfDocId</code>	<code>INT</code> (PK)	Pre-existing	Primary key
<code>Prefix</code>	<code>NVARCHAR</code>	Pre-existing	Short prefix (legacy use)
<code>FullPrefix</code>	<code>NVARCHAR</code>	Pre-existing	Full prefix with slashes (Tier 3)
<code>StartingDocNo</code>	<code>INT</code>	Pre-existing	First serial number to use (default <code>1</code>)
<code>DocNoPrefix</code>	<code>NVARCHAR(30)</code>	Migration 035	Dash-format prefix (Tier 2). <code>NULL</code> = Tier 3.
<code>DocNoPadding</code>	<code>INT</code>	Migration 035	Serial zero-padding width (default <code>5</code>)
<code>ModuleCode</code>	<code>NVARCHAR(20)</code>	Migration 046	Module code for Tier 1 number (e.g. <code>"W0"</code>)
<code>ProjectCode</code>	<code>NVARCHAR(20)</code>	Migration 046	Project short name for Tier 1 (e.g. <code>"GC"</code>)

Column	Type	Added by	Purpose
FinYearReset	BIT	Migration 046	1 = reset counter each fin year (Tier 1). Default 1.
links_to	NVARCHAR	Pre-existing	Comma-separated module names — used by the frontend to filter doc types per module
IsActive	BIT	Pre-existing	0 rows are excluded from all queries

Unique index: `UX_TypeOfDoc_DocNoPrefix` — enforces that each `DocNoPrefix` value is used by at most one active row.

`dbo.DocNumberSequence`

The lock ledger. One row per issued document number. The `UNIQUE` constraint on `DocNo` is the mechanism that makes concurrent collision detection work.

Column	Type	Added by	Purpose
TypeOfDocId	INT (FK)	Pre-existing	Links to TypeOfDoc
DocNo	NVARCHAR(100)	Pre-existing	The issued document number (UNIQUE)
DocNoPrefix	NVARCHAR(50)	Migration 035	Effective prefix for this row
DocYear	SMALLINT	Migration 035	Calendar year (Tier 2 only)
DocSerial	INT	Migration 035	The numeric serial within the scope
RecordId	INT	Migration 035	FK to the actual record; stamped after INSERT by <code>backPatchRecordId()</code>
TableName	NVARCHAR(100)	Pre-existing	Name of the target table
IssuedBy	NVARCHAR(200)	Pre-existing	User email from <code>req.user.email</code>
FinYear	NVARCHAR(10)	Migration 046	Financial year string (Tier 1 only, e.g. "26-27")

Indexes:

- `UQ_DocNumberSequence_DocNo` – unique constraint, the collision-detection mechanism
- `IX_DocNumberSequence_PrefixYear` – on `(DocNoPrefix, DocYear)` for Tier 2 MAX queries
- `IX_DocNumberSequence_ModuleFinYear` – on `(DocNoPrefix, FinYear)` for Tier 1 MAX queries

`dbo.vw_DocLineage`

A reporting view created by migration 035 that joins `DocNumberSequence` with `ParentDocNo` and `RootExBDocNo` columns from GRNs, Material Issues, Payments, and Expense Bookings. Use this view to trace the lineage chain of any document number.

Seeded TypeOfDoc Rows

Migration 035 seeds the standard dash-format rows. Migration 046 adds `BOQ` and `WO_PO`.

<code>DocNoPrefix</code>	Description	<code>ModuleCode</code>	<code>links_to</code>
<code>PO</code>	Purchase Order	<code>PO</code>	<code>Purchase Order</code>
<code>WO</code>	Work Order	<code>WO</code>	<code>Work Order</code>
<code>GRN</code>	Goods Received Note	<code>GRN</code>	<code>GRN</code>
<code>PAY</code>	Payment (Outgoing)	<code>PAY</code>	<code>Payment</code>
<code>REC</code>	Received Payment	<code>RECP</code>	<code>Received Payment</code>
<code>ISS</code>	Material / Stock Issue	<code>MIS</code>	<code>Material Issue</code>
<code>ExB</code>	Expense Booking	<code>ExB</code>	<code>Expense Booking</code>
<code>ExB-PO</code>	Expense Booking — Purchase Order	<code>ExB</code>	<code>Purchase Order, Expense Booking</code>
<code>ExB-WO</code>	Expense Booking — Work Order	<code>ExB</code>	<code>Work Order, Work Order PO</code>
<code>ExB-GRN</code>	Expense Booking — GRN (direct)	<code>ExB</code>	<code>GRN, Expense Booking</code>
<code>ExB-PO-GRN</code>	Expense Booking — GRN against ExB-PO	<code>ExB</code>	<code>GRN, Expense Booking</code>
<code>ExB-WO-GRN</code>	Expense Booking — GRN against ExB-WO	<code>ExB</code>	<code>GRN, Expense Booking</code>
<code>ExB-PAY</code>	Expense Booking — Outgoing Payment	<code>ExB</code>	<code>Payment, Expense Booking</code>

DocNoPrefix	Description	ModuleCode	links_to
ExB-ISS	Expense Booking — Material Issue	ExB	Material Issue, Expense Booking
BOQ	Bill of Quantities	BOQ	BOQ
WO_PO	Work Order for Materials	WO_PO	Work Order PO

Public API

```
lockNextDocNumber(pool, sql, opts) → Promise<string>
```

Generates and atomically locks the next document number. Call this during the INSERT handler for a new document, before the main table INSERT.

Parameters:

Parameter	Type	Required	Description
pool	mssql.ConnectionPool	Yes	Direct pool — not transaction
sql	mssql	Yes	The mssql module
opts.docTypeId	number	Yes	TypeOfDoc.TypeOfDoc
opts.finYear	string	Tier 1 & 3	Financial year, e.g. "27". Auto-derived today for Tier 1 i
opts.tableName	string	Yes	Target table name "WorkOrderHeads
opts.docNoColumn	string	No	Column holding the number. Defaults "DocNo".
opts.issuedBy	string	No	User email, stored DocNumberSequenc
opts.parentDocNo	string	No	Parent document's stored for lineage

Returns: The locked document number string, e.g. "GC-WO/0001/26-27".

Example — Work Order creation:

```

const { lockNextDocNumber, resolveDocTypeId, backPatchRecordId }
= require('../utils/docNumberLock');

// In your POST /api/work-orders handler:
const docTypeId = await resolveDocTypeId(pool, sql, 'W0');

const docNo = await lockNextDocNumber(pool, sql, {
  docTypeId,
  finYear: '26-27',
  tableName: 'WorkOrderHeader',
  issuedBy: req.user?.email,
  parentDocNo: req.body.parentPoNo ?? null,
});

// INSERT the work order row
const result = await pool.request()
  .input('DocNo', sql.NVarChar(100), docNo)
  /* ... other inputs ... */
  .query(`INSERT INTO dbo.WorkOrderHeader (DocNo, ...) OUTPUT
INSERTED.WOId VALUES (@DocNo, ...)`);

const newId = result.recordset[0].WOId;

// Stamp the new RecordId back into DocNumberSequence
await backPatchRecordId(pool, sql, docNo, 'WorkOrderHeader',
newId);

```

`resolveDocTypeId(pool, sql, prefix)` → `Promise<number>`

Resolves a `TypeOfDocId` from a canonical `DocNoPrefix` string. Use this when you know the prefix string but not the ID.

```

const docTypeId = await resolveDocTypeId(pool, sql, 'ExB-P0-GRN');
// → e.g. 11

```

Throws if no active row is found for the given prefix.

`resolveGRNPrefix(parentDocNo)` → `string`

Determines the correct `DocNoPrefix` for a new GRN based on the parent document's number. Call this before `resolveDocTypeId` when creating a GRN.

Decision tree:

Parent DocNo pattern	Resulting GRN prefix
<code>GC-WO/0012/26-27</code> (Tier 1, any project)	<code>GC-GRN</code> (project code + <code>-GRN</code>)
Starts with <code>ExB-PO-</code>	<code>ExB-PO-GRN</code>
Starts with <code>ExB-WO-</code>	<code>ExB-WO-GRN</code>
Starts with <code>ExB-</code>	<code>ExB-GRN</code>
Any other / <code>null</code>	<code>GRN</code>

```
const { resolveGRNPrefix, resolveDocTypeId, lockNextDocNumber } =
  require('../utils/docNumberLock');

const grnPrefix = resolveGRNPrefix(req.body.parentDocNo); //
  e.g. "ExB-PO-GRN"
const docTypeId = await resolveDocTypeId(pool, sql, grnPrefix);
const docNo = await lockNextDocNumber(pool, sql, {
  docTypeId, tableName: 'GoodsReceiptNotes', ... });
```

`backPatchRecordId(pool, sql, docNo, tableName, recordId)` → `Promise<void>`

Stamps the newly inserted record's primary key back into `DocNumberSequence.RecordId`. Call this immediately after every INSERT that uses a locked document number.

For `ExpenseBooking`, it also keeps `ExpenseBooking.RootExBDocNo` in sync with the document's own `EDocNo`.

```
await backPatchRecordId(pool, sql, docNo, 'GoodsReceiptNotes',
  newGrnId);
```

Parameters:

Parameter	Type	Description
<code>docNo</code>	<code>string</code>	The locked document number
<code>tableName</code>	<code>string</code>	The target table name
<code>recordId</code>	<code>number</code>	The primary key of the new record

Silently returns without doing anything if `docNo` or `recordId` is falsy.

```
previewNextDocNumber(pool, sql, docTypeId,
finYear?) → Promise<object>
```

Reads the next likely document number without acquiring a lock. Use this for UI previews only — never treat the result as a guaranteed number.

Unlike `lockNextDocNumber`, this function uses a single SQL round-trip (a join between `TypeOfDoc` and `DocNumberSequence`) rather than two sequential queries.

Returns:

```
{
  nextDocNo: "ExB-PO-GRN-2026-00009", // the preview string
  prefix:    "ExB-PO-GRN",           // the effective prefix
  nextSeq:   9,                      // the next serial number
  finYear:   null                    // fin year (Tier 1 only,
else null)
}
```

Backend route: `GET /api/document-type/:id/next-number?finYear=26-27`

```
currentFinYear(date?) → string
```

Derives the Indian financial year string for a given date (defaults to today). April–March cycle.

```
currentFinYear(new Date('2026-05-01')) // → "26-27"
currentFinYear(new Date('2026-01-15')) // → "25-26"
```

Exported for testing; route handlers that need the current fin year should use this rather than re-implementing the April–March logic.

DocNumberPreview Component

File: `src/pages/material/ExpenseBooking/DocNumberPreview.tsx`

A controlled React component that renders a document type selector and a live document number preview. The component is stateless with respect to the selected document number — it calls `onSelect` with the new value and lets the parent own the state.

Props

Prop	Type	Required	Description
<code>module</code>	<code>DocModule</code>	No	Filters the dropdown to doc types tagged with this module code.
<code>finYear</code>	<code>string</code>	No	Financial year string (e.g. <code>"26-27"</code>). Pass from <code>FinYearContext</code> . Required for Tier 1 and Tier 3 doc types.
<code>selectedDocTypeId</code>	<code>number</code> <code>null</code>	Yes	The currently selected <code>TypeOfDocId</code> . Owned by the parent.
<code>onSelect</code>	<code>(id: number null, preview: string) => void</code>	Yes	Called when the user picks a type or when a preview refreshes. The parent stores both values.
<code>preview</code>	<code>string</code>	Yes	The current preview string, owned and passed in by the parent.
<code>refreshTrigger</code>	<code>number</code>	No	Increment this to force a preview refresh without changing the selected type (e.g. after a save).
<code>readOnly</code>	<code>boolean</code>	No	When <code>true</code> (edit mode), suppresses auto-fetching of a new doc number and disables the dropdown. The parent's saved doc number is shown as-is.

DocModule type — valid values for the **module** prop:

```
"PO" | "WO" | "WO_PO" | "GRN" | "BOQ" | "EB" | "MIS" | "PAY" |  
"RECP" | "DN" | "WD" | "MR" | "ISS"
```

Behaviour

On mount / module change: Fetches the list of doc types filtered by the **module** prop via `GET /api/document-type?module=WO`. Auto-selects and previews the first result if no **selectedDocTypeId** is set and **readOnly** is false.

Special case for `module="WO"`: The component filters out doc types whose prefix starts with `ExB-` or `WO-PO` / `WO_PO`. These share the same **links_to** tag but belong to separate workflows.

On `selectedDocTypeId` / `finYear` / `refreshTrigger` change: Re-fetches the preview via `GET /api/document-type/:id/next-number`. Skipped in **readOnly** mode.

Edit mode fallback: In **readOnly** mode, if the saved **selectedDocTypeId** is not present in the module-filtered list (because the doc type was created before **links_to** tagging), the component fetches all doc types and injects the matching one into the list.

Loading states: The component shows `"Generating..."` while an initial fetch is in flight and `"Refreshing..."` during subsequent preview refreshes. Both states spin the refresh icon.

Exported Standalone Helpers

The component file also exports two functions for use in pages that need to call the API directly without rendering the component:

```
// Fetch a preview for a given doc type  
fetchNextDocNumber(docTypeId: number, finYear?: string):  
Promise<string>  
  
// Fetch the filtered list of doc types for a module  
fetchDocTypes(module?: DocModule | string): Promise<DocType[]>
```

Usage Example

```

import { DocNumberPreview } from
'@/pages/material/ExpenseBooking/DocNumberPreview';
import { useFinYear } from '@/contexts/FinYearContext';

function NewPurchaseOrderForm() {
  const { finYear } = useFinYear();
  const [docTypeId, setDocTypeId] = useState<number | null>
(null);
  const [docNo, setDocNo] = useState('');
  const [refreshTrigger, setRefreshTrigger] = useState(0);

  function handleSelect(id: number | null, preview: string) {
    setDocTypeId(id);
    setDocNo(preview);
  }

  async function handleSave() {
    await savePurchaseOrder({ docNo, docTypeId, /* ... */ });
    // Force a fresh preview number for the next document
    setRefreshTrigger(t => t + 1);
  }

  return (
    <
      <DocNumberPreview
        module="PO"
        finYear={finYear}
        selectedDocTypeId={docTypeId}
        onSelect={handleSelect}
        preview={docNo}
        refreshTrigger={refreshTrigger}
      />
      <button onClick={handleSave}>Save</button>
    </>
  );
}

```

Common Pitfalls & Troubleshooting

`lockNextDocNumber()` throws "Selected document type not found or inactive"

The `TypeOfDocId` passed does not exist or has `IsActive = 0`. Verify the row exists in `dbo.TypeOfDoc` and run migration 035 if the table is missing expected prefix rows.

`resolveDocTypeId()` throws for a prefix that should exist

The seeded row may not have been created because the migration ran before `dbo.Entry_Type` was populated. The migration aborts with a `RAISERROR` if `Entry_Type` is empty. Re-run migration 035 after seeding `Entry_Type`.

Document numbers are duplicating across financial years (Tier 1)

`finYear` was not passed to `lockNextDocNumber()` and the auto-derivation from today's date is producing the wrong year — for example, calling the function in March for a document that belongs to the previous financial year. Always pass the explicit `finYear` from the user's active financial year context rather than relying on auto-derivation.

`backPatchRecordId()` silently does nothing

Either `docNo` or `recordId` is falsy (zero, null, or undefined). Check that your INSERT query returns the new primary key via `OUTPUT INSERTED.<PK>` and that you're passing the correct column name.

Preview shows a number that is already taken by the time the form saves

This is expected. `previewNextDocNumber()` is a read-only estimate with no lock. Under concurrent usage, two users can see the same preview. The actual number is locked only when `lockNextDocNumber()` runs during the save. The collision-retry logic (up to 5 attempts) handles this transparently.

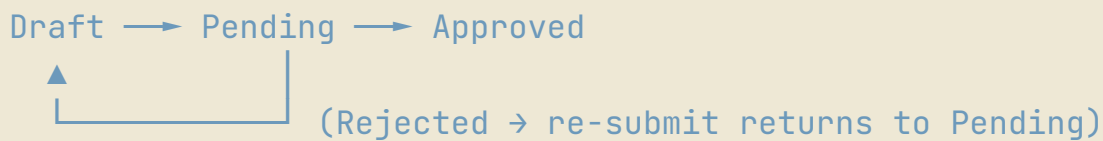
Approval Workflow Engine

Type: Reference + Explanation

The approval workflow engine enforces multi-level approval gating across all transactional modules in CivilierERP. It controls status progression (Draft → Pending → Approved or Rejected), prevents edits to in-flight records, and writes a complete audit trail for every status transition. The engine lives in `backend/services/approvalService.js` and is consumed by ten module route files and the frontend `ApprovalActions` component.

How It Works

Every transactional record moves through a linear status lifecycle:



Draft is the initial state on creation. Most modules auto-submit new records to **Pending** immediately after the INSERT, so users never see the Draft state in normal flow. **Pending** means the record is awaiting approval and cannot be edited. **Approved** is the terminal success state. **Rejected** returns the record to an editable state so it can be corrected and re-submitted.

Multi-level approval works by counting how many **Approved** audit log entries exist for a record. Each call to `transition(... "Approved")` increments the level count. The record's status column only flips to **"Approved"** once the level count reaches the total configured in `ApprovalWorkflows.Levels`. Until then, the record remains **"Pending"** and additional approval calls continue incrementing the level.

Database Schema

`dbo.ApprovalWorkflows`

Stores workflow configuration per module. Created by migration 022.

Column	Type	Description
<code>Id</code>	<code>INT</code> (PK)	Primary key
<code>Name</code>	<code>NVARCHAR(100)</code>	Human-readable workflow name
<code>Module</code>	<code>NVARCHAR(100)</code>	Module slug — must match a key in <code>MODULE_MAP</code> (e.g. <code>"purchase-orders"</code>)
<code>Levels</code>	<code>INT</code>	Number of approval levels required. Default <code>1</code> .
<code>Approvers</code>	<code>NVARCHAR(500)</code>	Comma-separated approver emails (informational — not enforced by the engine)
<code>Status</code>	<code>NVARCHAR(20)</code>	<code>"Active"</code> or <code>"Inactive"</code>
<code>Description</code>	<code>NVARCHAR(500)</code>	Optional notes
<code>CreatedBy</code> / <code>UpdatedBy</code>	<code>NVARCHAR(100)</code>	User name of last writer

Column	Type	Description
CreatedAt / UpdatedAt	DATETIME2	Timestamps

Index: `IX_ApprovalWorkflows_ModuleStatus` on `(Module, Status)` – used by `getWorkflow()` on every approval action.

Only the most recently created `Active` workflow for a module is used. If no active workflow exists for a module, the engine defaults to a single-level approval (`Levels = 1`).

`dbo.ApprovalAuditLog`

Immutable audit trail. One row per status transition per record. Created by migration 015a.

Column	Type	Description
<code>Id</code>	<code>INT</code> (PK)	Primary key
<code>TableName</code>	<code>NVARCHAR(100)</code>	Target table name without <code>dbo.</code> prefix (e.g. <code>"PurchaseOrders"</code>)
<code>RecordId</code>	<code>INT</code>	Primary key value of the record being acted on
<code>Level</code>	<code>INT</code>	Approval level. <code>0</code> = submit action. <code>1</code> , <code>2</code> , ... = sequential approval levels.
<code>Role</code>	<code>NVARCHAR(100)</code>	Role of the acting user
<code>ApproverEmail</code>	<code>NVARCHAR(200)</code>	Email of the acting user
<code>ActionStatus</code>	<code>NVARCHAR(50)</code>	<code>"Pending"</code> (submit), <code>"Approved"</code> , or <code>"Rejected"</code>
<code>Note</code>	<code>NVARCHAR(500)</code>	Optional rejection note supplied by the approver
<code>ActionAt</code>	<code>DATETIME2(3)</code>	Timestamp of the action

Index: `IX_ApprovalAuditLog_Record` on `(TableName, RecordId, Level)` – used by `getApprovedLevelCount()`.

MODULE_MAP

`MODULE_MAP` maps module slugs to their target SQL table, primary key column, and status column. The engine uses this map for all reads and

writes to module tables.

Module slug	Table	PK column	Status column
"expense-booking"	dbo.ExpenseBooking	Eid	EStatus
"purchase-orders"	dbo.PurchaseOrders	PurchaseOrderID	Status
"work-orders"	dbo.WorkOrderHeader	Id	Status
"boq"	dbo.BOQ	BoqID	Status
"work-done"	dbo.WorkDone	ID	Status
"grn"	dbo.GoodsReceiptNotes	GRNID	Status
"goods-receipt"	dbo.GoodsReceiptNotes	GRNID	Status
"payments"	dbo.NewPayment	PPaymentID	Status
"material-requests"	dbo.MaterialRequests	MRId	Status
"material-issues"	dbo.MaterialIssues	IssueId	Status

"grn" and "goods-receipt" are aliases — both point to `dbo.GoodsReceiptNotes`. The GRN route file uses "goods-receipt" when calling `transition()` and `guardEdit()`, and the approval inbox returns "goods-receipt" as the module slug. The "grn" key is retained for backward compatibility.

Passing any slug not in this map to `transition()` or `guardEdit()` throws immediately: "Unknown module: <slug>" .

Service API

```
transition(module, id, targetStatus, userEmail, userRole, note?) → Promise<object>
```

The main entry point. Validates the requested status change, enforces role authorization, manages multi-level progression, updates the record's status column, and writes to `ApprovalAuditLog`.

Parameters:

Parameter	Type	Description
<code>module</code>	<code>string</code>	Module slug from <code>MODULE_MAP</code>
<code>id</code>	<code>number</code>	Record primary key
<code>targetStatus</code>	<code>"Pending" "Approved" "Rejected"</code>	The requested new status
<code>userEmail</code>	<code>string</code>	Calling user's email — stored in the audit log
<code>userRole</code>	<code>string</code>	Calling user's role — used for authorization and stored in the audit log
<code>note</code>	<code>string \ null</code>	Optional rejection reason. Stored in <code>ApprovalAuditLog.Note</code> . Only meaningful for <code>"Rejected"</code> .

Transition rules:

<code>targetStatus</code>	Allowed from	Role required	Behaviour
<code>"Pending"</code>	<code>"Draft"</code> or <code>"Rejected"</code>	Any authenticated user	Sets status to <code>Pending</code> . Writes level <code>0</code> audit entry.
<code>"Approved"</code>	<code>"Pending"</code>	<code>admin</code> , <code>super_admin</code> , or <code>dba</code>	Increments the approved-level count. If count reaches <code>workflow.Levels</code> , sets status to <code>Approved</code> . Otherwise stays <code>Pending</code> .
<code>"Rejected"</code>	<code>"Pending"</code>	<code>admin</code> , <code>super_admin</code> , or <code>dba</code>	Sets status to <code>Rejected</code> . Writes level <code>0</code> audit entry with note.

Return value:

For a submit (`"Pending"`):

```
{ "newStatus": "Pending" }
```

For a full approval (level count reaches total):

```
{ "newStatus": "Approved", "level": 2, "totalLevels": 2 }
```

For an intermediate approval (more levels remain):

```
{ "newStatus": "Pending", "level": 1, "totalLevels": 2,
  "remainingLevels": 1 }
```

For a rejection:

```
{ "newStatus": "Rejected" }
```

Errors thrown (callers return 400 or 403):

- "Unknown module: <slug>" – slug not in MODULE_MAP
- "Record <id> not found in <module>" – no matching row
- "Cannot submit from status \"<current>\"" – status is not Draft or Rejected
- "Cannot <action> from status \"<current>\"" – approve/reject attempted on non-Pending record
- "You are not authorized to approve or reject records." – role check failed (callers return 403)

```
guardEdit(module, id) → Promise<void>
```

Blocks PUT/edit requests on records that are locked for approval. Call this at the top of every edit handler, before any database writes.

```
const { guardEdit } = require('../services/approvalService');

router.put('/:id', async (req, res) => {
  await guardEdit('purchase-orders', req.params.id);
  // ... proceed with edit
});
```

Throws one of two errors:

- "Cannot edit a record that is pending approval. Reject it first." – status is "Pending"
- "Cannot edit an approved record." – status is "Approved"

The caller is expected to catch these and return `400`. Any other status (`Draft`, `Rejected`) passes through silently.

Modules that enforce `guardEdit`: Purchase Orders, Work Orders, GRNs, B0Q, Work Done, Expense Booking, Material Issues.

`validateApprovalModuleMap(log?)` → `Promise<Array>`

Startup validation. Checks that every module registered in `MODULE_DOC_LINKS` has at least one matching active `TypeOfDoc` row (via the `links_to` column). Called non-fatally from `startServer()` in `server.js` – a failure warns but does not prevent the server from starting.

```
// In server.js – called once at startup
validateApprovalModuleMap(logger).catch((err) => {
  logger.warn({ event: 'APPROVAL_MODULE_VALIDATION_FAILED', err
}, '...');
});
```

Returns an array of `{ module, expectedLink }` objects for any modules without a matching `TypeOfDoc` row. Returns an empty array if everything is correctly configured.

This check exists because `lockNextDocNumber()` resolves doc types by their `links_to` value. If a module has no matching `TypeOfDoc` row, document number generation will fail for that module at runtime.

`getWorkflow(module)` → `Promise<object | null>`

Fetches the most recently created active workflow for a module. Returns `null` if none exists. Used internally by `transition()` to determine the total number of required approval levels.

Per-Module HTTP Routes

Every module that participates in approval exposes three routes following this convention:

Route	Method	Action	Body
<code>/:id/submit</code>	PUT	Transition <code>Draft / Rejected</code> → <code>Pending</code>	None
<code>/:id/approve</code>	PUT	Advance approval level; may set <code>Approved</code>	None
<code>/:id/reject</code>	PUT	Transition <code>Pending</code> → <code>Rejected</code>	<code>{ "note": "...", "..." }</code> (optional)

The `note` field on reject is optional but strongly recommended — it is stored in `ApprovalAuditLog` and surfaced in the approval inbox.

Auto-submit on creation: All ten modules call `transition(module, newId, "Pending", ...)` immediately after a successful INSERT. This means new records never require a manual submit in normal flow. The auto-submit failure is non-fatal (a `console.warn` is emitted) to prevent a document numbering or INSERT success from being lost due to a transient approval engine error.

Re-submit after rejection: Because `Rejected` is included as an allowed source state for the `"Pending"` transition, users can correct a rejected record and re-submit it without admin intervention. The `ApprovalActions` component shows a Submit button when `status ≡ "Rejected"`.

GET `/api/approval-workflows`

Auth: Required (JWT)

Cache: 60 seconds (`approval-workflows` namespace, invalidated on write)

Returns all approval workflows ordered by creation date descending. The `approvers` field is returned as an array (split from the comma-separated string stored in the database).

Example response:

```
[
  {
    "id": 3,
    "name": "Purchase Order – Two Level",
    "module": "purchase-orders",
    "levels": 2,
```

```
    "approvers": ["accounts@company.com",  
    "director@company.com"],  
    "status": "Active",  
    "description": "Requires finance and director sign-off",  
    "createdAt": "2025-05-01T10:22:00.000Z",  
    "createdBy": "admin"  
  }  
]
```

POST /api/approval-workflows

Auth: Required (JWT)

Creates a new approval workflow. Only `name` and `module` are required. If no active workflow already exists for the module, this new workflow takes effect immediately on the next approval action.

Request body:

Field	Type	Required	Description
<code>name</code>	<code>string</code>	Yes	Display name
<code>module</code>	<code>string</code>	Yes	Module slug (must match a <code>MODULE_MAP</code> key)
<code>levels</code>	<code>number</code>	No	Number of approval levels. Default <code>1</code> .
<code>approvers</code>	<code>string[]</code>	No	Approver email list (informational).
<code>status</code>	<code>string</code>	No	<code>"Active"</code> (default) or <code>"Inactive"</code>
<code>description</code>	<code>string</code>	No	Notes

Example:

```
curl -X POST /api/approval-workflows \  
  -H "Authorization: Bearer <token>" \  
  -H "Content-Type: application/json" \  
  -d '{  
    "name": "GRN Two-Level",  
    "module": "goods-receipt",  
    "levels": 2,  
    "approvers": ["warehouse@co.in", "accounts@co.in"],
```

```
"description": "Warehouse lead + accounts sign-off"
}'
```

Response (201):

```
{ "message": "Workflow created" }
```

PUT /api/approval-workflows/:id

Auth: Required (JWT)

Updates an existing workflow. All fields are optional on update. Bumps cache version on success.

PATCH /api/approval-workflows/:id/toggle

Auth: Required (JWT)

Toggles the workflow's **Status** between **"Active"** and **"Inactive"**. Use this to deactivate a multi-level workflow and revert a module to the default single-level behaviour without deleting the configuration.

DELETE /api/approval-workflows/:id

Auth: Required (JWT)

Hard-deletes the workflow row. This does not affect records already in a Pending or Approved state.

GET /api/approval-inbox

Auth: None enforced at the route level (the caller is expected to be authenticated via the frontend)

Query param: **?module=<slug>** (optional) — filters to a single module

Returns all **Pending** records across every module (or just the specified module) as a unified list, ordered by **LastModified DESC**. This is the data source for the Admin Approval Inbox page.

Supported module filter values:

purchase-orders , work-orders , payments , received-payment , goods-receipt , expense-booking , work-done , boq , material-requests , material-issues

Response shape (each item is a normalised row):

Field	Type	Description
Module	string	Module slug (use as the endpoint prefix in ApprovalActions)
ModuleLabel	string	Human-readable label, e.g. "Purchase Order"
RecordId	string	Primary key cast to string
Reference	string	Document number or auto-generated fallback (e.g. "EB#42")
RecordDate	string \ null	Document date
Status	string	Always "Pending" for items returned by this endpoint
SupplierName	string \ null	Supplier or counterparty name where available
Amount	number \ null	Document amount where available
CreatedBy	string \ null	User who created the record
ApprovedBy	string	Email of approver (empty string if not yet approved)
RejectedBy	string	Email of rejecter (empty string if not rejected)
RejectionNote	string	Rejection reason (empty string if none)
LastModified	string \ null	Last update timestamp — used for ORDER BY

Expense Booking filter: Auto-created split-booking records (those with ERemarks LIKE 'Auto-created for remaining items from GRN%') are excluded from the inbox. These are system-generated and should not require a separate manual approval.

GET /api/approval-inbox/count

Auth: None enforced at the route level

Returns a single integer: the total count of **Pending** records across all ten modules. Use this to populate notification badges.

```
{ "count": 7 }
```

The count query runs ten scalar subqueries in a single SQL statement. It applies the same Expense Booking auto-created exclusion filter as the main inbox query.

ApprovalActions Component

File: `src/components/ApprovalActions.tsx`

A self-contained React component that renders the Submit, Approve, and Reject buttons on any transactional form. It reads the user's role directly from the JWT in `localStorage` and shows only the buttons the current user is permitted to act on.

Props

Prop	Type	Required	Description
<code>status</code>	<code>string</code> \ <code>null</code> \ <code>undefined</code>	Yes	Current record status
<code>recordId</code>	<code>number</code> \ <code>string</code>	Yes	Record primary key
<code>endpoint</code>	<code>string</code>	Yes	API base path, e.g. <code>"/api/purchase-orders"</code>
<code>onSuccess</code>	<code>(action: "submit" \ "approve" \ "reject") => void</code>	No	Callback fired after a successful action — use to trigger a data refetch
<code>className</code>	<code>string</code>	No	Additional CSS classes for the container
<code>submitOnly</code>	<code>boolean</code>	No	When <code>true</code> , hides Approve/Reject buttons. Use on module list pages where those actions belong only to the Approval Inbox. Default <code>false</code> .

Render Logic

The component is intentionally invisible on most records:

- Returns `null` for `status === "Approved"` or `status === "Fully Received"` – no actions needed on terminal states.
- **Submit button** – shown when `status === "Rejected"`, `"Issued"`, or `"Partially Received"`. Visible to all roles. New records auto-submit on creation, so this button only appears when re-submitting after a rejection.
- **Approve / Reject buttons** – shown when `status === "Pending"` AND the current user has role `admin`, `super_admin`, or `dba`. Hidden when `submitOnly` is `true`.
- Returns `null` if neither condition applies (e.g. status is `"Draft"` or `"Pending"` for a non-approver role).

Reject Flow

Clicking Reject opens a modal dialog prompting for an optional rejection note before the action fires. The note is sent as `{ "note": "..."}` in the PUT request body and stored in `ApprovalAuditLog`.

API Call Pattern

All three actions call `PUT {endpoint}/{recordId}/{action}`. For example, approving Purchase Order 42:

```
PUT /api/purchase-orders/42/approve
Authorization: Bearer <token>
```

Rejecting Work Order 17 with a note:

```
PUT /api/work-orders/17/reject
Authorization: Bearer <token>
Content-Type: application/json

{ "note": "Vendor quote expired – re-raise with updated pricing"
}
```

Usage Example

```
import { ApprovalActions } from '@components/ApprovalActions';
```

```
function PurchaseOrderDetail({ po, refetch }) {
  return (
    <div>
      <h1>{po.PurchaseOrderNo}</h1>
      <p>Status: {po.Status}</p>

      <ApprovalActions
        status={po.Status}
        recordId={po.PurchaseOrderID}
        endpoint="/api/purchase-orders"
        onSuccess={() => refetch()}
      />
    </div>
  );
}
```

Common Pitfalls & Troubleshooting

transition() throws "Cannot submit from status 'Pending'"

The record was already auto-submitted on creation. A double-submit is being attempted. This can happen if the frontend calls the submit route after a successful save that already auto-submitted. Check that the save handler does not manually call `/:id/submit` — auto-submit is handled by the route.

transition() throws "You are not authorized to approve or reject records."

The caller's role is not in `APPROVER_ROLES` (`admin`, `super_admin`, `dba`). The route handler should return `403`. Verify that the user's JWT contains the correct role and that `req.user.role` is being passed — not `req.user.email`.

A record is stuck at "Pending" after being approved once in a two-level workflow

This is correct behaviour. The record stays `"Pending"` until `getApprovedLevelCount()` reaches `workflow.Levels`. A second approver must call the approve route. Check `ApprovalAuditLog` to see which level was last approved: `SELECT * FROM dbo.ApprovalAuditLog WHERE TableName = 'PurchaseOrders' AND RecordId = <id>`.

`guardEdit()` blocks an edit with "Cannot edit a record that is pending approval. Reject it first."

The record's status is `"Pending"`. To edit it, an approver must first reject it via `PUT /:id/reject`. After rejection, the record returns to an editable state.

`validateApprovalModuleMap()` warns at startup about a missing `TypeOfDoc` link

A module in `MODULE_DOC_LINKS` has no matching active `TypeOfDoc` row with the expected `links_to` value. Run migrations 035 and 046, or manually insert the missing `TypeOfDoc` row. The approval engine continues to function, but `DocNumberPreview` will not load doc types for the affected module.

The approval inbox returns duplicate records

The inbox query uses `UNION ALL`. If a module's status column is inconsistently populated (e.g. some rows have `NULL` defaulted to `'Pending'` via `ISNULL(Status, 'Pending')`), more records than expected may appear. The Material Issues query includes this pattern — check that `Status` is explicitly set on all rows for that module.